

Content Guide



Tools Overview

Providing content for a modern game engine is a complicated task involving many sophisticated tools: tools for modeling characters, tools for making world architecture, tools for creating textures, tools for making music, and tools for many other tasks. Jupiter provides state-of-the-art tools to content creators and multiple ways to design a given asset or effect. This document describes the functions of the many tools included with Jupiter and also helps you understand how they work so that you can pick the proper ones for the proper tasks.

This section is intended for use by content creators, but other project personnel are encouraged to read the introductions to level design and modeling to develop a basic understanding of how they work. Likewise, non-programmers may find certain sections of the *Programming* section helpful to understanding the overall architecture and capabilities of Jupiter.

This section contains the following topics on using the tools provided with the Jupiter system:

- [About Tools and Plugins](#)
- [Model and Animation Exporters](#)
- [Vertex-Animated Models](#)
- [Content Considerations](#)
- [Quick Start: Viewing your Model in Jupiter](#)
- [Types of Geometry](#)
- [LithTech Resource Types](#)
- [Tutorial: Adding an Attachment to your Model](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006. All Rights Reserved.

Content Guide



About Tools and Plugins

Jupiter includes two primary editing tools, and a variety of additional tools and plugins for creating games that work with the Jupiter engine. This section provides a brief description of each of the tools provided in the Jupiter installation. Read the following topics in this section to familiarize yourself with the tools and plugins included in your installation.

- [Primary Editing Tools](#)
- [Additional Editing Tools](#)
 - [Plugins](#)

Primary Editing Tools

The DEdit and ModelEdit applications are the primary tools used to develop levels and prepare models for your game.



DEdit is the integration point for most game resources, as well as the final assembly point for your levels. Using DEdit, you build the world your players will explore and experience when they play your game. You can use DEdit to build your buildings, skies, doors, windows and other objects, or assemble these things from parts built in a supported 3D package. You can also apply the level's textures to add more shape to the world if you have not done so already. You can create, preview and refine the level's lighting. You can add game objects, such as creatures, characters, tools, and weapons, to the world to make it interactive. Finally, you use DEdit to tune your world for the best performance possible.



ModelEdit enables you to view and edit models and animations created in 3D Studio Max or Maya. This tool is vital in ensuring that models are optimized for inclusion in a Jupiter game. Artists can also use ModelEdit to define specific properties, assign texture IDs, and manipulate model animation, among other functions. ModelEdit ensures models are optimized to interact correctly with environments and other objects.

Additional Tools

In addition to DEdit and ModelEdit, Jupiter is supplemented by a large group of smaller tools. These include converters from outside or older file formats. All of the files listed below are executable files (.exe files) and are located in the following directory:

<Installation Root>\Development\Tools\

ButeCheck

ButeCheck.exe checks for errors in .BUT files.

ButeEdit

ButeEdit.exe allows you to view and edit .BUT files graphically. As an example, DEdit uses one common .BUT file (Classhlp.BUT) to display helpful tips about objects and properties. Although it's usually easier to create .BUT files (which look like Windows .INI files in structure) by hand in Notepad, once they get large enough they're easier to handle in a graphical interface like ButeEdit's.

dtxutil

Dtxutil.exe runs from the command line and provides advanced, but rarely used, .DTX modification functions.

FXED ^{FX}

FXED.exe is an editor that artists can use to create spectacular special effects. FXED removes the need for programmer interaction (in creating effects) and therefore speeds up development.

HeightMapToNormalMap



HeightMapToNormalMap.exe converts a texture with height map data—0 to 255 height value in any of the red, green, blue, or alpha channels—to a texture with normal map data. The default is alpha channel. The normal map is used with dot3 bump mapping. The normal map specifies x, y, z normal values in the r, g, b channels of the output texture. There is an input scale value that scales the input height values. The normal vectors are calculated at each texel by triangulating the scaled height values, crossing two legs of the triangle, and normalizing.

LithRez

LithRez.exe creates the .REZ files that store all of your project's game resources. This file is not compressed, but it saves some space and adds convenience during installs by reducing the number of files you must copy to the player's machine from hundreds or thousands down to a single one. It greatly increases resource loading times. Running the program with no parameters displays its options.

ltaview

Ltaview.exe allows you to view .LTA files in a tree view.

ltbhead

Ltbhead.exe is a command-line utility that reads an .LTB file and sends the header information to STDOUT.

LTC

LTC.exe compresses .LTA files or to decompress .LTC files. This capability is built into DEdit and ModelEdit.

Model_Packer

Model_Packer.exe is a command line utility that converts .LTA files to the .LTB binary format for output to Direct3D.

pma

Pma.exe converts a premultiplied .TGA file to an unpremultiplied .TGA file. A premultiplied .TGA file has color components scaled by the alpha value ahead of time. Jupiter expects unpremultiplied .TGA files, so images generated by some applications might need to be run through this utility before using them with the engine.

Processor

Processor.exe is used by DEdit to process your worlds into optimized .DAT files.

PropHelpGen

PropHelpGen.exe creates a skeleton .BUT file including all the objects and properties, but with no descriptions.

Recurse_run

Recurse_run.exe is helpful for use in batch processing files with other command-line tools. It recursively runs any specified program on multiple files instead of one.

RenderStyle_Packer



RenderStyle_Packer.exe compiles your render styles for the PC. It supports a fallback mechanism to ensure full compatibility across a wide range of video cards. It can be run from the Render Styles Editor.

RenderStylesEditor



RenderStylesEditor.exe is a tool for developers. It allows you to create render styles using existing game project resources, such as textures. Using the RenderStylesEditor you can map the Render Styles to standardized sample models such as globes, walls, and The Utah Teapot and preview the results in real-time in the tool. When you create a new Render Style, the Render Styles Editor saves it as an .LTB file that is attached to a model in the game code.

RenderStylesViewer



RenderStylesViewer.exe is a tool for content artists. You can use the Render Styles Viewer to preview the new Render Styles created with the Render Styles Editor.

WaveEdit



WaveEdit.exe allows you to edit and optimize your .wav files for the Jupiter system. Using WaveEdit you can control how .wav files are streamed and decompressed during execution of your game.

WinLTC

WinLTC.exe is a Windows version of the LTC.exe tool. Used to compress .LTA files or to decompress .LTC files.

[Top](#)

Plugins

The following plugins provide model, animation, world, and terrain importing and exporting functionality to Maya and 3D Studio Max.

All of the files listed below are located in the following directory:

<Installation Root>\Development\Tools\

LithTechWorldExportOptions.mel

Exporter interface Maya Embedded Language (MEL) script for the Maya Terrain and Brush Exporter and Importer. See the MayaWorldExport30.mll listing below.

MaxModelExport30.dle

.LTA Model Exporter for 3D Studio Max 3.0. Exports models from Max to ModelEdit format. To install the exporter, copy theMaxModelExport30.dle file to the plug-in path used by 3D Studio Max.

MaxModelExport40.dle

.LTA Model Exporter for 3D Studio Max 4.0. See the MaxModelExport30.dle listing above for a description and installation instructions.

MaxWorldExport30.dle

.LTA Model Exporter for 3D Studio Max 3.0. Exports worlds from Max to DEdit format. To install the exporter, copy theMaxWorldExport30.dle file to the plug-in path used by 3D Studio Max.

MaxWorldExport40.dle

.LTA Model Exporter for 3D Studio Max 4.0. See the MaxWorldExport30.dle listing above for a description and installation instructions.

MaxWorldImport30.dli

.LTA World Importer for 3D Studio Max 3.0. This importer plug-in complements the Max Terrain Exporter but is more versatile in that it will import any full .LTA level file into Max. Every brush and terrain object in the level will be imported into Max as an independent mesh, with textures aligned exactly as they were in DEdit.

To install the importer, place the plug-in file MaxWorldImport30.dli within a plug-in path used by 3D Studio Max. After that, the plug-in is accessed through the File menu Import command within 3D Studio Max.

MaxWorldImport40.dli

.LTA World Importer for 3d Studio Max 4.0. See MaxWorldImport30.dli listing above for description and installation instructions.

MayaModelExport30.mll

Maya Model Exporter plug-in file for use with Maya 3.0. This exporter plug-in for Maya allows creation of models and animation within Maya. The plug-in consists of two parts. There is the UI front-end, which consists of a Maya file translator plug-in and the associated UI MEL script.

To install the plug-in, the plug-in file (MayaModelExport30.mll) must be placed within a plug-in path used by Maya. The exporter interface MEL script (LithTechModelExportOptions.mel) must be placed within a script path used by Maya. Finally, the plug-in must be loaded using the Maya Plug-in Manager dialog.

MayaModelExport40.mll

Maya Model Exporter plug-in file for use with Maya 3.0. See MayaModelExport30.mll listing above for description and installation instructions.

MayaWorldExport30.mll

Maya Terrain and Brush Exporter and Importer plug-in file for use with Maya 3.0. This exporter plug-in for Maya allows creation of brushes and terrain within Maya. The plug-in consists of two parts. There is the UI front-end, which consists of a Maya file translator plug-in and the associated UI MEL script.

To install the plug-in, the plug-in file (MayaWorldExport30.mll) must be placed within a plug-in path used by Maya. The exporter interface MEL script (LithTechWorldExportOptions.mel) must be placed within a script path used by Maya. Finally, the plug-in must be loaded using the Maya Plug-in Manager dialog.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



—QUICK START— Viewing Your Model in Jupiter

This section describes how to quickly view one of your models in Jupiter. The following general steps describe how to import and convert models created in Maya or Max and view them in DEdit:

1. [Install the Model Exporter](#)
2. [Export your model](#)
3. [Convert the .LTA File to an .LTB File](#)
4. [Copying Your Resource Files to the Project Folder](#)
5. [Convert your textures in DEdit](#)
6. [Place the Model in the World Using DEdit](#)
7. [Process and run the world through DEdit](#)

Step One—Install the Model Exporter

Jupiter includes a Model Exporter plug-in for both the Max and Maya modeling applications. You must install the plug-in for one of the

applications using the directions below:

To Install for use with MAX

If you use MAX3, then from the \Development\Tools\Plugins\ folder, copy the MaxModelExport30.dle file to the plug-in path used by MAX.

If you use MAX4, then from the \Development\Tools\Plugins\ folder, copy the MaxModelExport40.dle file to the plug-in path used by MAX.

To Install for use with Maya

From the \Development\Tools\Plugins\ folder, copy the MayaModelExport25.mll (or MayaModelExport30.mll if you are using Maya 3) file to the plug-in path used by Maya.

Also from the \Development\Tools\Plugins\ folder, copy the LithTechModelExportOptions.mel file to the scripts path used by Maya, typically \scripts\others.

Step Two—Export Your Model

Using your modeling application, you must export your model to the format that ModelEdit can utilize. Follow the instructions below for the modeling program you use.

To Export from MAX

1. In MAX, open your model file.
2. In the File menu, select Export.
3. Enter a filename and select LithTech Model (*.LTA, *.LTC).
4. Click Save.
5. In the LithTech Model Exporter dialog, name the animation and select the UseMAX Texture Information. In the Base Texture Path directory, browse to the folder where you have stored your texture files in the Targa (.TGA) format. Click Use Path and then click OK.

To Export from Maya

1. In Maya, open your model file.
2. In the Window menu, select **Setting>Preferences** and click the Plug-in Manager.
3. In the check box associated with LithTech Model Exporter, select Load.
4. In the File menu, select Export All.
5. Click the Options button and select File Type as .LTA, name the animation, and click OK.

Step Three—Convert the .LTA File to an .LTB File

In ModelEdit, you must convert the exported .LTA file into a compiled .LTB file DEdit can understand.

To convert the LTA file

1. Open ModelEdit.
2. In the File menu, select Open and find the .LTA file you created in the previous step and then click OK.
3. In the File menu, select Compile for Platform and select PC D3D.
4. In the Save As dialog, provide a filename. In the Save As Type field, select .LTB Files, and then click Save. When the compile is complete, click ENTER to continue.
5. Close ModelEdit.

Step Four—Copying Your Resource Files to the Project Folder

All resource files for your model must reside in a folder in the project area. This project area is \Development\TO2\Game\.

To copy your resource files

- Copy the .LTA and .LTB files you created to the \Chars\ folder.
- Copy the .TGA texture files for your model into the \Chars\ folder.

Step Five—Convert Your Textures in DEdit

To convert your textures

1. Open DEdit.
2. In the File menu, select Open Project. Navigate to the TO2 project located at \Development\TO2\Game\to2.dep.
3. In the Project Window, open the Textures tab. In the upper Textures tab area, open the Chars folder.
4. In the middle of the Textures tab area, right-click and select Import TGA Files. Navigate to your .TGA file(s) in the Chars folder and click OK.

5. In the Project Window, open the Worlds tab, select the `World\Retail\SinglePlayer\` folder and open the world called `c06s02` by double-clicking it.

Step Six—Place the Model in the World Using DEdit

In DEdit, you must place the model in the world. The TO2 development build includes several useful objects, including Prop, which allows the placement of a model.

To place the model in the World

1. In DEdit, press and hold the X key as you position the green marker where you want the object to appear. Release the X key.
 2. Right-click and choose Add then Object.
 3. In the Select Object Type dialog, select Prop, then click OK.
4. In the Project Window, open the Properties tab. You must specify the Filename property. The Filename property is the name of the .LTB file for the model you want to see in the sample. Click on the B (for browse) button next to the Filename field. Browse and find your .LTB model file in `\Development\TO2\Game\Chars`.
5. In the Properties tab, you must also specify the Skin property. The Skin property is the filename for the model's texture. Click on the B (for browse) button next to the Skin field. Browse and find your .DTX texture file in `\Development\TO2\Game\Chars`.

Step Seven—Process and Run your World Through DEdit

To process and run your world you must perform the following three steps. The proceeding text describes each of the following three steps in greater detail.

1. Set up DEdit's Run/Process Tab
2. Process the world
3. Run the world

Step One—Set up DEdit's Run/Process Tab

DEdit can only store the run information for one project at a time. It takes a few steps to configure, so it's best to use it only for your main project.

1. Click Options in the Edit menu.
2. In the DEdit Options dialog, click the Run tab.
3. In the Executable box, enter the complete path to the version of LITHTECH.EXE that your game uses. Use the Browse button to find it. If you have a debug and a release version, you should specify the one you use most frequently here.
4. In the Working directory box, enter the location from which your game is run. Usually, your executable, project directory and config files will all be in one place, and that place is the location of your working directory. If your project's executables and project directory are stored in different locations, consult your programmers for more information on what the correct working directory is for your project.
5. In the Program Arguments box, enter all the needed parameters to run your game. This includes loading the engine .REZ file, the game .REZ file, any config files in addition to AUTOEXEC.CFG, and any supplemental commands specific to your game. DEdit will, if asked, pass a `%WorldName%` parameter that tells the game to load the current level when it's started from DEdit, which is very handy.
6. Click OK.

Step Two—Process the World

1. In DEdit, in the World menu, select Process or click the toolbar button that looks like a brick wall.
2. In the Processing Options dialog, set the Project Directory field. Click the ... button to browse and navigate to the file `\Development\TO2\Game\to2.dep`. Select the file `to2.dep` and click Open.
3. Click OK to begin processing the level.
4. When the Processor is finished, it will show an OK button. Click OK to exit.

Step Three—Run Your World

- o In DEdit, in the World menu, select Run (CTRL+ALT+R), or click the toolbar button that looks like a rocket.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Model and Animation Exporters

The engine supports the basic skinning operators available in the supported modelers. However, all modeling packages have many more

features than the game engine does. Exporting new animated models into an existing .LTA file lets you create a file containing many animations. To do this requires that the hierarchy of the model being exported is identical to that of the existing file. You can import animations from other models with matching skeletons. Each animation must be exported as an .LTA file. In ModelEdit you select multiple .LTA files when you import.

All exporters work similarly, but because of the feature differences of each package, each exporter exerts different restrictions on modelers. The modeler is going to have to become familiar with the limitations of the packages in relation to the LithTech model format. When first starting try making simple models, test them in ModelEdit and a simple game level to see how it all fits. The following applications and utilities are capable of exporting animated models:

- [3D Studio Max](#)
 - [Maya](#)
 - [Exporting Models from Maya](#)
 - [Maya Exporter MEL Commands](#)

3D Studio Max

Models can be skeletally animated using Biped or MAX native bones. In MAX 3.x, the Physique modifier must be used as the deforming modifier, but MAX 4.x also supports the native Skin modifier. It is a good idea to try a test model when trying new animation techniques to make sure the exporter supports those techniques.

Models animated with the Physique modifier must be linearly deformed, which means that you cannot use the muscling features of Physique for any meshes that will be skeletally animated by LithTech. Another important setting is the **Vertex - Link Assignment** option. This option must be set to **Rigid**. Multiple links can still affect the vertex position, but this will ensure that the bones remain rigid, rather than using the default spline-based bones. If **Deformable** is used instead, there will be minor differences between the mesh seen in MAX and the mesh seen in ModelEdit. This option is usually set when a mesh is first bound to a node hierarchy. It may also be modified in the Envelope sub-object mode within the **Active Blending** section of the rollout.

Models animated with the Skin modifier cannot use any of the Deformer Gizmos. This includes the Joint Angle deformer, Bulge Angle deformer, and the Morph Angle deformer.

The MAX model exporter supports specifying extra information to be exported by using the **User Defined Properties** tab of the **Object Properties** dialog. Open this dialog by right clicking on an object and selecting **Properties**. These properties can be used to force nodes to export, create sockets, and assign Render Styles and texture indices. Some of these actions can also be done with prepended strings on object names. Here is an sample list of all the properties (although no real model would have all of these properties set at once):

- Texture0=5
- Texture1=2
- Texture2=3
- Texture3=9
- RenderStyle=2
- ForcedNode=true
 - Socket=true
- VertexAnimated=true

The **TextureN** properties specify the texture indices for the piece (as seen in the Piece Info dialog in ModelEdit.)

The **RenderStyle** property specifies the Render Style index for the piece (as seen in the Piece Info dialog in ModelEdit.)

By default, the exporter will only export nodes that are bound to a physique or skin modifier. All parents of bound nodes will also be exported in order to ensure proper animation. If there is a node that isn't used by the deformer modifier but you would still like it (and its parents) to be exported, specify the **ForcedNode=true** user property. The same result can be achieved by having the object name begin with **n_**.

An object can be turned into a socket by specifying the **Socket=true** user property, or by having the object name begin with **s_**. The specified object should not also be a node used for skeletal animation. The best technique is probably to create a Dummy object and make a bone its parent. The socket object should not be animated other than movement caused by parent object animation.

An object can be specified as being a vertex animated piece by using the **VertexAnimated=true** user property, or by having the object name begin with **d_**.

Textures can optionally be assigned to objects in MAX and show up immediately in ModelEdit. To do this, assign a Standard material to the object and place a Bitmap texture in the diffuse slot of the material. At least one texture index for the object must be set up using the user properties as described above. The texture base path should be entered the same way as described in the [3D Studio MAX Brush Exporter](#) section of this manual, but a brief summary follows:

If you have this texture assigned in MAX:

```
\\%install%\Max Scenes\ProjectModels\textures\wood\pine1.tga
```

And this texture in your LithTech resources:

```
\\%install%\Project\d3drez\textures\wood\pine1.dtx
```

You would enter `\\%install%\Max Scenes\ProjectModels` as your base texture path so that the project relative filename `textures\wood\pin1.dtx` would be stored in the model .LTA file.

Maya

The Maya exporter supports polygon models that use the smooth skinning operation on Maya's native joints. Do not use FFDs or any other deforming method unless you plan to export the piece as vertex animated. Bone scaling (stretching) is lost in the exporter. It is usually best to triangulate your models before exporting, but the exporter will also do this for you if you forget. Everything that is to be exported must have a smooth skinning operation done on it. Remember to load the plug-in from the Window\General Editors\Plug-in Manager dialog.

The Maya model exporter supports specifying extra information to be exported by adding custom attributes to nodes. These custom attributes are added within the Attribute Editor in Maya. These attributes can be used to force nodes to export, create sockets, and assign Render Styles and texture indices. Some of these actions can also be done with prepended strings on object names. Here is a list of all the custom attributes and their types (although no real model would have all of these set at once):

- Texture0 (integer)
- Texture1 (integer)
- Texture2 (integer)
- Texture3 (integer)
- RenderStyle (integer)
- ForcedNode (boolean)
- Socket (boolean)

The **TextureN** attributes specify the texture indices for the piece (as seen in the **Piece Info** dialog in ModelEdit.)

The **RenderStyle** attribute specifies the Render Style index for the piece (as seen in the **Piece Info** dialog in ModelEdit.)

By default, the exporter only exports nodes that are bound to a skin cluster. All parents of bound nodes are also exported in order to ensure proper animation. If there is a node that isn't used by the skin cluster but you would still like it (and its parents) to be exported, create the **ForcedNode** user attribute and set it to TRUE. The same result can be achieved by having the object name begin with **n_**.

An object can be turned into a socket by specifying the Socket attribute and setting it to TRUE, or by having the object name begin with **s_**. The specified object should not also be a node used for skeletal animation. The best technique is probably to create a Locator and make a bone its parent. The socket object should not be animated other than movement caused by parent object animation.

An object can be specified as being a vertex animated piece by having the object name begin with **d_**.

Textures can optionally be assigned to objects in Maya and show up immediately in ModelEdit. To do this, assign a file texture in the color slot of a material and assign it to the piece. At least one texture index for the object must be set up using the custom attributes as described above. The texture base path should be entered the same way as described in the [3D Studio MAX Brush Exporter](#) section of this manual, but a brief summary follows:

If you have this texture assigned in Maya:

`\\%install%\Maya Scenes\ProjectModels\textures\wood\pine1.tga`

And this texture in your LithTech resources:

`\\%install%\Project\d3drez\textures\wood\pine1.dtx`

You would enter `C:\Maya Scenes\ProjectModels` as your base texture path so that the project relative filename `textures\wood\pin1.dtx` is stored in the model .LTA file.

Exporting Models from Maya

The easiest way to use the exporter is to use the File/Export All... and File/Export Selection... menu options in Maya. These will launch a dialog with all the options supported by the exporter except for animation appending.

Optionally, the `littechModelExport` MEL command can also export the model. This supports all the options the UI front-end does, as well as the ability to append animations to existing model files. If you wish to append animations to a model from within Maya, you must use the MEL exporter command directly.

The following list displays the selections from the exporter dialog box followed by the MEL syntax to use if you are using an MEL command.

Animation Name—The name of the animation as it shows up in ModelEdit.

`littechModelExport animation name`

Scale Modifier—The exported model is scaled by the amount entered in this edit box. Defaults to 1.0.

`littechModelExport -scale <float value>`

Ignore Bind Pose—This should only be checked if you encounter problems with unmatched input and output geometry for a skin cluster. If this happens, the first animation to be exported MUST have the object in its bind pose at the first frame. If not, the geometry will NOT animate correctly within ModelEdit or the engine. When checked, the exporter considers the first frame of the animation to be the bind pose. By default this is not checked, and the exporter attempts to automatically find the bind pose information.

```
lithtechModelExport -ignoreBindPose
```

Use Playback Range—When on, the exporter only exports keyframes within the currently selected playback range. Otherwise the exporter tries to find the full range of keyframes and export them all. Defaults to on.

```
lithtechModelExport -usePlaybackRange
```

Use Maya Texture Information—When on, the exporter looks for assigned textures and adds this information to the file for use within ModelEdit. Note that this requires at least one 'TextureN' custom attribute on pieces that you wish to export texture information for. Defaults to off.

```
lithtechModelExport -useTexInfo
```

Base Texture Path—This path is stripped off the front of the full path of all textures that are exported. For example, if your LithTech world has a texture `%install%\project\d3drez\textures\wood\pine1.dtx` and you have an equivalent Maya texture at `%install%\Maya Scenes\textures\wood\pine1.tga`, you would enter `%install%\Maya Scenes` as the base texture path. This will result in `textures\wood\pine1.dtx` being stored in the .LTA file.

```
lithtechModelExport -useTexInfo <base path>
```

Maya Exporter MEL Commands

As noted in the previous topic, the Maya Embedded Language (MEL) command for exporting models to an .LTA file is **lithtechModelExport**. You can use this command in conjunction with the following dash delimited options to export models usable with DEdit.

```
lithtechModelExport [-all]
                    [-usePlaybackRange]
                    [-ignoreBindPose]
                    [-append]
                    [-scale <float value>]
                    [-useTexInfo <base path>]
                    animation name
                    filename
```

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Vertex-Animated Models

LithTech also supports vertex-animated models. Vertex-animated models don't use skeletal animations, but use vertex positions to draw the model. This allows for animations not possible with skeletal animation. LithTech supports vertex-animated models exported from 3D Studio MAX and Maya.

Vertex-animated model files tend to be very large. In vertex animation, every vertex has a position stored for every keyframe. In MAX, you can use any technique to animate the vertices and the exporter writes a keyframe for every frame. In Maya you must skin your model, but it exports only set keyframes, and only the bone scaling effects are preserved. Vertex-animated models can be mixed with skeletally animated models, and attachments to skeletally animated models can be vertex-animated models.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



Content Considerations

When creating content for a game powered by Jupiter, you will be working with the following classes of art and code assets:

Texturing—Textures are the bitmapped graphic image files used to add detail and color to models, world geometry, sprites, and surfaces. Textures are also used as a basis for special effects and as UI elements.

Jupiter uses its own form of texture, the .DTX file. Textures can be up to 32-bit images, although you can obviously choose to make lower-color textures. The .DTX files also contain flags for additional information used by the engine and the game. You can create your textures in Adobe Photoshop or PC Paint Shop Pro and save them as ZSoft PC Paintbrush (.PCX) or Truevision Targa (.TGA) files. You then convert them into Jupiter .DTX files by importing them into DEdit.

Game Objects—Simply put, anything a player interacts with that moves, lights up, shoots, grows, or goes into the player's inventory is an object. In DEdit, you can place any kind of object that your game's code or the engine's code can create. Objects, also referred to as "Entities", must be created by your programmers. The Jupiter game code provides many powerful and useful game objects. These objects are called "game objects" (to distinguish them from the engine objects inside the engine code), and are stored in a file called Object.lto in the same directory as the project's .DEP file.

World Geometry—A Jupiter world is composed of several different classes of geometry, each with a different purpose:

- **Basic brushes**, which are static, are used to build the non-moving parts of a world, such as the walls, floor, or ceiling of a building or dungeon. They also block the engine's line of sight, which is important for performance optimization.
- **Terrain brushes** have far fewer restrictions on their shape than basic brushes. Although they don't block visibility, terrain brushes can be concave shapes or even one-sided, such as an outdoor field with gentle rolling hills.
- **WorldModels** are used for objects that need some of the properties of models (translucency, mobility, destructibility) but need to be closely physically fitted to a place in the world, or need per-polygon collisions. They are used for doors, windows, elevators and moving machinery.
- **Other types** of world geometry exist that the engine uses for specialized purposes such as changing the visibility blocking in an area, specifying the space for triggers and special effects, or displaying the sky.

Lighting—Jupiter provides several lighting modes, including lightmapping on the world, Gouraud and flat shading on the world and on models, and Direct3D hardware lighting on models. Lights can have various shapes and emission characteristics. You can create dynamic lights that light up the world and the models.

Visibility—Engine visibility is a factor that determines how efficiently your world will render and run. Before you begin building your levels, you should understand how the engine decides what is visible and what is not.

Modeling—Use 3D Studio Max or Maya to create models and animations and then prepare them for use in Jupiter using the ModelEdit tool. The Max and Maya exporters included with Jupiter support many features of Max and Maya.

Render Styles—LithTech Render Styles provide a consistent mechanism to customize how models are rendered. The customization encompasses parts of both lighting and rasterization from the basic render states and texture passes to dynamic UV generation and lighting material properties. Programmers can use Render Styles Editor to create unique render styles that enable visually stunning models. Artists can use the Render Styles Viewer to preview any render styles that you apply to your model. Jupiter includes numerous render styles that can be found in the \Development\Tools\RenderStyles\ folder.

Sound—You can use the sound creation application of your choice to compose .WAV format sounds for your Jupiter game.

DirectMusic—You can use Microsoft's DirectMusic Producer to compose interactive musical scores for your Jupiter game. Jupiter supports the bulk of DirectMusic features for DirectX 7 and 8.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.



LithTech Resource Types

Jupiter can access and use a wide variety of file types and other resources. The list below is arranged by resource type, and each resource type includes a list of the sorts of files Jupiter can use in that type, as well as a description of those files.

Directories—The subdirectories that contain your other game resources are a type of resource in themselves. The structure of these directories defines the tree views you see in many of the tabs in DEdit's Project window. If you find a game's .DEP file and look at the

subdirectories in the same folder with it, you'll see names such as TEXTURES, WORLDS, SOUNDS, and MODELS. These folders contain whatever resource they're named for, so if you want to add new textures to a game or copy a world from one game to another, you should look in the directories named Textures and Worlds, respectively.

Note: Inside each of these directories is a tag file with a name that tells DEdit what resource it contains. A file called DIRTYPETEXTURES tells DEdit that the directory contains textures, DIRTYPEWORLDS says the directory contains worlds, and so on. If you want to add new subdirectories or reorganize your resources, just copy the proper tag file into any new directories that don't have one, and they will appear on the proper tabs in DEdit.

Models—Models in Jupiter are in a custom format, the .LTA file. This contains the mesh/geometry for the model and other information used by the game engine. You create these files in an external editor such as Maya or 3D Studio Max, then save them in the .LTA file format using a plug-in. Use ModelEdit to modify files in the .LTA format once they've been created.

Prefabs—A prefab is a collection of objects, both brushes and code objects that are stored in their own .LTA files (just like a regular game world) in a separate set of directories. Anything you make in a level can be exported as a prefab to make it easier to re-use later. Examples are benches, camera systems, hallways, doorways, and statues. Prefabs can be useful if your group needs a way to distribute standard-looking objects to its members.

Project—As mentioned before, the project is the total of your whole game and all its resources. A .DEP (DEdit Project) file at the top of your tree of game resource directories serves as a guide to DEdit. The .DEP file is a pointer to the rest of the resources for DEdit and includes some of the information about your game resources. The rest of your game's files exist in subdirectories of the directory where your .DEP file resides.

Simulation Objects—These are objects that programmers construct using code. Some of this code exists in the engine and doesn't appear in the project directories. However, many objects are created in code written for the specific game. These objects are called game objects (to distinguish them from the engine objects inside the engine code) and are stored in a file called OBJECT.LTO in the same directory as the project's .DEP file.

Sounds—Jupiter supports standard .WAV files directly with no custom modifications. When you double-click a .WAV inside of DEdit's Project Window, it will open in your system's default sound editor so that you can preview it.

Sprites—Sprites (.SPR files) consist of a series of .DTX files linked together as an animation with a set frame rate. They're commonly used for animations and special effects such as smoke, bullet holes, and liquid droplets.

Textures—Jupiter uses its own form of texture, the .DTX file. Textures can be up to 32-bit images, although you can obviously choose to make lower-color textures. The .DTX files also contain flags for additional information used by the engine and the game. You can convert .PCX and .TGA files into .DTX files by importing them inside of DEdit.

Note: Paint Shop Pro™ and some other programs have the capability of writing compressed run-length encoded Targa (.TGA) files. Though this compression makes the files smaller to store, it renders them unreadable by DEdit when you try to create a .DTX texture. You can choose the Uncompressed option in Paint Shop Pro when you save a .TGA file to make that file readable by DEdit.

Worlds—Worlds come in two forms: .LTA files and .DAT files. If you think of worlds as programs, the .LTA file is the source code: Editable and understandable by the user and DEdit, but not executable. The .DAT file is like an actual program: it can be run in the game, but since it's been "compiled" (processed, in this case), the user can no longer modify it. The .LTA files are what you change and save changes to inside of DEdit, and the .DAT file is the output of Processor.exe, which prepares your world to run in the game and optimizes its performance.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Types Of Geometry

The term "geometry" in Jupiter actually describes several very different classes of objects: Models, WorldModels and World Geometry. It's useful to know and remember the difference between these three classes of geometry, since they are made very differently and can be used in very different ways.

There is overlap between the various classes. You can build a vehicle as either a Model or a WorldModel, depending on how flexibly you need it to move. You could even build a vehicle out of World Geometry if it was never going to move. Likewise, trees can be made as any of the three depending on what's most convenient. For complexity, the Model is preferable. For interactive, solid world objects, the WorldModel is usually best. For solidity and visibility control, World Geometry is best.

LithTech supports the following classes of geometry objects:

- [Models](#)
- [WorldModels](#)

- [World Geometry](#)

Models

Models are, as the name implies, objects built in 3D Studio or another external modeling package. Their mesh is stored in an .LTA file that is exported from that package and loaded by Jupiter.

Models are usually used for objects in the world that have a high polygon count, require complex animation or would be very difficult to make inside DEdit. Players, weapons and props like dishes and chairs usually are made with a model. Doors, walls and other large, structural features are usually not.

Models can't block visibility the way world geometry can, and they generally don't integrate with the brushes in the world. Instead, they sit on top of brushes. This makes them very suitable for props like chairs and desks, since you can quickly place a large number of them with less work. It makes them unsuitable, however, for making a whole building or parts of terrain.

WorldModels

WorldModels are a hybrid: they consist of an object with a brush or brushes bound to it, which provide its mesh. The brushes are built and textured inside of DEdit, which allows better integration with the rest of the level. Unlike regular brushes, though, WorldModels can be translucent, can move and can be destroyed and removed from the level.

WorldModels are often used for parts of a level like large mechanical props, vehicles, doors, glass and fences. They usually don't block visibility but can be made to do so with customized game code to an extent. They're not useful for character models, since they're not easily deformable.

[Top](#)

World Geometry

World Geometry is made of brushes. These brushes can be built and textured in DEdit or constructed in any supported 3D package. World Geometry makes up the vast bulk of almost all worlds in a DEdit game. It's used for walls, ceilings, columns, ground, sky and many other solid, immovable objects in a world. World geometry is very cheap to render.

However, due to World Geometry's limitations (it can't move or be translucent), world geometry must be bound up in a WorldModel to make things like doors and models are more appropriate for complex objects like chandeliers or moving trees.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Using DEdit



DEdit is a tool that allows you to build the ground, sky, walls, ceilings and shapes for your levels. It is also used to assemble parts of your level that you've built in a supported external 3D package, such as 3D Studio Max or Maya. You can then apply or modify textures, props, prefabs, lights and sounds to give them a realistic appearance. No One Lives Forever 2 was created using literally thousands of these assets. You can also add the simulation objects that allow other players to interact with your world.

Creating levels is an extremely complex and time consuming process, and the following instructions are merely the starting point for those who want to learn the basics of building worlds for Jupiter. If you have previously designed 3D or near-3D games, then you have probably worked with tools like DEdit.

Note: While you can use the No One Lives Forever 2 objects and code provided with Jupiter, the art resources are included as samples only and cannot be included in any released games. TO2 textures, models, levels, and animations are the property of Monolith Productions, and may not be included in any released product.

The following table describes the topics and tutorials available in this section for learning and using the DEdit tool:

DEdit Topics	Description
About Dedit	Defines the DEdit GUI, shortcut keys, and the basic DEdit interface
Basic DEdit Terminology	Defines the basic terms used in DEdit tutorials and instructions
Working With Brushes	Provides procedural instruction on using brushes in DEdit
Working With Geometry	Provides procedural instruction on using geometry in DEdit
Working With Objects	Provides procedural instruction on using objects in DEdit
Working With Textures	Provides procedural instruction on using textures in DEdit
Using the Camera	Describes how to use the camera to change your view
Adding Simulation Objects	Describes how to use simulation objects
Creating World Models	Describes how to use the world model
Lighting the Map	Describes how to apply and modify lighting
Organizing World Components	Describes how to use the nodes view to organize world components
Optimizing Your Level	Describes how to optimize your level
DEdit Tutorials	
Tutorial: Building Your First World	Provides instruction on creating a simple world
Tutorial: Using Brushes	Provides instruction on using brushes
Tutorial: Adding Sky to Your Level	Provides instruction on adding a sky box

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



About the DEdit Interface

The DEdit interface allows you to insert and manipulate all of the components necessary to create a level using the Jupiter system. This section introduces you to the following DEdit interface and defines some of the key functionality you must know to create your levels.

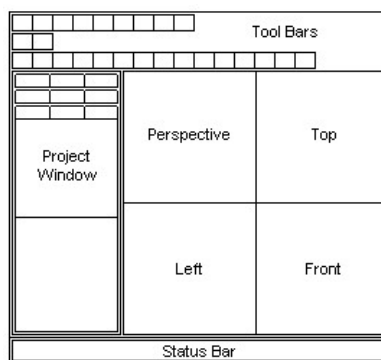
- [UI Interface Components](#)
- [Understanding the Toolbars](#)
 - [Using the Viewports](#)
 - [Moving the Camera](#)

UI Interface Components

The following image shows the DEdit user interface. It is composed of menus, buttons, tabs, and an editing area.



The following image identifies the components of the DEdit UI in the previous image. All sections in this document use the terms illustrated below when referring to the DEdit UI.



Use the toolbar buttons to make new worlds, switch modes, and run your world. You can learn an individual button's function by hovering your mouse over the button and viewing its tool tip.

In the **Project Window**, DEdit gives you access to all of the resources in the game. Most tabs contain resources you can add to your game. There are two special tabs, the **Properties** tab and the **Nodes** tab, that relate to objects inside your maps.

There are four viewports that provide windows onto the world you're building. Each one presents a different view of the world. The upper left window shows you a **Perspective** view shot from a movable camera point in the level. The other three views have a fixed-view camera that displays either **Top**, **Front**, or **Left** view (in clockwise order).

In the **Status Bar**, DEdit provides information about the level or about the view you're currently in. Use the Tool Tips and the Status Bar to learn more about what the controls do.

[Top](#)

Understanding the Toolbars

DEdit toolbars contain buttons that activate a variety of useful functionality. The following table describes each of the specialized buttons on the DEdit toolbars.

	Single-Node Selection Limits selection to a single object.
	Multi-Node Selection Allows you to select more than one object.
	Brush Edit Allows you to edit brushes and objects.
	Geometry Edit Allows you to edit the vertices of a brush or object.
	Object Edit Allows you to edit the vertices of an object.
	Hollow Converts the faces of a brush to individual polygons of a specified thickness.
	Shrink Grid Adds grid lines to the grid in the currently selected view, increasing the resolution of the grid and allowing the placement of more closely spaced points.
	Expand Grid

	Removes gridlines from the grid in the currently selected view, decreasing the resolution of the grid and allowing the placement of more widely spaced points.
	Apply Color Applies color to a brush or object.
	Apply Texture Applies texture to a brush or object
	Center View Centers the view to the current selection, or on the marker if no brush or object is selected.
	Center View on Marker Centers the view on a previously set marker.
	Marker to Selection Sets a marker to the selected brush or object.
	Marker to Camera Sets the marker to the current camera location.
	Marker to Vector Sets the marker to a specified vector using a period delimited three digit X, Y, and Z coordinate.
	Object Search Opens the Object Search dialog box allowing you to generate a list of the current objects in the current level.
	Level Textures Opens the Level Textures dialog box, allowing you to generate a list of the current textures in the current level.
	Build Processes and compiles the current world.
	Launch Launches the currently built world using the Jupiter engine.

[Top](#)

Using the Viewports

Move your mouse pointer over the upper-right-hand viewport (top view). The green crosshair (also called the marker or the insertion point) operates like the cursor in a word processor: it shows your insertion point for drawing brushes.

[Top](#)

Moving the Camera

To move the camera in DEdit, use the **I**, **O** and **C** keys in conjunction with the mouse.

The following table lists a few of the most important key and mouse combinations for moving the camera view.

Press	Result
I + Mouse 	In the any view, hold the I key and move the mouse to scroll in the X and Z planes. Additionally, you can increase the speed of the movement by holding the I + SHIFT keys.
I + Right Mouse 	In Perspective view, hold the I key and the right mouse button to scroll in the Y plane. In all other views, this combination scrolls the view in the X and Z planes.
O + Mouse 	In the Perspective View, hold the O key and move the mouse to pan the camera in all directions. In all other views, this combination zooms in and out of the displayed geometry.
O + Left Mouse Button 	In Perspective View, hold the O key and the left mouse button to move or “fly” forward through the level. You can continue to pan the camera by moving the mouse during this operation.
O + Right Mouse Button	In the Perspective View, hold the O key and the right mouse button to move or “fly” backward through the

	level. You can continue to pan the camera by moving the mouse during this operation.
C + Mouse 	In the Perspective view, hold the C key and move the mouse forward to increase the clipping plane, and backward to decrease the clipping plane. The far clipping plane is the portion of your world rendered in the DEdit level editor. If you have a large number of polygons to render, DEdit may slow down to process them. Use far clipping to decrease the amount of your world rendered and speed up DEdit. Far clipping does not effect how your game appears when it is compiled, and only works in Perspective view.

How to Zoom

- With the **Top**, **Left**, or **Front** view selected, hold the **O** Key and **move the mouse up** to zoom in or **down** to zoom out. Zoom operates the same in any view, allowing you to zoom in or out on any portion of the map.
- With **Perspective** view selected, hold the **O** key and the **left mouse button**. The objects in the view window get larger. You are not actually zooming in or changing the scale of the grid, rather you are moving the camera forward as if you were "flying" through the level.
- With **Perspective** view selected, hold the **O** key and the **right mouse button**. The objects in the view window get smaller. You are not actually zooming out or changing the scale of the grid, rather you are moving the camera backward as if you were "flying" through the level.

Note: Zoom operates differently in **Perspective** view than it operates in **Top**, **Left**, and **Front** view. In **Perspective** view you can pan while zooming, using the mouse to move the camera on its axis while you "fly" through the level. In **Top**, **Left**, and **Front** view, you cannot move the camera on its axis while zooming, you can only zoom in or out of the current view.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



DEdit Shortcut Keys

Control and creation of objects in DEdit involves a large number of shortcut keys. For maximum control over the DEdit application you are advised to memorize some of the most commonly used shortcut keys.

Shortcut keys for some functions do not exist when DEdit is started, and if you wish to use them you must define a shortcut key for them. Functions without assigned shortcut keys are listed in the User Defined Shortcut Keys section.

Note: To access a full list of shortcut keys, on the Edit menu, click Edit Key Configuration.

For instructions on adding or changing DEdit shortcut keys, see [Editing Shortcut Keys](#).

For a print-formatted version of this list, click [DEdit Shortcut Keys Long List](#). For a two page print ready version of this list without descriptions, click [DEdit Shortcut Keys Short List](#).

This section describes the following commonly used shortcut keys:

- [Brush Keys](#)
 - [Create Box](#)
 - [Create Cylinder](#)
 - [Create Dome](#)
 - [Create Pyramid](#)
 - [Create Sphere](#)
 - [Draw Poly](#)
- [Draw Poly.Insert Edge](#)
 - [Draw Poly.Undo](#)
 - [Extrude Poly](#)
- [Extrude Poly.Brush](#)
- [Extrude Poly.Snap](#)
- [Move Immediate Vert](#)
 - [Nudge Down](#)
 - [Nudge Left](#)
 - [Nudge Right](#)
 - [Nudge Up](#)
- [Select Brush Color](#)
- [Triangulate Brushes](#)

- [Camera, Navigation, and View Keys](#)
 - [Lock Focus](#)
 - [Maximize Active View](#)
 - [Maximize View 1](#)
 - [Maximize View 2](#)
 - [Maximize View 3](#)
 - [Maximize View 4](#)
 - [Nav Move](#)
 - [Nav Move Fast](#)
 - [Nav Move Perp](#)
 - [Nav Orbit](#)
 - [Nav Orbit Move](#)
 - [Nav Rotate](#)
 - [Nav Rotate Forward](#)
 - [Nav Rotate Backward](#)
 - [Navigator Next](#)
 - [Navigator Previous](#)
 - [Reset Viewports](#)
 - [Zoom.Fast](#)
- [Editing Keys](#)
 - [Brush Size](#)
 - [Carve Brush](#)
 - [Copy Selection](#)
 - [Delete](#)
 - [Delete Edges](#)
- [Delete Tagged Polies](#)
 - [Edit Curve](#)
 - [Far Distance](#)
 - [Flip Normal](#)
 - [Freeze Selection](#)
 - [Group Selection](#)
 - [Join Brushes](#)
 - [Move Node Handle](#)
 - [Move Node Perp](#)
 - [Move Node Snap](#)
 - [Move Poly Vert](#)
 - [Move Selected Vert](#)
 - [Node Move.Clone](#)
 - [Paste in Place](#)
 - [Paste to Marker](#)
 - [Redo](#)
 - [Remove Edges](#)
 - [Rotate Node](#)
 - [Rotate Node Step](#)
 - [Rotate Selection Left](#)
 - [Rotate Selection Right](#)
 - [Scale Immediate Vert](#)
 - [Scale Poly](#)
 - [Scale Poly Divide](#)
 - [Scale Poly Extrude](#)
 - [Split Brush](#)
 - [Split Edge](#)
 - [Undo](#)
 - [Unjoin Brushes](#)
 - [Grid Keys](#)
 - [Expand Grid Spacing](#)
 - [Grid Size](#)
 - [Grid To Poly](#)
 - [Set Grid Dir Forward](#)
 - [Set Grid Dir Right](#)
 - [Set Grid Dir Up](#)
 - [Shrink Grid Spacing](#)
 - [Hide/Show Keys](#)
 - [Hide Inverse](#)
 - [Hide Selected](#)
 - [Toggle Class Icons](#)
 - [Unhide Inverse](#)
 - [Unhide Selected](#)
 - [Marker Keys](#)
 - [Center Marker](#)
 - [Move Marker](#)
 - [Mode Keys](#)
 - [Brush Edit Mode](#)
 - [Geometry Edit Mode](#)
 - [Object Edit Mode](#)
 - [Object Keys](#)

- [Add Object](#)
 - [Object Size](#)
- [Selection Keys](#)
 - [Advanced Select](#)
 - [Select Depth](#)
 - [Select None](#)
 - [Tag All](#)
 - [Tag Poly Verts](#)
 - [Tag.Add](#)
 - [Tag.Invert](#)
- [Texture Keys](#)
 - [Apply Texture Click](#)
 - [Apply Texture To Sel](#)
 - [Fit Texture To Poly](#)
 - [Init Texture Space](#)
 - [Map Texture Coords](#)
 - [Remove Textures](#)
 - [Texture Move](#)
 - [Texture Move.Rotate](#)
 - [Texture Move.Scale](#)
 - [Texture Move.Step](#)

Brush Keys

Create Box

Key: **CTRL + SHIFT + 1**

Description: Inserts a box primitive.

Create Cylinder

Key: **CTRL + SHIFT + 2**

Description: Inserts a cylinder primitive.

Create Dome

Key: **CTRL + SHIFT + 5**

Description: Inserts a dome primitive.

Create Pyramid

Key: **CTRL + SHIFT + 3**

Description: Inserts a pyramid primitive.

Create Sphere

Key: **CTRL + SHIFT + 4**

Description: Inserts a sphere primitive.

Draw Poly

Key: **SPACE**

Description: Inserts a new vertex (for brush drawing in Brush or Geometry mode) at the cursor point. Each successive press of SPACE adds a new connected vertex to the brush or object you are drawing.

Draw Poly.Insert Edge

Key: **E**

Description: Inserts an edge on the selected polygon by connecting the two selected vertices.

Draw Poly.Undo

Key: **BACKSPACE**

Description: Erases the most recently drawn vertex.

Extrude Poly

Key: **Q**

Description: Extrudes a polygon as part of the current brush.

Extrude Poly.Brush

Key: **CTRL + Q**

Description: Extrudes a polygon as a new brush.

Extrude Poly.Snap

Key: **SHIFT**

Description: Snaps the extrude amount to the grid size of the current view.

Move Immediate Vert

Key: **M + LEFT MOUSE BUTTON**

Description: Moves the vertices underneath the cursor.

Nudge Down**Key: DOWN KEY****Description:** Nudges the selected brush down one grid line. Use **SHIFT + DOWN KEY** to move by pixels.**Nudge Left****Key: LEFT KEY****Description:** Nudges the selected brush left one grid line. Use **SHIFT + LEFT KEY** to move by pixels.**Nudge Right****Key: RIGHT KEY****Description:** Nudges the selected brush right one grid line. Use **SHIFT + RIGHT KEY** to move by pixels.**Nudge Up****Key: UP KEY****Description:** Nudges the selected brush up one grid line. Use **SHIFT + UP KEY** to move by pixels.**Select Brush Color****Key: CTRL + G****Description:** Sets the current color to the color of the first selected brush polygon.**Triangulate Brushes****Key: CTRL + E****Description:** Triangulates the selected brushes.[Top](#)

Camera and View Keys

Lock Focus**Key: SHIFT****Description:** Locks the input focus to the currently active view.**Maximize Active View****Key: 5****Description:** Maximizes the Active View window.**Maximize View 1****Key: 1****Description:** Maximizes the Upper Left window.**Maximize View 2****Key: 2****Description:** Maximizes the Upper Right window.**Maximize View 3****Key: 3****Description:** Maximizes the Lower Left window.**Maximize View 4****Key: 4****Description:** Maximizes the Lower Right window.**Nav Move****Key: I + MOUSE****Description:** Sets the mouse to control the camera position.**Nav Move Fast****Key: I + SHIFT + MOUSE****Description:** Same as Nav Move, but increases the speed at which the camera is moved.**Nav Move Perp****Key: I + RIGHT MOUSE BUTTON****Description:** Sets the mouse to control movement perpendicular to the plane of viewing. In the perspective view, this typically allows you to move the camera vertically.**Nav Orbit****Key: L + MOUSE****Description:** In **Perspective** view, centers and orbits the camera around the selected brush or point.**Nav Orbit Move****Key: L + LEFT MOUSE BUTTON**

Description: In **Perspective** view, when orbiting, causes the camera to move forward.

Nav Rotate

Key: **O + MOUSE**

Description: In **Perspective** view, moves the camera in all directions about its axis, allowing you to pan in any direction.

Nav Rotate Forward

Key: **O + LEFT MOUSE BUTTON**

Description: When rotating in **Perspective** view, pressing the left mouse button moves the camera forward.

Nav Rotate Backward

Key: **O + RIGHT MOUSE BUTTON**

Description: When rotating in **Perspective** view, pressing the right mouse button moves the camera backward.

Navigator Next

Key: **CTRL +]**

Description: Moves to the next navigator position in the list.

Navigator Previous

Key: **CTRL + [**

Description: Moves to the previous navigator position in the list.

Reset Viewports

Key: **6**

Description: Resets viewports to default setting. This only has an effect if you have resized the viewports. You can resize the DEdit viewports by clicking and dragging the grey frame border between the Perspective, Top, Front, and Left viewports.

Zoom.Fast

Key: **I + SHIFT + MOUSE**

Description: In Top, Front, and Left view modes, pressing **SHIFT** in conjunction with the **I** key (horizontal zoom) increases the rate of camera movement.

[Top](#)

Editing Keys

Brush Size

Key: **LEFT MOUSE BUTTON**

Description: In **Brush Edit** mode, left-click and drag the brush handles to size the brush. The brush snaps to the gridlines, so change the grid size to control the maximum and minimum size of the brush.

Carve Brush

Key: **H**

Description: Uses the selected brush to carve the surrounding geometry. The selected brush is deleted along with any overlapping geometry. DEdit extrapolates an approximate cut, and you may need to edit the new geometry to ensure correct connection between faces. Carving uses CGS subtraction and often creates unnecessary brushes, geometry that is difficult to edit, and geometry that may not make sense to a human engineer or artist. Carving is a useful tool, but requires careful attention to use correctly.

Copy Selection

Key: **CTRL + C**

Description: Standard copy operation. Copies all of the selected items (brushes, geometry, objects, etc) to the clipboard for pasting.

Delete

Key: **DELETE**

Description: Deletes the currently selected brush.

Delete Edges

Key: **END**

Description: In **Geometry** mode, deletes the selected edges.

Delete Tagged Polies

Key: **PAGE DOWN**

Description: In **Geometry** mode, deletes all currently selected polygons, allowing you to remove faces from brushes.

Edit Curve

Key: **P + LEFT MOUSE BUTTON**

Description: Allows the editing of curves for objects with curves set as a parameter. Typically used with KEY objects.

Far Distance

Key: **C**

Description: Moves the clipping plane. Hold C and move the mouse down to shrink the clipping plane (rendering less). Hold C and move

the mouse up to expand the clipping plan (rendering more).

Flip Normal

Key: **F**

Description: Flips the normals of the selected polygon, reversing the visible or invisible state of its faces.

Freeze Selection

Key: **J**

Description: Freezes the currently selected brushes. You cannot select or edit frozen brushes. To unfreeze a brush, uncheck the “freeze” checkbox for the brush(es) in the Nodes list. You can also freeze other objects such as lights, props and AIs. You cannot freeze geometry, but it becomes frozen when you freeze the brush.

Group Selection

Key: **CTRL + SHIFT + S**

Description: In Multi-Node Selection mode, allows you to select multiple brushes and group them for movement. Press **U** to deselect the group.

Join Brushes

Key: **SHIFT + J**

Description: Joins the selected brushes. Joining takes individual faces or even whole brushes and binds them together into one physical brush. Prior to using Join Brushes you must use **CTRL + J** to [Unjoin](#) a brush.

Move Node Handle

Key: **LEFT MOUSE BUTTON**

Description: In Top, Front, or Left view, allows you to move the selected brush by clicking and dragging the handle at its center. You can also use the **ARROW** keys or **SHIFT + ARROW** keys to move selected brushes. (see the [Nudge](#) keys in the [Brush Keys](#) section)

Move Node Perp

Key: **M + RIGHT MOUSE BUTTON**

Description: In Top, Front, or Left view, moves the selected brush perpendicular to the plane of viewing by pixels (without snapping the object to the grid).

Move Node Snap

Key: **M + LEFT MOUSE BUTTON**

Description: Snaps the brush to the nearest grid position as it is moved.

Move Poly Vert

Key: **SHIFT + RIGHT MOUSE BUTTON**

Description: Moves the selected vertices, extruding brushes in a direction perpendicular to the selected face.

Move Selected Vert

Key: **S + LEFT MOUSE BUTTON**

Description: Moves the selected vertex to the location you click.

Node Move.Clone

Key: **D + MOUSE**

Description: When moving a brush, clones the selection and moves the clone.

Paste in Place

Key: **CTRL + ALT + V**

Description: Pastes a copy of objects from the clipboard directly onto the X,Y,Z coordinates they were copied from.

Paste to Marker

Key: **CTRL + V**

Description: Pastes a copy of objects from the clipboard at the current marker.

Redo

Key: **CTRL + Y**

Description: Repeats the last command.

Remove Edges

Key: **HOME**

Description: Deletes the currently selected edges.

Rotate Node

Key: **N + MOUSE**

Description: Rotates the currently selected brush. Hold **N** and move mouse Left or Right to rotate.

Rotate Node Step

Key: **N + SHIFT + MOUSE**

Description: When rotating a brush, holding **SHIFT** rotates the brush in discrete amounts.

Rotate Selection Left

Key: CTRL + LEFT ARROW

Description: Rotates the selection left around its own axis.

Rotate Selection Right

Key: CTRL + RIGHT ARROW

Description: Rotates the selection right around its own axis.

Scale Immediate Vert

Key: CTRL + SHIFT + W

Description: Scales the selected vertex relative to the marker. Moves the current vertices on a straight line that starts at the marker and then passes through the vertices. It also creates new faces, if needed, to fill in gaps that left when movement occurs.

Scale Poly

Key: W

Description: Scales the currently selected face. **MOUSE UP** to expand, **MOUSE DOWN** to decrease.

Scale Poly Divide

Key: SHIFT + W

Description: Divides the currently selected face into sub-faces and scales the middle polygon.

Scale Poly Extrude

Key: W + RIGHT MOUSE BUTTON

Description: Extrudes the selected face.

Split Brush

Key: S

Description: Splits the selected brush. You must first select a brush in Brush mode and draw a single line accross it prior to pressing the **S** key. For more information about splitting brushes, see [Splitting Brushes](#).

Split Edge

Key: INSERT

Description: Splits the selected edge.

Undo

Key: CTRL + Z

Description: Undoes the last command.

Unjoin Brushes

Key: CTRL + J

Description: Unjoins the selected joined brushes.

[Top](#)

Grid Keys

Expand Grid Spacing

Key: NUMBERPAD PLUS KEY (+)

Description: Increases grid spacing in the current view.

Grid Size

Key: Z + RIGHT MOUSE BUTTON

Description: In perspective view, allows the mouse to shrink or expand the grid size (not the grid spacing, but the size of the entire grid). Move the **MOUSE UP** to increase the gride size, **DOWN** to decrease the grid size.

Grid To Poly

Key: G

Description: In Geometry view, aligns the grid to the currently selected face.

Set Grid Dir Forward

Key: 8

Description: Sets the grid to lie on the X/Y plane. (Default setting).

Set Grid Dir Right

Key: 9

Description: Sets the grid to lie on the Y/Z plane.

Set Grid Dir Up

Key: 0

Description: Sets the grid to lie on the X/Z plane.

Shrink Grid Spacing

Key: **NUMBERPAD MINUS KEY (-)**
Description: Decreases the grid spacing in the current view.

[Top](#)

Hide/Show Keys

Hide Inverse

Key: **;**

Description: All unselected items become hidden.

Hide Selected

Key: **[**

Description: All selected items become hidden.

Toggle Class Icons

Key: **SHIFT + D**

Description: Toggles the display of class icons in all viewports.

Unhide Inverse

Key: **'**

Description: All unselected hidden items become visible.

Unhide Selected

Key: **]**

Description: All selected hidden items become visible.

[Top](#)

Marker Keys

Center Marker

Key: **SHIFT + X**

Description: Positions the marker at the center of the current selection.

Move Marker

Key: **X**

Description: Positions the marker at the current mouse location.

[Top](#)

Mode Keys

Brush Edit Mode

Key: **CTRL + B**

Description: Switches to **Brush Edit** mode.

Geometry Edit Mode

Key: **CTRL + G**

Description: Switches to **Geometry Edit** mode.

Object Edit Mode

Key: **CTRL + H**

Description: Switches to **Object Edit** mode.

[Top](#)

Object Keys

Add Object

Key: **CTRL + K**

Description: Adds an object to the world at the marker.

Object Size

Key: LEFT MOUSE BUTTON

Description: Sizes the currently selected object using the size handles.

[Top](#)

Selection Keys

Advanced Select

Key: SHIFT + A

Description: Opens the **Advanced Selection** dialog box.

Select Depth

Key: T + LEFT MOUSE BUTTON

Description: Allows sequential selection of items below the mouse arrow based on their order of depth.

Select None

Key: U

Description: Unselects all currently selected items.

Tag All

Key: LEFT MOUSE BUTTON

Description: Selects the item below the mouse pointer.

Tag Poly Verts

Key: Y

Description: In **Geometry** mode, selects all the vertices of the currently selected polygon.

Tag.Add

Key: SHIFT

Description: When selecting items, holding **SHIFT** allows for multiple selections.

Tag.Invert

Key: CTRL

Description: When selecting items, pressing **CTRL** inverts the selection status of an individual item (selects or deselects it).

[Top](#)

Texture Keys

Apply Texture Click

Key: A + Y + LEFT MOUSE BUTTON

Description: Applies the currently selected texture to the brush you select.

Apply Texture To Sel

Key: CTRL + T

Description: Applies the currently selected texture to the currently selected brush or patch.

Fit Texture To Poly

Key: SHIFT + C

Description: In **Geometry** mode, fits the applied texture to the surface of the currently selected polygon.

Init Texture Space

Key: K

Description: Initializes (resets) texture space. With brushes, this resets the state of the texture's alignment, scaling and plane of projection. With a single face, multiple presses causes the texture to re-align itself, trying to fit to each edge of the polygons on the face. You can use Init Texture Space to align a texture with a sloping edge on a brush.

Map Texture Coords

Key: A

Description: Opens the Map Texture dialog to map the texture coordinates of the selected brush entities.

Remove Textures

Key: CTRL + SHIFT + T

Description: Removes the texture from the current selection.

Texture Move

Key: R

Description: Sets the mouse to move the texture on the selected face.

Texture Move.Rotate**Key: R + RIGHT MOUSE BUTTON****Description:** Sets the mouse to rotate the texture on the selected face.**Texture Move.Scale****Key: R + LEFT MOUSE BUTTON****Description:** In **Geometry** mode, sets the mouse to scale the texture on the selected face.**Texture Move.Step****Key: SHIFT****Description:** While manipulating textures, pressing the shift key allows manipulation in small finite steps.[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide

Editing Shortcut Keys

DEdit uses many shortcut keys, and in most cases it is far easier to use the shortcut keys than to use the menus. Some options, such as camera movement, require shortcut keys and are not available on the menus. For this reason DEdit has a shortcut key editor that allows you to change the key combinations associated with commands. Additionally, some commands are unassociated with key combinations by default, and in order to enable them you must assign a key combination to the command. For information about the currently assigned shortcut keys, see [DEdit Shortcut Keys](#), or in the DEdit **Edit** menu, click **Edit Key Configuration**.

This section covers the following shortcut key editing topics:

- [Navigating the Current Key Configuration Dialog](#)
 - [Changing or Adding a Shortcut Key](#)
 - [Clearing and Resetting Shortcut Keys](#)

Navigating the Current Key Configuration Dialog

In the DEdit Current Key Configuratin dialog box, you can move rapidly between entries by pressing the first letter of the name of the entry you wish to view or edit.

To move to an entry quickly

1. On the **Edit** menu, click **Edit Key Configuration**.
2. In the **Current Key Configuration** dialog box, click the first entry.
3. Type the first letter of the entry, for example "E" if you are looking for Edit Curve.

The dialog box jumps to the first entry beginning with the letter "E."

[Top](#)

Changing or Adding a Shortcut Key

You can change or add key assignments to any command entry in the Current Key Configuration dialog box.

To change or add a key assignment

1. Select the entry you wish to change or add an assignment to.
2. Click the **Bind** button.
3. In the **Keys** area, select the keys to assign.
4. In the **Modifiers** area, select **Ctrl**, **Shift**, or both to add a modifier to the key assignment.
5. Click the **OK** button.

DEdit warns you if the key is already assigned and gives you the option to cancel or override the assignment.

[Top](#)

Clearing and Resetting Shortcut Keys

The DEdit Current Key Configuration dialog box allows you to both clear and reset shortcut keys. When you clear a shortcut key, its key assignment becomes empty. When you reset a single or all shortcut keys, the default key assignments are restored.

To clear a shortcut key

1. In the Current Key Configuration dialog box, select the shortcut key to clear.
2. Press the **Clear** button.

To reset a single shortcut key

1. In the Current Key Configuration dialog box, select the shortcut key to reset.
2. Press the **Reset** button.

To reset all shortcut keys

- Press the **Reset All** button.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Basic DEdit Terminology

The following terms are used when describing specific operations using the DEdit tool. You can find these and other additional gaming terminology in the [Appendix G: Glossary](#).

Brush	The basic unit of world geometry. A brush is made up of planes that define its faces, lines that define its edges, and vertices that define its corners. You can manipulate either an entire brush at once or any of the vertices, lines, or planes in the brush.
Mode	DEdit has several different modes that allow you to change certain elements of your world (that is, just brushes or just objects) without accidentally affecting others. They're designed to simplify interacting with your world while editing, so it's important to choose the right mode for your task.
Objects	Anything a player interacts with that moves, lights up, shoots, growls, or goes into the player's inventory is an object. DEdit allows you to place any kind of object that your game code or the engine code can create.
Prefab	A prefab is like a primitive, but more complex. If you build a street lamp complete with textures and a light source that gives off just the right shade of light, you can select the lamp and its associated source (or any group of objects and brushes inside DEdit) and save it as a prefab. Thereafter, you can copy and paste that streetlamp into your world without having to rebuild the whole object.
Primitive	A simple shape that you can add to the world as the foundation of a more complex shape. Typical primitives are things such as cubes, pyramids, and cylinders.
Processing	When you process your world, DEdit creates a second version of the world with some information removed (parts only you need to know about), and other information added (parts only Jupiter needs). Processing is a necessary step you'll take in getting your world up and running in the game, and is often the first place where you will discover problems with your level.
Project	A project in DEdit represents the sum of all the resources in the game: all the code, all the textures, all the sounds, all the worlds, and so on. Unlike other world geometry editors, DEdit first opens the project, rather than a world file.
Texture	In DEdit, textures are used like wallpaper, paint, or plaster to cover the raw plywood of your walls, floors, ceilings, doors and so forth. Without a texture, your brush will appear flat-shaded in the game, almost always with unpleasant results. Just as you wouldn't want a house with raw plywood walls, you always want to apply textures to the brushes in your level.
Unit	The basis of measurement in DEdit and Jupiter. Units don't equate directly to real-world values. Instead, the game designers can select a scale of game units to real-world measurements. As an example, in No One Lives

	Forever 2 the following values are used: railings are 48 units tall, a chest-high box would be 64 units, and a typical doorway would be 128 units high.
World	When your players walk around in your game, it is a world that they're standing in. Worlds divide your game up into sections where different parts of the game take place. In other games, these are sometimes referred to as maps, levels, scenes, or episodes. World-editing is the primary focus of DEdit and each world is stored in a separate file on the hard drive.
WorldGeometry	Parts of the world that act as walls, floors and sky are generally made of world geometry. Such geometry is solid, immovable, and generally never changes.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Working With Brushes

This section explains how to draw and manipulate brushes in DEdit. Brushes are three dimensional polygons inserted directly into DEdit when you are creating a level. You cannot deform brushes, but you can programmatically (through code) move and/or rotate them. This makes brushes ideal for static objects such as buildings, streets, walls, and the basic content of your world. Brushes are also used to create zones, or areas, in your game that trigger specific events coded by your developers.

All brushes have properties controlling various aspects of how they appear and function in your game. For more information about brush properties, see [Working With Properties](#).

All brushes can accept textures. For more information about textures, see [Working With Textures](#).

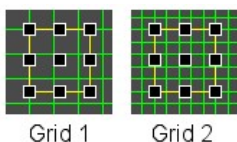
This section contains the following brush topics and procedures:

- [About Grid Spacing](#)
- [About Brush Editing](#)
- [About Brush Properties](#)
 - [Drawing a Brush](#)
- [Inserting a Brush Primitive](#)
 - [Sizing a Brush](#)
 - [Moving a Brush](#)
 - [Carving a Brush](#)
 - [Hollowing a Brush](#)
 - [Splitting a Brush](#)
 - [Rotating a Brush](#)
 - [Grouping Brushes](#)
- [Setting Normal Visibility](#)

About Grid Spacing

When you draw a brush, DEdit asks you for a brush thickness. The thickness of brushes is controlled in game units. Game units are equal to one square of the grid when the grid size is set to one. Changing your grid size does not effect game units, so if your grid size is set to sixteen, and you insert a brush with a thickness of eight, the thickness of the brush will appear to be half of one grid square even though it is actually eight game units thick. If you change your grid size back to one, you will see that the brush has a width of eight grid squares (although it may not be aligned with the grid).

The image below illustrates how grid spacing varies depending on your current grid spacing. Note that the grid color has been changed to green for the purpose of this illustration.



Grid 1

Grid 2

In the image above, a cube was drawn with grid spacing set at two (Grid 2). Then the grid was resized to grid spacing one (Grid 1). You can see that the edges of the polygon no longer all reside on the grid. When you size polygons by dragging the size handles, they automatically snap to the current gridlines. Because of this snapping, you may wish to set your grid spacing before sizing in order to ensure you are able to size your polygons correctly.

To change grid spacing

- With your mouse in the desired view, press the **+** or **-** key on the **numeric keypad**.

Plus increases grid spacing (for less detailed brushes).

Minus decreases grid spacing (for more detailed brushes).

Alternatively you can click the **Grid Shrink**  or **Expand Grid**  buttons.

About Brush Editing

To edit an entire brush (not just the vertices or faces) you must be in brush editing mode. Brush editing mode allows you to move, delete, and resize entire brushes. Any operation you apply to a brush in brush editing mode get applied to the entire brush. For example, if you apply a texture to a brush selected in brush editing mode, all of the faces of the brush receive the texture. For information about editing individual faces and vertices, see [Working With Geometry](#).

To enter Brush Edit mode.

- Press **CTRL + B** at any time to enter **Brush Edit** mode.
- Alternatively, you can also press the **Brush Edit**  button.

Note: All procedures in this section assume you are in Brush Edit mode.

[Top](#)

About Brush Properties

All objects in DEdit have properties, and brushes are no exception. Brushes have very important property settings that allow you to alter the way the brush gets rendered in your world. For additional information about brush properties, click the property name in the **Properties** tab for the desired brush.

To alter brush properties

1. Select the desired brush.
2. In the **Project Window**, click the **Properties** tab.

The following list explains some of the more common brush properties:

Name—Allows you to change the name of your brush. The name you specify also appears in the Nodes tab, so is useful in identifying important brushes.

Type—Specifies the behavior of the brush. Normal is the default type.

Lighting—Specifies the type of lighting applied to the brush.

NotAStep—If True, specifies that this brush cannot be used by the character as a stair step.

Detail—If True, specifies that the brush is used for world geometry. All worlds must have one, and only one, detail brush set to false.

Light Control—Sets the following options controlling how the brush deals with light:

AmbientLight—Specifies the color and intensity of light added to the brush during the lighting phase. All surfaces of the brush not lit by external light get set to AmbientLight. Darkness on the surface of the brush cannot drop below the ambient light setting.

ReceiveLight—Specifies if the brush will accept light, or only use ambient lighting. If False, then the AmbientLight value gets applied evenly over the entire surface of the brush (causing a glowing or dimming effect depending on the surrounding environment).

ReceiveShadows—Determines if this brush can have shadows cast on it by other brushes. If False, the surface of the brush ignores shadows and accepts light as if the surrounding brushes are transparent.

RecieveSunlight—Specifies if the brush can recieve sunlight from the skybox.

TextureEffect—For dynamic movement of textures, these effects are typically determined by a *.TFG file containing a script that controls the animation of the texture.

[Top](#)

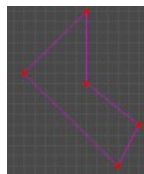
Drawing a Brush

You can draw a brush in any view except perspective view. The **mouse** controls the brush vertex insertion point, and the **space bar** inserts a vertex at the current mouse position.

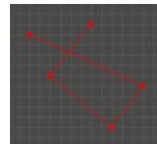
To draw a brush

1. Press **CTRL + B** to switch into **Brush** mode. Brush mode is reserved for moving and adding brushes to the world.
2. In the **Top**, **Front**, or **Left** view, move your mouse to a grid intersection, and press the **Space Bar**.
3. Repeat the previous step until you have created a geometric shap ending at your starting point.

Note: If the color of the lines changes to violet, your brush has a concave face. If the color of the lines changes to red, then you have crossed your lines and your brush is invalid. Concave faces can be corrected by [splitting](#), but invalid brushes always appear incorrectly in games. You should always avoid invalid brushes, and avoid or correct concave brushes.



Concave Brush



Invalid Brush

4. In the **Enter Brush Thickness** dialog box, type the thickness of your brush in game units. For more information about game units, see [About Grid Spacing](#).

Inserting a Brush Primitive

Primitives are simply ready-made geometric brushes that you can insert into your world and manipulate. DEdit can insert five types of primitives: **box**, **cylinder**, **pyramid**, **sphere** and **dome**. Primitives always insert at the marker current marker (set the marker using the **X** key).

To insert a brush primitive

1. In the **Top**, **Front**, or **Left** view, move your mouse to the location you wish to insert your primitive and press the **X** key. This places the marker at the selected grid location.
2. On the **Brush** menu, move to **Add Primitive**, and then click the brush primitive you wish to add.
3. In the **Primitive** dialog box, enter the geometry options for your primitive.
4. Click **OK**.

[Top](#)

Sizing a Brush

You can change the size of any brush drawn in DEdit by left-clicking and dragging the sizing handles. When you resize brushes, they automatically snap the the current gridlines. The direction the brush gets resized depends on the **Top**, **Front**, and **Left** view you use during the resizing operation.

To resize a brush

1. In the view containing the direction you wish to resize the brush, click the brush to select it.
2. Press the **+** or **-** keys (on the **numeric keypad**) to increase or decrease the grid spacing. This controls the increments you can resize the brush.
3. Click and drag the **sizing handle** on the side of the brush you wish to change.

[Top](#)

Moving a Brush

You can move any selected brush (or group of brushes) to any location in your world by clicking and dragging the brush.

To move a brush

1. In any view, click the brush you wish to move.

Note: Press CTRL + G to group brushes if you wish to move multiple brushes simultaneously. Press and hold the D key during the move operation to move a duplicate of the selected brush without changing the original brush.

2. In the view you wish to move the brush, click and drag the **center handle** of the brush to the new location. The view you move your brush in controls the X,Y, and Z position of the brush's movement.
3. Press **U** to deselect the brush.

[Top](#)

Carving a Brush

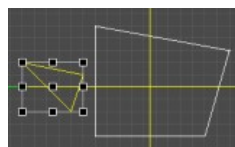
Carving requires two brushes: a brush to carve, and a brush to carve with. When you perform a carving operation, DEdit removes the portion of the brushes overlapping the selected brush.

Note: Carving is often avoided because it requires DEdit to determine the new geometry of the carved brushes, and may result in T junctions (vertices connecting to faces).

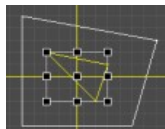
To carve a brush

1. Select or [draw](#) the brush you wish to use for carving.
2. [Move](#) the carving brush to overlap the brush you wish to carve.
3. Press the **H** key.

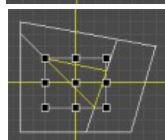
The following images illustrate the carving of a brush. The small pyramid carves a hole in the center of the large polygon.



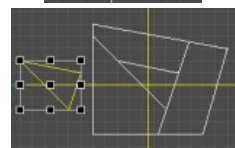
In **Top** view, the small pyramid is selected. The small pyramid is the carving tool, and its volume gets subtracted from any brushes it overlaps.



The small pyramid is moved to the carving point. This determines the area that is carved away. May sure to check Front and Left view to ensure you are carving the desired area.



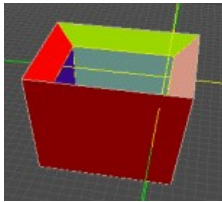
The **H** key is pressed. DEdit creates vertices for the carved faces of the large polygon. Notice that in several places DEdit has created T-junctions.



The small pyramid is moved away to show the carved area. The large polygon now consists of three smaller brushes. You can select and manipulate each of the new brushes individually or as a group.

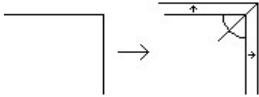
[Top](#)

Hollowing a Brush



Hollowing a brush empties out the center of the selected brush, breaks the brush into multiple smaller brushes, and creates sides of a specified width for each face of the brush. The image above shows a hollowed cube with the top brush deleted. As you can see, the edges between each of the hollowed faces get cut at an angle equal to half the radius of the initial angle. Thus, a 90 degree angle results in a 45 degree angle cut between the new brushes that form the sides of the box. Likewise, a 120 degree angle results in a 60 degree cut. The angle varies depending on the shape of the polygon.

The following image illustrates a 45 degree cut on a cubic face during a hollowing operation. Notice that hollowing expands the brush outward. When you perform a hollowing operation, the brush expands the width you select in every dimension.



To Hollow a Brush

In any view, select the brush you wish to hollow.

On the **Brush** menu, select **Hollow**, or on the toolbar, press the **Hollow**  button.

Enter the width of the faces resulting from the hollowing operation, and click **OK**.

[Top](#)

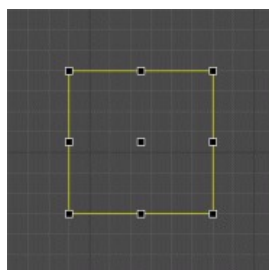
Splitting a Brush

Splitting a brush allows you to create two brushes from one brush. Splitting a brush is usually done to either alter the geometry of your world, or to add additional vertices for Gouraud shading.

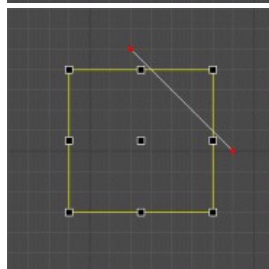
To split a brush

1. Select the brush to split.
2. In **Top**, **Front**, or **Left** view, press the **Space Bar** to start a line.
3. Move your mouse across the face of the selected brush along the angle to split the brush.
4. Press the **Space Bar** again to draw a single line (do not draw a brush, just a line).
5. Press the **S** key.

The following images illustrate the process of splitting a simple cubic brush in the **Top** view.

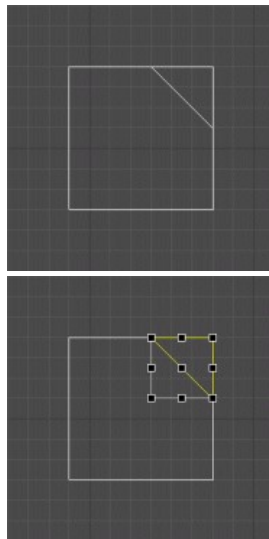


First select the cubic brush to split.



Next use the **Space Bar** to draw a line across the brush.

The **S** key is pressed to perform the split operation.



You now have two separate brushes, and any operation you can normally perform on a single brush can be performed on either of the two brushes.

[Top](#)

Rotating a Brush

You can rotate any brush or group of brushes around a central point. This allows you to create all types of off center geometry. The view that you rotate the brush in (Top, Front, or Left) determines the X,Y, and Z of the rotation.

To rotate a brush

1. Select the brush or group of brushes to rotate.
2. Press the **X** key to set the rotation point of the brush.
3. Move you mouse to the **Top**, **Front**, or **Left** view determining the angle of rotation.
4. Hold the **N** key, and move your mouse left or right to rotate the brush.
5. Release the **N** key when you have completed your rotation.

[Top](#)

Grouping Brushes

Grouping brushes allows you to perform any operation you would normally perform on one brush on an entire group of brushes. This includes movement, texture application, resizing, splitting, etc. Brushes are automatically ungrouped when they are unselected. There is no difference between selecting multiple brushes and grouping brushes. Grouping is not joining or merging, and groups are not permanent.

To group brushes

- In Top, Front, or Left view, click and drag over the brushes to group.

—or—

- In any view, hold down the **Shift** key, and click the brush to group. Click any brush again to remove it from the group.

[Top](#)

Setting Normal Visibility

Normal visibility controls the direction the faces of the brush are visible in the game. Faces are only visible from one direction, and any player looking at the opposite side of a brush face sees directly through those polygons. In other words, the term "normal" refers to the side of a polygon that the engine renders. When you set the normals in Geometry Edit mode, you set the normals for individual faces of the brush, but to set the normals for all of the polygons on a brush, you must enter Brush Edit mode. For information about setting the normals for individual faces of a brush, see [Setting Normal Visibility Direction](#) in the [Working With Geometry](#) section.

To set the normal visibility

1. Select the desired brush.
2. Press the **F** key to flip the normals of all faces on that brush.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Working With Geometry

This section explains how to draw and manipulate geometry in DEdit. Geometry is composed of faces, vertices, and edges. Each brush in DEdit is composed of multiple geometric elements (polygons) whose edges are determined by vertices. Just like Brush Edit mode allows you to work with entire brushes, Geometry Edit mode allows you to work with the individual geometric elements composing the brushes.

All polygons (individually or as a group) can accept textures. For more information about textures, see [Working With Textures](#).

Geometry mode, and polygons in general, are subject to the same grid spacing rules as brushes. For more information about grid spacing, see [About Grid Spacing](#) in the [Working With Brushes](#) section.

This section contains the following geometry topics and procedures:

- [About Geometry Editing](#)
 - [Drawing a Polygon](#)
 - [Moving Vertices](#)
 - [Deleting Faces](#)
- [Setting Normal Visibility Direction](#)

About Geometry Editing

To edit vertices, faces, and edges you must be in **Geometry Edit** mode. If you enter Geometry Edit mode with a brush selected, you can only apply changes to the geometry of the selected brush. If you enter Geometry Edit mode without a brush selected, you can apply changes to any geometry in your world.

To enter Geometry Mode

- Press CTRL + G to enter **Geometry Edit** mode at any time.
- Alternatively, you can also click the **Geometry Edit**  button.

Note: All procedures in this section assume you are in **Geometry Edit** mode.

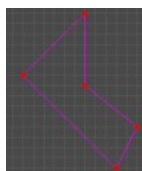
Drawing a Polygon

You can draw an individual polygon in Geometry Edit mode just as you can draw an individual brush in Brush Edit mode. The resulting polygon you create is considered a brush by DEdit, but it is actually only a face.

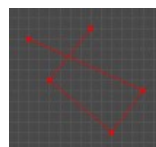
To draw a polygon

1. In **Top**, **Front**, or **Left** view, place your mouse over a grid intersection and press the **Space Bar**.
2. Move your mouse to the next vertices and press the **Space Bar** again.
3. Repeat the previous step until your polygon is complete and the line color changes to yellow.

Note: If the color of the lines changes to violet, then your polygon has a concave face. If the color of the lines changes to red, then you have crossed your lines and your polygon is invalid. You should always avoid invalid polygons, and avoid or correct concave polygons.



Concave Polygon



Invalid Polygon

[Top](#)

Moving Vertices

You can move vertices individually or as a group. The Top, Front, or Left view you move the vertices determines the X,Y, and Z the vertices are moved in.

To move a vertex

1. In the **Top**, **Front**, or **Left** view for the direction to move, place your mouse cursor over the vertices to move.
2. Hold the **M** key and the **LEFT MOUSE** button.
3. Move the mouse to the new position.
4. Release the mouse and the **M** key.

[Top](#)

Deleting Faces

Deleting faces is often used to break holes in split brushes. This allows you to create doorways and openings in world geometry.

To Delete a Face

1. In **Brush Edit** mode, select the brush containing the face to delete.
2. Press **CTRL + G** to switch to **Geometry Edit** mode.
3. In the **Perspective** view, move your mouse cursor over the face so that it's edges are highlighted.
4. Press the **Page Down** key.

Note: If you enter **Geometry Edit** mode without a brush selected, you can delete any face using this procedure. However, you risk selecting and deleting faces behind the desired face. It is always safest to select the brush in **Brush Edit** mode first prior to entering **Geometry Edit** mode. This prevents geometry editing operations from working on any brush other than the selected brush.

[Top](#)

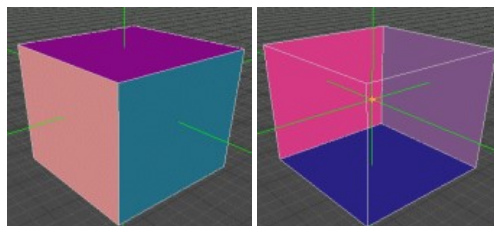
Setting Normal Visibility Direction

Normal visibility controls the direction the polygon is visible in the game. Polygons are only visible from one direction, and any player looking at the opposite side of a polygon sees directly through that polygon. In other words, the term "normal" refers to the side of the polygon that the engine renders. When you set the normals in Brush Edit mode, you set the normals for all of the polygons on a brush, but to set the normals for individual polygons, you must enter Geometry Edit mode.

To set the normal direction

1. Move your mouse cursor over the desired face.
2. Press the **F** key to flip the polygon's normal visibility.

The following image shows the same cubic brushes with normal visibility flipped. The left image shows normals facing out, and the right image shows normals facing in.

[Top](#)



Working With KeyFrames

When considering how to animate WorldModels in DEdit, many Level Designers choose to use KeyFrames to create paths for their object. KeyFrames allow you to lock world geometry bound to a WorldModel onto a path. When activated, the WorldModel and all its bound objects move at a set speed from key to key along the path. You can send commands triggered by the arrival of the object from each key point along the path, and you have control over many properties of the animated object. KeyFrames can control more than just WorldModels. Cameras, explosion debris, special effects, and living props are all examples of objects that often get KeyFramed in a level.

This section contains the following topics explaining how to insert and use KeyFrame objects:

- [General Steps to KeyFraming](#)
- [KeyFraming a WorldModel](#)
 - [Modifying a Key Path](#)
 - [Orienting a WorldModel](#)
 - [Using KeyFramer Commands](#)
- [Troubleshooting KeyFrame Animations](#)

General Steps to KeyFraming

This topic lists the general steps required to KeyFrame an object. The following steps are covered in more detail in successive topics. If you are familiar with KeyFraming, then you can use this list to quickly start keyframing an object.

You must perform the following steps to animate an object using KeyFrames:

- Bind the Geometry to a WorldModel object.
 - Insert a KeyFramer object.
- Set the KeyFramer ObjectName property to the name of the WorldModel object to animate.
- Insert the Key objects that define the path the WorldModel moves along during the animation.
- Ensure all Key objects are numbered sequential order along the KeyFrame path. (Key0, Key01, Key02, etc).
- Set the KeyFramer BaseKeyName property to the name of the first Key, truncating the number. (For example, if the first key is Key00, then you must use Key, without the number, as the BaseKeyName).
- Bind all Key objects to the KeyFramer object, and set the KeyFramer object to Path.
 - Adjust the Bezier curve of each Key, and set the TimeStamp for each key.

For more specific information on each of these steps, see the topics below.

[Top](#)

KeyFraming a WorldModel

This topic provides procedures for KeyFraming a simple WorldModel object. This topic assumes that you have already created the world geometry you want to animate.

KeyFraming involves three primary object types. A WorldModel (or other game object) to bind the geometry to. A KeyFramer object to control the animation. And a series of Key objects to move the animation between.

To KeyFrame a WorldModel

1. Bind the world geometry to a WorldModel object.
 - A. Select the world geometry.
 - B. Right-click the world geometry, move to Selection, and then click Bind to Object.
 - C. Select WorldModel, and then click OK.
- D. In the Properties tab for the WorldModel object, set the IsKeyFramed property to TRUE.
2. Insert the KeyFramer and Key objects.
 - A. Use the X key to position the marker near the WorldModel.
 - B. Press CTRL + K, select KeyFramer, and then click OK. The KeyFramer object appears.
 - C. Use the X key to position the marker where you want the animation occur.
 - D. Press CTRL + K, select Key, and then click OK. A single Key object appears.

Note: In the Properties tab for the first Key object you insert, renumber the object's name to zero. For example, if the Key object's name is Key1, name the object Key0. If the Key object's name is MyKey1, rename the object MyKey0. The first Key in a KeyFramed animation must start with zero, and all successive keys must follow the numbering linearly. For example: Key0, Key01, Key02, etc. The KeyFramed animation moves between each of the

keys in numeric order.

- E. Each Key object represents a point in the animation path. Repeat steps C and D until you have placed all of the Key objects you want for your animation.
 3. Set the KeyFramer properties.
 - A. Select the KeyFramer object, and then click the Properties tab.
 - B. In the ObjectName property, type the name of the WorldModel from step one.
 - C. In the BaseKeyName property, type the base name of the Key objects.
For example, if Key0 is the first Key object, then Key is the base name.
 - D. Set the StartActive and Looping properties to TRUE.
 - E. In the Properties tab for each Key object, set the TimeStamp to specify the number of seconds it takes to get from the previous node to the selected node.
 4. Bind the Key objects and set them to create a path.
 - A. Select all of the Key objects used in the animation.
 - B. In Nodes view, right-click the KeyFramer object, and then click Move Tagged Nodes Here.
 - C. In Nodes view, right-click the KeyFramer object, and then click Set Path.

This concludes the creation of your KeyFramed animation. Save and process your level. You should see your WorldModel moving between the Key objects. To modify the Bezier curves between the Keys, making them curved, read the next topic in this section.

[Top](#)

Modifying a Key Path

Each Key object has two possible Bezier curve points. One point controls the curve before running into the Key object, and one point controls the curve running away from the Key object. Each point is represented by an arrow. The first and last Key objects in the path only have one point. Use the following procedure to alter the Bezier curves between each of the Key objects in your path.

To modify a Key path

1. Select a Key object.
2. Hold the P key, and then click and drag the arrow from the center of the Key object.
3. Repeat step two to move the other arrow (as needed) away from the Key object.
4. Continue holding the P key, and drag the arrows until the curve appears in the correct shape.
5. Repeat this procedure for each of the Key objects in the animation.

[Top](#)

Orienting a WorldModel

There are three primary ways of orienting the direction a KeyFramed WorldModel faces.

1. Target an object to face using the TargetName property. The forward facing vector of the WorldModel always attempts to point at the specified target.
2. Offset the forward vector of the WorldModel targeting an object by setting the TargetOffset property. The forward facing vector of the WorldModel always attempts to point at the specified target but is offset from the target by the specified vector.
3. Face the forward vector of the path by setting AlignToPath to TRUE. The forward facing vector of the WorldModel always attempts to align itself (become parallel) to the Bezier curve between each Key object.

[Top](#)

Using KeyFramer Commands

You can send the following command messages to a KeyFramer object from any object capable of sending commands.

ON—Activates the KeyFramer and starts moving the WorldModel.

OFF—Stops the WorldModel at its current position.

PAUSE—Causes the WorldModel to pause at its current position.

RESUME—Causes the WorldModel to resume movement after pausing or stopping.

FORWARD—Causes the WorldModel to move forward.

BACKWARD—Causes the WorldModel to move in the reverse direction.

REVERSE—Causes the WorldModel to move in the reverse direction.

TOGGLEDIR—Reverses the current direction of the WorldModel.

GOTO <key>—Specifies a Key object in the KeyFrame to jump the WorldModel to. The WorldModel jumps immediately to the specified Key object and stops.

MOVETO <key> [destination cmd]—Specifies a Key object in the KeyFrame to move the WorldModel to. Also allows you to specify a command to send the KeyFramer when the WorldModel reaches the Key object. The WorldModel moves from its current position to the specified key, and then stops or follows the optional command. For example: `msg KeyFramer0 (MOVETO Key01 (msg KeyFramer0 RESUME))` causes the WorldModel to move to Key01 and then resume its normal course.

TARGET <object>—Specifies a new TargetName object.

CLEARTARGET—Removes the TargetName object from the TargetName property.

TARGETOFFSET <x> <y> <z>—Sets the offset when using TargetName to control the forward facing vector of the KeyFramer.

VISIBLE <bool>—Sets the visibility of the WorldModel.

SOLID <bool>—Sets the solid/intangible state of the WorldModel.

HIDDEN <bool>—Sets the visibility of the WorldModel.

REMOVE—Removes the KeyFrame object from the level.

[Top](#)

Troubleshooting KeyFrame Animations

The following information contains common questions and answers regarding the use of the KeyFramer object. Use these questions to help troubleshoot your animation if it is not working correctly.

The WorldModel is not moving.

- Ensure the ObjectName property in the KeyFramer object specifies the correct WorldModel object.
- Ensure the KeyFramer object's BaseKeyName property is correct and does not contain a number.
 - Ensure the Timestamp in each Key object has a value.
- Ensure all Key objects contain the same base name, and that the numbers proceed linearly. For example, Key0, Key01, Key02, etc. The first Key object MUST start with zero. All successive Key objects must be numbered 0X where X is the next number in a linear numeric set.
 - Ensure the IsKeyFramed property in the WorldModel object is set to TRUE.

The Animation is too fast or too slow between some Key objects.

- Adjust the Timestamp property of the Key objects. The Timestamp property specifies the amount of time in seconds the WorldModel takes to move from the previous Key object to the currently selected Key object.

The WorldModel jumps when the animation starts

- Try moving your WorldModel object closer to the first Key object, or move the first Key object closer to the WorldModel.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Working With Nodes

The **Nodes** tab in DEdit allows you to easily control all of the brushes and objects in your level. Using the **Nodes** tab you can group brushes and objects, select any brush or object (hidden or otherwise), place brushes or object in containers, and freeze brushes or objects. You can also use the **Nodes** tab to attach a brush or object to another brush or object.

This section contains the following topics and procedures for using the DEdit Nodes tab:

- [About the Nodes Tab](#)
- [Adding a New Container](#)
- [Moving Selections to a Container](#)
- [Hiding and Unhiding Selections](#)
- [Freezing and Unfreezing Selections](#)
- [Naming and Renaming Objects](#)
- [Naming Groups of Objects](#)
- [Using Containers to Organize Your Level](#)

About the Nodes Tab

As your levels grow larger and more complex, it becomes increasingly important to have ways of grouping areas of your map into subsections for easier selection and management. The Nodes tab is designed to help you do this, as well as to manage doors, windows, and other objects that are composed of a brush bound to a simulation object, rather than a simulation object or brush on its own.

Note: Binding brushes to objects is discussed in more detail in the [Adding Simulation Objects](#) section.

Your level may also have groups of simulation objects like AI paths or scripting paths that would benefit from being in groups so that you can easily find them.

The Nodes tab is a tool for organizing your level, for viewing its structure and for quickly locating objects in it. As mentioned previously in our section about selection, the nodes view reflects objects you select in the other views and vice versa. If you select a brush or object in one of the viewports, it becomes selected in the Nodes view. If you select a node, it will be highlighted in the viewports as well.

Go to the Nodes tab and look around at your level. What you'll see is that the tab lays out your level not as it's visually organized, but as a tree, showing the connections and relationships between objects in the level. At the very top of the tree is the root node. This object has no properties, and selecting it selects all of the child nodes below it, but doesn't actually select the root. Selecting a node that contains other nodes always selects the contained nodes, but usually it also selects the parent node as well. In this case, the root node is special. You cannot delete, change, move or rename the root node, so there is no reason to select it other than as a quick way to select all objects in the level.

You should also notice that there are multiple levels in the tree of nodes. Some nodes are hanging off of nodes other than the root node. Most of the objects that contain other nodes probably have the name Container. Container objects are special. When an object is contained as a child node below another object, it inherits the parent's properties. This is called binding and will be talked about in the section on objects. The reason that containers do not effect the objects that they hold is because Containers don't have any properties. They only exist to hold and organize the other objects in your level.

As mentioned before, the Nodes tab also shows the relationship between the objects in the level, which can be very important in the case of objects.

The **Nodes** tab operates in a tree view similar to the Windows explorer. Each container in the **Nodes** tab view can contain multiple objects or brushes. If the top level container is selected or frozen, then all of the brushes and objects within the container are also selected or frozen.

The following image shows a sample of a typical Nodes tab:



The two checkboxes (Select/Freeze boxes) to the left of each folder or item in the Nodes tab controls the selection and freeze state of that item.

 The **left** box controls the selection, and the **right** box controls the freeze state.

The view selections above the node window control how the names are displayed.

- **Node name**—lists each node in the tree by its designated name.
- **Class type**—lists each node in the tree by its class type (such as light, or brush).
- **Both**—lists each node by its Node name followed by its Class name in parenthesis.

Each folder in the Nodes list represents a container object. Containers are not actually nodes, they are simply named folders that contain a sequence of nodes. When you add new nodes, containers are automatically created for you with default container names. To better

organize the contents of your world you should provide unique descriptive names for each of your containers, and fill them with the brushes and objects that match the description you have given.

About Parent-Child Relationships



Brushes bound to objects appear below the parent object they are bound to (as displayed in the previous image). This not only **indicates** the parent-child dependency, it also **creates** the parent-child dependency. If you move a brush beneath an object capable of taking a brush as an attachment, then DEdit attempts to bind the brush to that parent object. This binding is useful if you have several brushes you want to bind to the same object. For example, if you have a WorldModel object setting a texture to transparent and you want several nearby brushes to have transparent textures, you can click and drag the brushes to the WorldModel object name. The brushes moved beneath the WorldModel object automatically become child objects of that WorldModel object. For an alternative way of performing the same movement operation, see [Moving Selections to a Container](#).

[Top](#)

Adding a New Container

You can add a new container to the Node view at any time. New containers are useful ways of keeping track of which objects and brushes are associated for each element in your level. For example, if you had a swimming pool section, creating a SwimmingPool container to hold all of the brushes and objects in that section would enable you to easily locate and alter the brushes and objects associated with the swimming pool.

To add a new container

1. In the **Nodes** tab, right-click the container to be the parent container, and then click **Add Container Node**.
2. Type the name for the new container, and then press **Enter**.

[Top](#)

Moving Selections to a Container

You can move any number of selected brushes and objects to a container or to a parent node.

To move selections to a container

1. Select the brushes and objects to move. You can do this in **Nodes** view by holding down the **Shift** key and selecting the left box for each item to move. Alternatively you can hold down the **Shift** key and select the items to move in any of the DEdit view windows.
 2. Right-click the node to be the parent container, and then click **Move Tagged Nodes Here**.
All of the selected nodes get moved to the parent container.
- Alternatively you can always click and drag a node to another container or parent node, although this process becomes more difficult as the number of nodes in your level increases.

Note: After moving your selections, they are still selected. If you are done working with the selections as a group, make sure to deselect them using the **U** key.

[Top](#)

Hiding and Unhiding Selections

Hiding selections makes them invisible in DEdit, but does not prevent them from alteration if they are selected. While you cannot select hidden brushes and objects in any of the view windows, you can select hidden objects in the Nodes tab, and if you select the parent of a hidden object, then all children including the parent get selected.

Hiding brushes and objects is useful if you want to perform adjustments on brushes and objects obscured by brushes and objects that you do not want to change. Hiding also reduces the amount of material DEdit needs to render and display, and therefore allows it to run faster in worlds with large numbers of polygons and large amounts of complexity.

Note: When a node is hidden its Select/Freeze boxes change from a solid outline to a dashed outline.

 —Hidden node.

 —Unhidden node.

To hide a selection

- The Standard method for hiding a selection is to press the **⌘** key, hiding all of the currently selected items.
- The Nodes tab's right-click menu allows you to perform the following additional operations on nodes:
 - **Hide Node**—Hides the currently selected node only.
 - **Hide Node and Children**—Hides the currently selected node and all child nodes.
 - **Unhide Node and Children**—Unhides the currently selected node and all child nodes.
 - **Hide All Nodes**—Hides all nodes in the Nodes tab.
 - **Unhide All Nodes**—Unhides all nodes in the Nodes tab.

[Top](#)

Freezing and Unfreezing Selections

Frozen selections still appear in the DEdit view windows, but you cannot select or change frozen nodes from the view windows. Additionally, you cannot change frozen selections from the Properties tab.

Freezing selections is useful when you are performing changes to a level and want to ensure that certain brushes and objects do not get altered. Additionally, because you can "select through" frozen brushes, freezing brushes that are in the way of brushes and objects you want to edit gives you access to them without the risk of making unwanted changes to the frozen brushes.

To freeze or unfreeze a selection

- In the **Nodes** tab, click the right box of the brush or object to freeze. A blue check appears in the box.
If the node is a container, then selecting the container also freezes all child nodes in the container.

[Top](#)

Naming and Renaming Objects

Many nodes have somewhat cryptic names like Container or Brush1. That's easily fixed, though. By default, DEdit creates object names based on the class of the object. When a new brush is added to DEdit, its name is automatically set to Brush#, where # is a unique number. DEdit tries to give each object a unique name to make each one easier to find.

If you don't like the names DEdit gives to objects or want to change them for other reasons, you can change the name of a node just like you'd change it in Windows Explorer.

1. Click on one of the node names.
2. Press F2 or single-click once again on the node's name. You'll see the name become highlighted, just like a file or folder in Windows.
3. You can now type a new name.
4. When you press ENTER the new name is applied.

[Top](#)

Naming Groups of Objects

Since the regular viewports are linked to the Nodes view, you can also rename objects in their Properties dialog, which is useful when you want a group of objects to have the same name.

To name groups of objects

1. Using the **Perspective** view, select all of the brushes to group.
2. Select the **Properties** tab.
3. In the **Name** field, type the new name for the group, and then switch to the **Nodes** tab.

[Top](#)

Using Containers to Organize a Level

Using containers to organize your level is a very good idea. Imagine a level with 200 brushes, 50 lights and 75 enemies and you can imagine

why. Using containers to group brushes by room, lights by area and enemies by squad turns your level from a massive jumble into a logical order.

Note: Now that you know about container nodes and renaming things, you may be tempted to carefully name each brush and object, or to build six layers of containers for all of your items. Don't spend more time than necessary on organizing things. It's wise to sort out your level in a general way to make things easy to find and to help others who may need to work on your level. However, it's easy to go overboard as well. Spending hours laying out your level in the Nodes tab is as much of a waste as the hours you may spend searching through an unorganized level to find a brush or simulation object. Be careful to find a balance.

You can have as many container nodes as you want in your level. Since containers are removed from the level when it's processed, they have no effect on the actual operation of the level in the game. They're present exclusively to help you structure your level within the editor.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Working With Objects

This section explains what objects are, how they operate, and how you can use them. Not all objects are listed or described in this section, although there are some specific examples. The best way to learn about objects is to insert them into your game and manipulate their properties to see how they operate.

This section contains the following object insertion and operation topics:

- [About Jupiter Objects](#)
- [Using Object Commands](#)
 - [Inserting an Object](#)
 - [Inserting a Light Object](#)
- [Inserting a LightGroup Object](#)
 - [Inserting a Decal Object](#)
 - [Inserting a DirLight Object](#)
 - [Inserting a Polygrid Object](#)
- [Inserting a GameStartPoint Object](#)
- [Inserting a VolumeBrush Object](#)
 - [Inserting a Spawner Object](#)
 - [Inserting a SpecialFX Object](#)
 - [Inserting a SoundFX Object](#)
- [Inserting a WorldProperties Object](#)
- [Inserting a PlayerTrigger Object](#)
- [Inserting a ScaleSprite Object](#)
- [Inserting a TeleportPoint Object](#)

About Jupiter Objects

Objects are sections of game code you can manipulate using Jupiter Tools. They can represent physical objects, special effects such as sounds and lights, or abstract things such as paths for a camera to follow. The **Properties** dialog boxes of each object control the essential information used by the object's code to produce the desired effect. Jupiter comes complete with a wide variety of objects ready for use, but these objects may not necessarily fit your needs. During the creation of your game, you will probably create your own objects, or at least modify the existing objects. For more information about creating your own objects, see the [Programming Guide](#).

Although you can insert an object in any mode, you can only select and move objects in **Object Edit** mode. Press **CTRL + H** to enter **Object Edit** mode at any time.

Alternatively, you can also press the **Object Edit** button  to enter **Object Edit** mode.

Using Object Commands

Most objects in DEdit are capable of sending messages (commands) to other objects. Sending commands is an important part of creating a level because it allows objects to interact with each other. Doors and triggers are particularly dependent upon message commands because they use the commands to send instructions to other objects in the game. Not all objects can send or receive commands. Lights, for example, do not take commands.

For more detailed information about commands and player messages, see the **Commands.doc** and **PlayerMessages.doc** files in the LithTech\LT_Jupiter_Src\Specs\GameSystems folder.

When you enter a command in the **Commands** property of an object (in the **Property** tab for the object), if the command is invalid the **Debugger** window opens and displays the commands that the object can take.

Note: Commands are not case sensitive. Commands can also be composed of multiple commands separated by semicolons.

The following list describes the syntax for commands (angle brackets indicate individual entries and should not be included in your command string):

- Msg <ObjectName> <ObjectCommand>
 - Delay <DelayTime> <Command>
- DelayID <CommandID> <DelayTime> <Command>
- Rand <command1 percent> <command 1> <command 2>
 - Loop <min delay> <max delay> <command>
- LoopId <command id> <min delay> <max delay> <command>
- Repeat <min times> <max times> <min delay> <max delay> <command>
- RepeatId <command id> <min times> <max times> <min delay> <max delay> <command>
 - Abort <command id>
 - Int <variable name> <value>
 - Set <variable name> <value>
 - Add <variable name> <value>
 - Sub <variable name> <value>
 - If <condition> THEN <commands>
 - When <condition> THEN <commands>

Use of Operators:

Two of the above commands (**If** and **When**) have a condition argument where you can compare a variable to a number value or another variable. The condition argument can be broken down into a single expression or a series of expressions. An expression has 3 arguments, the first parameter followed by an operator followed by the second parameter. Depending on the operator, either of the parameters can be another expression. Every operator can be expressed by either a mathematical symbol or the English phrase equivalent.

- <Variable> Equals (==) <Value>
- <Variable> Not_Equals (!=) <Value>
- <Variable> Not_Equals (!=) <Value>
- <Variable> Greater_Than (>) <Value>
- <Variable> Less_Than (<) <Value>
- <Variable> Greater_Than_Or_Equal_To (>=) <Value>
- <Variable> Less_Than_Or_Equal_To (<=) <Value>
- <Expression1> And (&&) <Expression2>
- <Expression1> Or (||) <Expression2>

Use of Parenthesis:

You must use parenthesis to group multi-term arguments together. This is most often necessary when a command is an argument to another command (for example, Repeat 2 5 0.25 2.0 (msg prop0 fire)). In this example **msg prop0 fire** is a single argument to the command **Repeat**. In order for the **Repeat** command to correctly parse the string **msg prop0 fire** as a single argument, the individual terms must be grouped together using parenthesis.

Possible Commands:

The following list contains some of the command messages that you can send between objects. Not all objects can use these commands, and DEdit automatically displays incorrect or invalid commands in the debugging window when you enter them.

- ACTIVATE
- ATTACH <object name(s)>
- ATTACHCLASS <class name>
 - BACKWARD
- CLEARTARGET
 - DESTROY
- DETACH <object name(s)>
 - FORWARD
- FOV <FovX> <FovY> <time>
 - GOTO <key>
- HIDDEN <bool>
 - LOCK
- MOVETO <camerapoint>

- MOVETO <key> [destination cmd]
 - OFF
 - ON
 - PAUSE
 - REMOVE
 - RESUME
 - REVERSE
 - SKIP
 - SOLID <bool>
 - TARGET <object>
- TARGETOFFSET <x> <y> <z>
 - TOGGLEDIR
 - TRIGGER
- TRIGGERCLOSE
 - UNLOCK
- VISIBLE <bool>

[Top](#)

Inserting an Object



Objects are always inserted at the current marker position in your world. Use the **X** key in any view to control the location of the marker. After objects are inserted you can move them in the same manner you would move a brush or polygon.

To insert an object

1. Position the marker where you want to insert the object.
2. Press **CTRL + K** to open the **Select Object Type** dialog box.
3. From the **Class Types** list, select the object to insert and click **OK**.

Alternatively, you can select from the **Recent** list to insert a new instance of any object you have previously inserted.

[Top](#)

Inserting a Light Object



All light objects cause illumination of your world or of objects in your world. The light object itself does not appear in your game (there is no ball of light hovering in the air), but the effect the light would have on the environment if it did exist results in the illumination of brushes in the range of the light that are set to respond to it (all brushes respond to light by default, but you can adjust this in the properties for each brush).

Creating correct lighting is an artform that requires experience and time to master. There are many different types of lighting and lighting effects. The following procedure only outlines how to insert and adjust a simple **Light** object.

To insert a light object

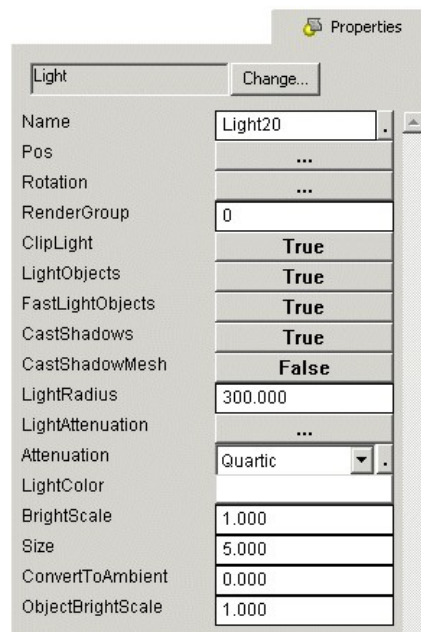
1. Position the marker where you want to insert your light.
2. Press **CTRL + K** to open the **Select Object Type** dialog box.
3. Press the **L** key to go directly to the **Light** object.
4. Press **Enter** to insert the selected **Light** object.

To adjust a Light object

1. Select the light you wish to adjust.
2. In the **Project Window**, click the **Properties** tab. If the **Properties** tab is not visible, on the **View** menu, click **Project Window**.

There are many different properties for Light objects. This topic explains a few of the primary adjustments you may wish to make. For additional information, In DEdit, click the name of the property for a pop-up box containing a brief description of that property's function.

The following image shows a typical Light object properties tab:



Name—Specifies the Name of your object. Using unique names for objects allows you to easily identify them in the **Nodes** tab, and also allows you to send messages from other objects.

Pos—Specifies the position of your light in world coordinates.

Rotation—controls the rotation of your light. This is not of much use for a standard light, but is important in direction lighting (such as the DirLight).

ClipLight—If True, cliplight causes world geometry to stop the light. If False, the light passes through world geometry and illuminates everything in its radius.

LightObjects—Determines if the light illuminates models.

FastLightObjects—If True, renders the light on models quickly, but only by adding light to the model's Ambient component, which is non-directional. If false, then the light acts as a D3D hardware light, which renders the light on models directionally and with more detail, but requires more graphics processing power.

CastShadows—Determines if brushes lit by your light cast shadows on other brushes.

LightRadius—Determines the radius of the light, or the complete distance the light will travel before ceasing. In Object mode the radius of the light appears as a blue circle.

Attenuation—Determines the equation used to control the intensity of the light as distance from the source increases.

Light Color—Specifies the color of the light.

BrightScale—Specifies the intensity of the light in respect to the brush surfaces effected by the light.

Size—Specifies the size of the light. Do not use this property. It is obsolete.

ConvertToAmbient—Specifies the amount of light (from 0 to 1) that gets applied to models as ambient light rather than direct light. This allows the light to "wrap around" models and causes them to appear more fully illuminated. This effect gives hardware lights a warmer effect on models.

ObjectBrightScale—Specifies the intensity of the light in respect to model surfaces effected by the light independently of the world, allowing you to match your model and world lighting more flexibly.

[Top](#)

Inserting a LightGroup Object

LightGroup objects allow you to create dynamic lighting that you can turn on or off. When your level is processed, a version of the brushes gets rendered in both on and off states. By turning LightGroup objects on or off you can toggle between the different renderings of the brushes touched by the lights in the LightGroup. LightGroups are commonly used for almost all lighting that turns on or off.

Note: LightGroup objects cannot turn on or off a sprite.

To insert a LightGroup object

1. Position the marker where you want to insert your LightGroup.
2. Press **CTRL + K** to open the **Select Object Type** dialog box.
 3. Select the **LightGroup** object.
 4. Press **Enter** to insert the selected **LightGroup** object.
5. Select the lights (DirLight, ObjectLight, or Light) to turn on or off with the group.
6. In **NodesView**, right-click the **LightGroup** object, and then click **Move Tagged Nodes Here**.

To turn a LightGroup on or off

- From any object send the message: `msg <LightGroupName> ON` or `msg <LightGroupName> OFF`.

LightGroup objects can take the following commands:

- TOGGLE
 - ON
 - OFF
- COLOR <value> or <RValue> <GValue> <BValue>

For example `msg LightGroup0 (Color 10 256 10)` turns all lights bound to LightGroup0 to a green hue.

[Top](#)

Inserting a Decal Object



Decal objects project a texture onto world geometry (brushes and brushes bound to WorldModel objects). Decal objects operate like a can of spraypaint. They can blend on the brushes they hit either translucently, additively, or not at all.

Two main uses for decals:

1. To conceal the seams between two different textures.
2. To add features to existing textures such as posters or scorch marks. This typically involves using an alpha mask to determine how the decal blends with the texture.

You can make decals translucent, or apply them additively so that they brighten the texture they are applied on. Performing translucent or additive blending requires binding the Decal object to a WorldModel object, and then using the WorldModels's blend mode settings. You do not need to bind Decal Objects to WorldModel objects if they do not require translucency.

Decal object restrictions:

- Decals used on brushes with Gouraud or flat shading cannot use translucency.
 - Brushes with lightmaps cannot blend with decals.
- Decals are not intended to wrap or tile. To make a tiling pattern you must use multiple decals or develop your own custom solution in game code.

To insert a Decal object

1. In DEdit, position the marker where you want to insert the Decal.
2. Press **CTRL + K** to open the **Select Object Type** dialog box.
3. In the **Class Types** list, select **Decal** and then press **Enter**.
4. In the Properties tab, set the following properties for the Decal:
 - **Texture**—Select the .DTX file the Decal projects.
 - **ProjectDims**—Set the scale and depth of the projected texture.
5. Use the **N** key to rotate the Decal object, aiming the Decal as desired.
 6. Process your world and test the Decal.

[Top](#)

Inserting a DirLight Object



DirLights are similar to standard Light objects except that they are directed. In addition to having a LightRadius, DirLights also have a field of view (FOV) property for specifying the angle of the lighting effect. The intensity of DirLight decreases away from the center in specified amounts, allowing you to create spotlight lighting, or just to light a specific segment of your world.

To insert a DirLight object

1. In DEdit, position the marker where you want to insert your light.
2. Press **CTRL + K** to open the **Select Object Type** dialog box.
3. In the **Class Types** list, select **DirLight**.
4. Press **OK**.

To adjust a DirLight object

1. In the **Top**, **Front**, or **Left** view, select the DirLight.
2. Press and hold the **N** key, and move the mouse to rotate the DirLight to the desired direction. The FOV for the DirLight rotates around the current marker.
3. In the **Properties** tab, set the properties for the DirLight object. These properties are similar to the Light object, however there are a few additional properties:

InnerColor—Specifies the color used on the central portion of the DirLight.

FOV—Specifies the angle (spread) of the DirLight

[Top](#)

Inserting a Polygrid Object



Polygrids are flexible flat sheets of polygons that deform according to parameters you specify. Polygrids can mimic effects such as water, flags, or billowing sheets. The following procedure creates a water effect using a polygrid.

To create water using a Polygrid object

1. In DEdit, draw a brush the size and volume of your water. When the character enters the boundaries of this box, they enter water and the game code associated with the water volume gets activated.
2. Right-click the water brush, move to **Selection**, and click **Bind to Object**.
3. In the **Class Types** list, expand the **VolumeBrush** class, select **Water**, and press **Enter**.
4. Press **CTRL + K**, select **Polygrid**, and press **Enter**.
5. In the **Properties** tab for the Polygrid, select the **SpriteSurfaceName** (.spr file) determining the surface of the polygrid if you want texture for your water. Alternatively you can use the **Color1** and **Color2** properties to create a polygrid using colors only. For additional information on creating a sprite for the polygrid object, see the next procedure, or the [Working With Sprites](#) section.
6. In the **Dims** property, set the **X**, **Y**, and **Z** dimensions of the polygrid. **X** controls the length of your polygrid, **Y** controls the wave height of your polygrid, and **Z** controls the width of the polygrid.
7. In the **Rotation** property, rotate your polygrid to face the surface in the desired direction (typically up for water, but you can set your polygrid to any angle).

Process and test your world to ensure your polygrid is operating as you intended. You may require several tries to get the polygrid to the correct dimensions and angle.

To create a sprite for the polygrid texture

1. In the sprite specified in **SpriteSurfaceName** (step 5 of the previous procedure), give the sprite one texture and a frame rate of one. The texture you select is only used as reference (its Command String gets applied) so the actual image on the texture is not important. You can simply create a copy of an existing texture with a unique filename.
2. In the **Texture** tab, open the texture file you selected in the sprite, and set the **Command String** property as follows:

```
EnvBumpMap <SurfaceTexture.dtx> <BumpMap.dtx>
```

All files you specify must contain full file path names. The <SurfaceTexture.dtx> file gets displayed on the surface of the polygrid, and the <BumpMap.dtx> file controls the environment bump mapping for the texture. For example, the following Command String sets the texture used by the sprite to display the UW_SubBay texture on the polygrid with bumpptest05.dtx as the environment map:

```
EnvBumpMap TexFx\Cubic\UW_SubBay.dtx TexFX\Normal\bumpptest05.dtx
```

[Top](#)

Inserting a GameStartPoint Object



GameStartPoint objects determine where in the level your character starts the game. You must always have at least one GameStartPoint object in your level.

To insert a GameStartPoint object

1. In any view, position the marker (using the **X** key) where you want the player to start the level.
2. Press **CTRL + K**, select **GameStartPoint**, and press Enter.
3. In the **Properties** tab, press the **Rotation** button.
4. Adjust the **Yaw**, **Pitch**, and **Roll** to determine the direction the character faces when spawning. The blue line coming from the **GameStartPoint** object indicates the facing direction of the character.
5. Press **OK**.

Process and test your world to ensure the character is facing the correct direction and does not spawn to an immediate death due to the location of the **GameStartPoint** object. A space of at least 8 units of clearance above, below and to each side of the start point allows the character sufficient space to spawn without dying.

[Top](#)

Inserting a VolumeBrush Object

Volume brush objects control physics, special effects, and triggers associated with a specific area. Volume brushes are not typically visible in a level, but rather alert the engine to perform specific operations on the character when they enter the area of the volume brush. Water is a standard example of a VolumeBrush. When a character enters an area surrounded by a Water VolumeBrush, the engine notes they are in water and treats the character with all of the physics and properties associated with being in water.

To insert a VolumeBrush object

1. In **Brush** mode, draw the brush you wish to associate with a volume effect. This brush defines the area of the volume effect.
2. Use the **X** key to position the marker on the surface of the desired brush.
3. Right-click the brush, move to **Selection**, and then click **Bind to Object**.
4. In the **Class Types** list, expand the **VolumeBrush** class, and select the type of volume to apply.
5. Press **OK**. The **VolumeBrush** object gets inserted at the current marker.
6. In the **Properties** tab, adjust the properties associated with the **VolumeBrush** object to determine the specific effects that occur when the character enters the area of the **VolumeBrush**.

Process and test your world. Enter the volume brush area and the character should experience the effect you associated with the **VolumeBrush** object.

Types of Volume Brushes

There are many different types of volume brushes available using the game code supplied with the Jupiter engine. The following table lists some of the common volume brushes and their effects.

Volume Brush	Effect
Burn	Creates a burn effect and does burning damage to the character when they enter the volume of the brush.
ColdAir	Creates a cold effect, shaking the screen and doing cold damage to the character when they enter the volume of the brush.
CorrosiveFluid	Similar to burn but includes liquid splashing sound effects.
Electricity	Creates an electrical effect and does electrical damage to the character when they enter the volume of the brush.
EndlessFall	Creates an echoing scream and a spinning effect. Additionally this volume brush kills the character (a falling crush death) when they enter the volume of the brush.
Filter	Applies a sound filter inside the volume allowing you to apply filters to ambient sounds and change the sound the character hears when they are inside the volume.
FreezingWater	Similar to water, but additionally the character takes cold damage while they are inside the volume of the brush.
Gravity	Creates a physics effect allowing you to alter the viscosity, friction, and gravity inside the volume of the brush. Viscosity controls how difficult it is to move through the brush. Friction controls the slickness of the surface in the brush. Gravity controls the character's ability to jump in the volume of the brush.
Ice	Creates a slick surface causing the character to move when they enter the volume of the brush. Fog and color properties for this volume brush are currently disabled.
Ladder	Creates a surface the character can climb.
PoisonGas	Creates a poison gas effect, coloring the characters view and warping the screen. Additionally pigeons appear to indicate a side effect of the nerve gas.
SafteyNet	Intended to create a surface to protect the character from damage during a fall. This volume brush is currently disabled.
Water	Creates a water effect that does not damage the character. While in the volume brush the sound filter switches to underwater, the character's view is switched to blue, and the character can move in all directions.

Weather	Creates a rain or snow effect. Use of this volume brush has been discontinued in favor of the SnowVolume under the VolumeEffect class.
Wind	Similar to Ice. Pushes the character in a specified direction.

[Top](#)

Inserting a Spawner Object



Spawner objects insert template objects into the game during runtime. Spawner objects are triggered by message commands using the `msg <Spawner object name> SPAWN` syntax. Spawner object can only spawn template objects.

Template objects do not exist at runtime, but appear as normal when they are spawned. The Template object specified in the Spawner object appears in the game at the location of the Spawner object when the Spawner object is triggered. Any object except world geometry can be specified as a template object.

Use the following procedures to create template objects and specify them in a Spawner object.

To create a Template object

1. In DEdit, select or insert any object you want to make a template.
2. In the **Properties** tab for the object, set **Template** to True.
3. Set any other properties you want for the object in the **Properties** tab.

The object is now a template and does not appear in the game until it is spawned.

To insert a Spawner object

1. In DEdit, use the **X** key to position the marker where you want the Template object to spawn.
2. Press **CTRL + K**, select **Spawner**, and press **Enter**.
3. In the **Properties** tab for the **Spawner** object, go to the **Trigger** box and type the name of the template to spawn.
4. In the InitialCommand box, type a command to send when the Spawner object is triggered. Leave this field blank if you do not wish to send a command when the object is spawned.

To trigger the Spawner from another object, use a `msg <Spawner object name> SPAWN` message command.

[Top](#)

Inserting a SpecialFX Object



The LithTech Jupiter system comes complete with a ready-made package of special effects objects you can insert into your game. Developers can also create their own special effects objects and make them available for Level Designers to insert through the DEdit tool. For information about creating special effects, see [Using FxEd](#).

To insert a SpecialFX object

1. In DEdit, use the **X** key to position the marker where you want the effect to occur.
2. Press **CTRL + K**, select **SpecialFX**, and press **Enter**.
3. In the **Properties** tab, click the **FxName** drop-down list and select the effect to apply.

Process and test your level to view the effect.

[Top](#)

Inserting a SoundFX Object



SoundFX objects create ambient sound that emanates in two set radii: an inner radius and an outer radius.

Note: You can trigger SoundFX objects on or off from another object using the `msg <object name> <on>` or `msg <object name> <off>` command. For example `msg boil1 off` instructs the SoundFX object named boil1 to turn off.

To insert a SoundFX object

1. In DEdit, use the **X** key to position the marker where you want the effect to occur.
2. Press **CTRL + K**, select SoundFX, and press **Enter**.
3. In the Properties tab, set the properties for the SoundFX object:
 - **Sound**—Specifies the .WAV file to play.
 - **Ambient**—True specifies the sound decreases in volume as the player increases distance from the object. If set to False, the sound simply cuts out when the player leaves the outer radius.
 - **Priority**—Specifies the priority of this sound when more than one sound is playing. Valid values are 0, 1, and 2. Lower values have lower priority.
 - **OuterRadius**—The space between the outer radius and the inner radius has a lower volume than the space inside the inner radius.
 - **InnerRadius**—Specifies the area where the sound plays at full volume.
 - **Volume**—Specifies the volume percentage of the recorded sound that plays in the inner radius.
 - **PitchShift**—Specifies the pitch (frequency) of the sound. For example 1.5 plays 50% faster and 0.5 plays 50% slower.

[Top](#)

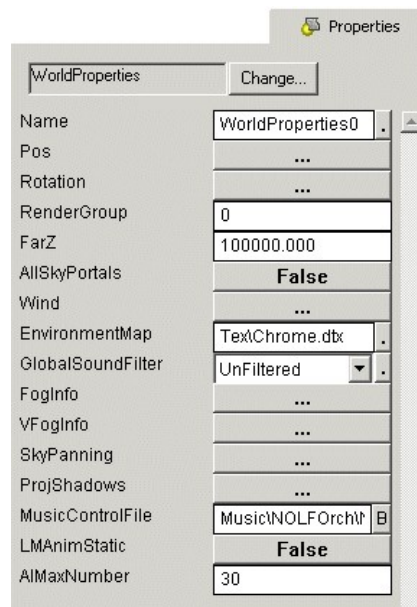
Inserting a WorldProperties Object

Each world (level) you create must have one, and only one, **WorldProperties** object. The **WorldProperties** object determines basic settings associated with your entire world (level).

To insert a WorldProperties object

1. In DEdit, use the **X** key to position the marker where you want the WorldProperties object. The actual location is not highly important, so long as the object resides in your world.
2. Press **CTRL + K**, select **WorldProperties**, and press **Enter**.
3. In the **Properties** tab, adjust the properties of the **WorldProperties** object to set the parameters of your world.

The following image shows the properties for the **WorldProperties** object:



FarZ—Specifies the FarZ clipping plane distance for the level.

AllSkyPortals—If TRUE, the engine renders all of the SkyPortals in the level, ignoring any visibility settings.

Wind—Specifies the wind current for the entire level.

EnvironmentMap—Specifies a texture used as an environment map for all of the models in the level. This setting is currently obsolete due to the addition of RenderStyles.

GlobalSoundFilter—Specifies a sound filter used for all of the sound in this level.

FogInfo—Specifies the overall fogging effect for the entire level based on the distance of objects from the camera location.

VFogInfo—Specifies the fog along the axis, such as ground fog or high cloud layers. This setting is currently obsolete and no longer used.

SkyPanning—Specifies the parameters of the skypanning effect, such as shadows of clouds moving across the ground.

ProjShadows—Specifies parameters controlling shadows used in the level.

MusicControlFile—Specifies the path to the .txt control file setting up the direct music files used in the level.

LMAnimStatic—Specifies a correction for light maps. Set to FALSE if you are using lightmaps and are experiencing flicker.

AIMaxNumber—Specifies the maximum number of AIs allowed in the level. Spawning AIs fail if the maximum has already been reached. Level AIs are unaffected by this value.

[Top](#)

Inserting a PlayerTrigger Object



PlayerTrigger objects allow you to send messages to other objects in the game when the player enters the trigger area. This allows you to trigger WorldModels, send messages to AIs, and basically perform any function required by player entry into the trigger area.

To insert a Trigger object

1. In DEdit, use the **X** key to position the marker where you want the PlayerTrigger object.
2. Press **CTRL + K**, select **PlayerTrigger**, and press **Enter**.
3. In the **Properties** tab, adjust the properties of the PlayerTrigger object. The following list defines a few of the most relevant properties:

Dims—Sets the dimensions of the trigger.

Commands—Specifies the object to receive the command and the command to send. More than one command may be sent and more than one object can receive a command.

NumberOfActivations—Specifies how many times the trigger operates in the level. Set this value to zero to send a command everytime the player enters the trigger area.

[Top](#)

Inserting a ScaleSprite Object



ScaleSprite objects create an auto-facing polygon with an optional lens flare. This makes ScaleSprites ideal for lights, but is also quite useful for other effects requiring an auto-facing polygon.

To insert a ScaleSprite object

1. In DEdit, use the **X** key to position the marker where you want the ScaleSprite object.
2. Press **CTRL + K**, select **ScaleSprite**, and press **Enter**.
3. In the **Properties** tab, adjust the properties of the ScaleSprite object. The following list describes some of the relevant properties for the ScaleSprite object:

StartOn—Sets the sprite on or off at runtime. You can use the msg command from other objects to turn the sprite on or off.

Filename—Specifies the name of the sprite (.spr file) the ScaleSprite displays.

Dims—Specifies the position of the bounding box containing the sprite.

Color—Tints the color of the sprite.

Alpha—Sets the alpha for the sprite. The normal alpharef value for the sprite textures is overridden in the ScaleSprite object, so you must set this value if you are using transparent textures.

ScaleX—Sets the size of the sprite in the X axis.

ScaleY—Sets the size of the sprite in the Y axis.

AdditionalFlags—Sets additional sprite properties such as clipping, scaling, and blend modes.

LensFlareProps—Sets the properties for the lens flare surrounding the sprite. The lens flare is optional and requires an additional sprite file.

[Top](#)

Inserting a TeleportPoint Object



Teleport point objects are target positions for trigger objects to move a character from one position in the level to another position in the same level. Teleport points do not work between levels.

To insert a TeleportPoint object

1. In DEdit, use the **X** key to position the marker where you want the TeleportPoint object.
2. Press **CTRL + K**, select **TeleportPoint**, and press **Enter**.
3. In the **Properties** tab, click the **Rotation** button and adjust the rotation by facing the blue line in the direction to face the character when they appear at the teleport point.
4. In the **Name** box, type a unique name for this teleport point.
5. In the Trigger object to activate the teleportation to this point, set the command to the following string:

`msg player (teleport <name>)`

The **<name>** string must match the name of the teleport point. For example, `msg player (teleport Transport1)` teleports the player to a TeleportPoint object named `Transport1`.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Working With Prefabs

The Jupiter prefab system allows you to create prefabricated objects. These objects are essentially new instances of the same object, with new names for each of the internal components. Prefabs allow you to reuse common objects, and to institute changes in all of the objects inheriting from the prefab throughout all levels of a project.

A chair is a good example of a simple prefab. You can create the chair in DEdit (or import the chair from Max or Maya) and then save it as a prefab. Place the chair prefab wherever you have a need for this specific chair. If you later decide you want to modify the chair's geometry, textures, or perhaps add an `AINodeUseObject - Chair` node to the chair, you can open the prefab and make the modifications. After saving the prefab, all the modifications you made automatically get applied to all the instances of this chair prefab.

Note: Changes to prefabs do not immediately appear in DEdit. You must close and reload your level in order for DEdit to update all of the prefab references. This does not effect processing or gameplay, and while the changes you make do not appear immediately in DEdit, they do appear in the game when you process and run your level.

Prefabs have a specific syntax associated with them that also allows you to send command messages between standard world objects and objects contained in the prefab. Inside the prefab you use a `#.<object name>` syntax to send messages to other objects inside the prefab. From outside the prefab you use the `<prefab name>.<object name>` syntax to specify the object inside the prefab you are sending a command to. For more information about sending messages from inside and outside of prefabs, see the [Using Commands With Prefabs](#) section.

Read the following topics to learn how to create, modify, and send commands to prefabs:

- [Creating a Prefab](#)
- [Modifying a Prefab](#)
- [Using Commands With Prefabs](#)
 - [Using NOLF 2 Prefabs](#)
- [Using the Sample Lighting Prefabs](#)

Creating a Prefab

Creating a new prefab is a relatively simple process. You can create a prefab from a new blank world, or you can create prefabs instantly from any items in a level you are working on.

To create a prefab

1. Create the geometry and objects in any world level as you usually would.
2. Select the objects and brushes you want to become a prefab.
3. Right-click the selection, move to Selection, and then click Save as Prefab.
 4. Browse to a location (typically the Prefabs folder).
 5. Enter a name for the prefab, and click Save.

If you have objects in your prefab that send messages to other objects within the prefab, then you must append the object name in the message with #. in order to make the command operate. For more information, see [Using Commands With Prefabs](#).

The brushes and objects you have saved as a prefab using the following procedure do not immediately become prefabs. They have simply been saved as prefabs and can be inserted from the Prefabs tab. If you want the brushes and objects you have just saved as a prefab to be replaced with the prefab you have created from them, then you must import the prefab from the Prefabs tab, and delete the geometry and objects you used to create the prefab.

If you want a thumbnail image to appear in the Prefabs tab of DEdit, then you must create a bitmap (.bmp) image of the prefab with the same name as the prefab and save it in the same folder as the prefab. The image must be perfectly square and the sizes must be a multiple of 2 (for example, 32 X 32, 64 X 64, 128 X 128 are all valid sizes).

[Top](#)

Modifying a Prefab

You can modify any prefab by opening the prefab and changing it. When you open a prefab, it opens in a separate level of its own, containing nothing but the brushes and objects in the prefab. All changes you make, including the orientation of the geometry, the textures, and the object commands, are reflected in all instances of the prefab throughout all the levels the prefab exists in.

To modify a prefab

1. In the DEdit Prefabs tab, right click a prefab, and then click Open.
The prefab opens in a separate DEdit level.
2. When you have completed your modifications, simply save the prefab and close it.

Remember, changes to prefabs do not immediately appear in DEdit. You must close and reload your level in order for DEdit to update all of the prefab references. This does not effect processing or gameplay, and while the changes you make do not appear immediately in DEdit, they do appear in the game when you process and run your level.

[Top](#)

Using Commands With Prefabs

When a prefab is placed in a level, the prefab and all of the objects contained within the prefab are given the same unique name. Because prefabs and all of their objects are given unique names, commands between prefab objects become invalid if they are not created using the special prefab syntax described in this topic.

For example, if you have a prefab with a door handle prop and a RotatingWorldModel object, you may be sending a message from the prop to the world model telling it to unlock when the door handle is successfully picked. The command from the prop might be `msg RotatingWorldModel0 UNLOCK`. If the prefab's name is GreenDoor, then when the prefab gets inserted into the level, the name of the RotatingWorldModel becomes `GreenDoor0.RotatingWorldModel0`. In order for the prop to send a message without knowing the name RotatingWorldModel0 gets when it is inserted, you must prepend the name of RotatingWorldModel0 with the #. syntax. When the prefab gets inserted into the level, the # symbol always gets replaced with the actual name of the prefab. Thus `msg #.RotatingWorldModel0 UNLOCK` becomes `msg GreenDoor0.RotatingWorldModel0 UNLOCK`.

For objects outside of a prefab sending commands to an object inside of a prefab, use the name of the prefab followed by a period and then followed by the name of the object inside the prefab. This does result in the problem of not knowing the actual name of the object inside of the prefab you want to send the message to. To find this out, you must open the prefab and select the object to determine what its name is. To prevent this problem, you should always document your prefabs, explaining what the objects are inside the prefab, and what commands you intend them to send or receive.

Note: You can also use the #. syntax to append the name of objects in commands. For example, `msg trigon.# TRIGGER` can send a message to a TriggerObject outside a prefab (this does require that you create a trigger object with the name `trigon.<prefab name>` to receive the message). Regardless of its position, the # sign inside a prefab always gets replaced with the name of the prefab.

To send a message between objects in a prefab

- Use `msg #.<object name> <Command>`.

Example: A CommandObject inside a light prefab could send a message to several different objects in order to turn the light off.

o `msg #.BulbOn (Visible 0)` could send a message to a WorldModel named BulbOn to turn off the visibility of bound

- light bulb brushes. This causes the yellow lit bulb to vanish.
- o `msg #.BlubOff (Visible 1)` could send a message to a WorldModel named BulbOff to turn on the visibility of bound light bulb brushes. This causes the dark unlit bulb to appear.
- o `msg #.LightGroup0 OFF` could send a message to a LightGroup to turn off all the lights bound to LightGroup0.

To send a message to an object inside a prefab

- Use `msg <prefab name>.<object name> <Command>`.

Example: A RotatingSwitch object might send a message to a light prefab telling it to turn on or off. If the name of the prefab is MyLight01, then the following messages would activate CommandObjects named TurnOn and TurnOff.

- o `msg MyLight01.TurnOn START`
- o `msg MyLight01.TurnOff START`

To send a message from an object inside a prefab to an object outside a prefab

- Use `msg <object name>.# <Command>`.

Example: You might want a computer prefab to activate an explosion SpecialFX object after it has been shot, but you want to be able to place the explosions after you place the prefab. The following messages could be sent from a Breakable WorldModel object to SpecialFX objects that do not exist until you place them.

- o `msg MyExplosion1.# ON`

After you create the prefab, place the SpecialFX object you want to call and name it MyExplosion1.<PrefabName>. For example, if the computer prefab gets named MyComputer01, then name the SpecialFX object MyExplosion1.MyComputer01.

[Top](#)

Using NOLF 2 Prefabs

The NOLF 2 game code contains a variety of different prefabs to experiment with. The lights, in particular, contain many objects and commands, and require the creation of outside triggers to link them to the switch prefabs. Many of the lights also use variables to keep the lights from coming back on if the bulbs have been destroyed.

The best way to understand the operation of a NOLF 2 prefab is to simply open the prefab (as described in [Modifying a Prefab](#)).

[Top](#)

Using the Sample Lighting Prefabs

Two lighting prefab samples are included in the following directory:

- `\Development\T02\Game\Prefabs\Samples`

The two prefabs, LightSample-Simple and LightSample-Fluorescent, show two different methods for creating dynamic lights that you can turn on or off.

Using the LightSample-Simple Prefab

The LightSample-Simple prefab shows an example of how to create a simple light prefab. LightSample-Simple is an indestructible metal light designed to go in ceilings or along floors. You can turn the light on or off by sending it a message from another object.

To use the LightSample-Simple prefab

1. In DEdit, use the **X** key to position the marker where you want to insert the prefab.
2. In the **Prefabs** tab, open the **Samples** folder and then double-click the LightSample-Simple prefab.
3. From any object capable of sending a command (such as a RotatingSwitch object on a light switch) send the following commands:
 - o `msg <name>.CO_ON START` to turn on the light.
 - o `msg <name>.CO_OFF START` to turn off the light.

Note: Replace <name> in the previous syntax with the name of the prefab after you have inserted it in your level. For example, if the prefab gets named LightSample-Simple0 then use `msg LightSample-Simple0.CO_ON START` to turn on the light.

Using the LightSample-Fluorescent Prefab

The LightSample-Fluorescent prefab shows an example of how to create a destructible light prefab. LightSample-Fluorescent turns on or off

unless it is destroyed. Three WorldModels control the visibility of the off, on, and destroyed brushes in this prefab. Three command objects test the broken or unbroken state of the light, and switch between the on, off, or destroyed WorldModel brushes as appropriate. The command objects also turns on or off the light group.

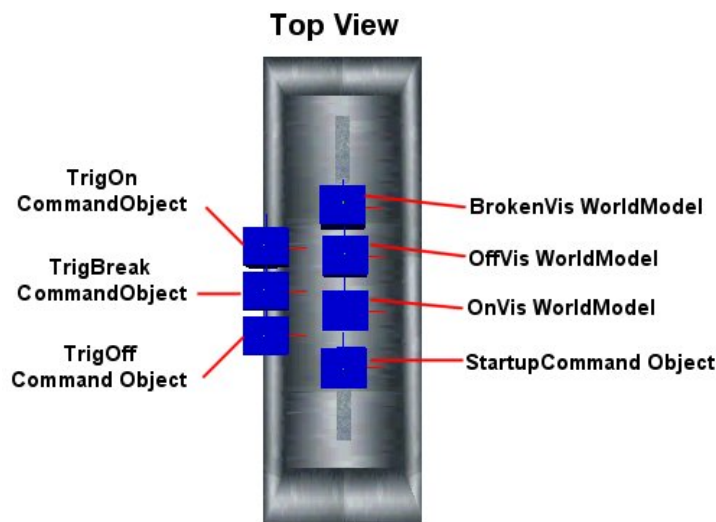
To use the LightSample-Fluorescent prefab

1. In DEdit, use the **X** key to position the marker where you want to insert the prefab.
2. In the **Prefabs** tab, open the **Samples** folder and then double-click the LightSample-Fluorescent prefab.
3. From any object capable of sending a command (such as a RotatingSwitch object on a light switch) send the following commands:
 - o msg <name>.TrigOn START to turn on the light.
 - o msg <name>.TrigOff START to turn off the light.

Note: Replace <name> in the previous syntax with the name of the prefab after you have inserted it in your level. For example, if the prefab gets named LightSample-Fluorescent0 then use msg LightSample-Fluorescent0.TrigOn START to turn on the light.

How the LightSample-Fluorescent Prefab Works

When considering how to construct a light, you must consider what you want the light to do. In the case of LightSample-Fluorescent, the light must break as well as turning on and off. To enable the three states of on, off, and broken, you must use a variety of different objects. There are many different ways you can enable the functionality of these states, and this example shows only one possible way of creating a breakable light.



The image above shows the top view of the LightSample-Fluorescent prefab. The objects in this image perform the following functions:

TrigOn CommandObject—Serves as the main interface for turning the prefab on. When TrigOn receives a START command, it sends the following commands:

- if (#.Broken == 0) then (msg #.OffVis (Visible 0)) **Turns off dark brushes.**
- if (#.Broken == 0) then (msg #.OnVis (Visible 1)) **Turns on light brushes.**
 - if (#.Broken == 0) then (msg #.LightGroup0 ON) **Turns on lights.**

TrigBreak CommandObject—Serves as the main interface for setting the prefab to broken. When TrigBreak receives a START command, it sends the following commands:

- msg #.OnVis (Visible 0) **Turns off light brushes.**
- msg #.OffVis (Visible 0) **Turns off dark brushes.**
- msg #.BrokenVis (Visible 1) **Turns on broken brushes.**
 - msg #.LightGroup0 OFF **Turns off lights.**

TrigOff CommandObject—Serves as the main interface for turning the prefab off. When TrigOff receives a START command, it sends the following commands:

- if (#.Broken == 0) then (msg #.OnVis (Visible 0)) **Turns off light brushes.**
- if (#.Broken == 0) then (msg #.OffVis (Visible 1)) **Turns on dark brushes.**
 - if (#.Broken == 0) then (msg #.LightGroup0 OFF) **Turns off the lights.**

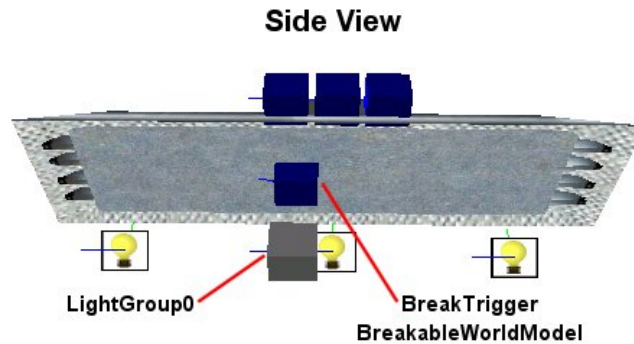
BrokenVis WorldModel—Controls the visibility of the broken brushes. All brushes to appear when the prefab gets broken are bound to this WorldModel object.

OffVis WorldModel—Controls the visibility of the dark brushes. All brushes to appear when the prefab lights are off and the prefab is not

broken are bound to this WorldModel object.

OnVis WorldModel—Controls the visibility of the light brushes. All brushes to appear when the prefab lights are on and the prefab is not broken are bound to this WorldModel object.

StartupCommand Object—Initializes the `#.Broken` variable to zero. The `#.Broken` variable is used by `TrigOn` and `TrigBreak` to test the broken/unbroken state of the prefab. This keeps `TrigOn` and `TrigOff` from switching brushes and activating lights when the prefab is destroyed. Notice that `Broken` begins with `#.` so that if multiple `LightSample-Fluorescent` prefabs get placed in the same level, they all get unique `Broken` variable names. If you do not do this, then the same variable gets tested for all the `LightSample-Fluorescent` prefabs, and when one prefab gets broken, all of the other `LightSample-Fluorescent` prefabs in your level cease to work.



The image above shows the side view of the `LightSample-Fluorescent` prefab. The objects in this image perform the following functions:

LightGroup0—Controls all of the lights in the prefab. For more information about `LightGroup` objects, see [Inserting a LightGroup Object](#).

BreakTrigger BreakableWorldModel—Bound to an invisible brush that surrounds the fluorescent tubes in the prefab, this `WorldModel` sends the following commands from its `DamageProperties - DeathCommand` parameter:

- `set #.Broken 1` **Sets the Broken variable to TRUE.**
- `msg #.TrigBreak START` **Activates the TrigBreak CommandObject.**

This concludes the description of the objects in the `LightSample-Fluorescent` prefab. The following information describes how the prefab works based in each of the three functions it performs: turning on, turning off, and breaking.

Turning ON

When the `TrigOn` `CommandObject` receives a `START` command, it turns off the dark brushes, turns on the light brushes, and turns on the lights in the `LightGroup` object. Each of these commands tests the `Broken` variable to see if the prefab is unbroken. If the prefab is broken, then none of these commands get sent.

Turning OFF

When the `TrigOff` `CommandObject` receives a `START` command, it turns off the light brushes, turns on the dark brushes, and turns off the lights in the `LightGroup` object. Each of these commands tests the `Broken` variable to see if the prefab is unbroken. If the prefab is broken, then none of these commands get sent.

Breaking

When the `Breakable WorldModel` gets destroyed, it sets `Broken` to one and starts the `TrigBreak` `CommandObject`. Setting `Broken` to one prevents any commands in the `TrigOn` and `TrigOff` `CommandObjects` from being sent. When `TrigBreak` starts, it turns off the visibility of the light and dark brushes, it turns on the visibility of the broken brushes, and it turns off the lights.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Working With Sprites

Sprites (.spr files) are composed of a texture or textures linked together. You can use sprites to create animations or to create effects using objects that require a sprite. You can compose sprites from any texture, including transparent textures.

ScaleSprite objects perform operations using sprites created with the procedures in this section. For more information about ScaleSprites, see [Inserting a ScaleSprite Object](#) in the [Working With Objects](#) section.

This section contains the following topics on basic sprite creation and use:

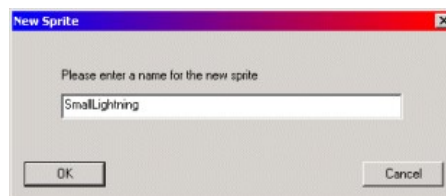
- [Creating a Sprite](#)
- [Applying a Sprite](#)
- [Editing a Sprite](#)

Creating a Sprite

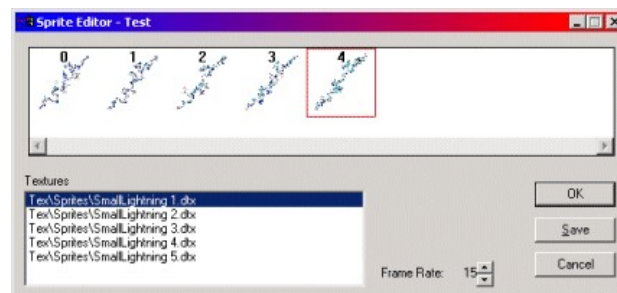
Creating a new sprite is a simple operation that involves the use of the Sprites tab in DEdit. The following procedure assumes you have already imported the texture files to compose your sprite into the DEdit Textures tab. For information about importing textures into your world, see [Importing a Texture](#).

To create a sprite

1. In the DEdit **Project Window**, click the **Sprites** tab.
2. Select the directory to store the sprite in.
3. In the window below **Name**, right-click and select **Create New Sprite**.
The **New Sprite** dialog box opens.



4. Type the name for the new sprite, and then click **OK**.
The **Sprite Editor** dialog box opens, and the **Project Window** switches to the **Textures** tab.



5. In the **Textures** tab, double-click each texture to add to the sprite. The textures you select are added to the **Sprite Editor** dialog box in the order you select them. To remove a texture from the **Sprite Editor** dialog box, double-click the texture.
6. Set the **Frame Rate** for the animation. The **Frame Rate** specifies the number of frames in the animation that appear each second. For example, the animation in the previous image has five frames and a **Frame Rate** of fifteen, therefore the entire animation appears three times each second.
7. Click **Save** to save the sprite.

Applying a Sprite

You can apply a sprite to any brush surface in **Brush Edit Mode** or any polygon in **Geometry Edit mode**. Additionally some objects require sprites to operate correctly. For example, the ScaleSprite object requires a sprite to create a polygon that always faces the character.

To Apply a Sprite

You can apply a sprite in DEdit the same way you apply textures. Simply select the sprite in the Sprites tab, select the brush to apply the sprite to, and then press **CTRL + T**.

[Top](#)

Editing a Sprite

There are two methods for editing sprite files:

- You can edit the textures composing the sprite.
- or—
- You can open the Sprite Editor to add or remove textures from a sprite.

The textures a sprite is composed of are referenced by the sprite, not contained in the sprite, so any edits you make to the textures the sprite is composed of are automatically applied to the sprite when your level is processed. For information about editing textures, see [Working With Textures](#).

To Edit a Sprite

1. In the DEdit **Project Window**, click the **Sprites** tab.
2. Double-click the sprite to edit. The **Sprite Editor** dialog box opens.
3. Double-click any sprite to remove it from the animation.
4. In the **Texture** tab, double-click any texture to add it to the animation.
5. When you have completed your edits, click **Save**.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Working With Textures

Textures are raster-based images that cover a brushes polygons. Think of textures as paint, and the surface of your brushes as the canvas. There are a variety of different applications that can create texture files, although all textures used by the LithTech Jupiter engine reside in .DTX format.

This section contains the following topics on creating, applying, and manipulating textures:

- [Creating a Texture](#)
- [Importing a Texture](#)
- [Applying Textures](#)
- [Adjusting Textures](#)
- [Using an Alpha Mask](#)

Creating a Texture

You can create textures as .TGA (Truevision Targa) files, and import them into your DEdit textures tab, or you can save them directly as .DTX files using plugins designed for that purpose. Which method you choose depends entirely on the application you use to draw your textures. Photoshop has a .DTX plugin that allows you to save in .DTX format, and other programs, such as PaintShop can make use of the .DTX plugin if they support Photoshop plugins. If you are making textures in a program that does not support Photoshop plugins, then the best option is to save your textures as .TGA files. Saving as .BMP or .PCX may work, but is not officially supported and can result in errors.

Texture considerations

- Because textures are often tiled, you should try and design the borders of your textures in a manner that allows them to seamlessly match together. Most image editors have tools allowing you to create seamless patterns, and you should research and use these tools whenever creating textures for tiling.
- You can create textures from virtually any digital image, but be wary of copying and using images that are not owned by your company. Copyright rules generally apply to all textured images.
- Alpha layers (also called "alpha masks") allow you to adjust the transparency or translucency of your textures. This technique allows you to create fences, grated windows, and ladders that require very little processing power in comparison to high polygon precision objects. It is not necessary to create an alpha layer for your texture if the texture will not be used for translucency. Textures using alpha layers can also use the `alpharef` command string to specify their level of translucency. For more information about alpha layers see [Using an Alpha Mask](#).
- Texture sizes are limited by current video cards and of the DirectX API. Texture sizes are restricted to a power of 2 squared ($(2^X)^2$). For instance you can create a texture with any width and height combination of 8, 32, 64, 128, 256, 512, and so on. Some examples of valid sizes are: 32X64, 64X128, 128X256, 512X512.
- Smaller textures take less memory to store and use, so the smaller the texture, the less it costs to render. Tiling small textures results in very fast rendering. The exact number and size of textures that is appropriate for your game depends upon the amount of video memory available on your target platform.

Importing a Texture

If you have created a .DTX file there is no need to import a texture. Simply save your texture directly into the folder where you plan to use it in your game. If you have created a .TGA file and wish to import it into your world, then use the following procedure.

To import a texture

1. In the DEdit **Project Window**, click the **Textures** tab.
2. Select the textures folder to import to.
3. Right-click in the list of textures, and then click **Import TGA files**.
4. Select the .TGA files to import, and then click **OK**.

DEdit imports the selected .TGA files and converts them to .DTX. The original source files remain in .TGA format at the source location.

Note: Some image applications create .TGA files that use compression or other options that DEdit cannot understand properly. Be sure that you save your .TGA texture files without using any compression options or DEdit may corrupt the texture or crash. In PaintShop Pro, for instance, the **Options** button in the **Save As** dialog box for .TGA files allows you to turn off compression.

[Top](#)

Applying Textures

You can apply textures to any selected brush in DEdit. **Brush Edit** mode allows you to apply textures to entire brushes, while **Geometry Edit** mode allows you to apply textures to individual polygons.

To apply a texture

1. In the DEdit **Textures** tab, select the texture you wish to apply.
2. In any view, select the brush or polygon to receive the texture.
3. Press **CTRL + T** to apply the texture.

The texture becomes immediately visible. You can use this same process to apply a sprite from the **Sprites** tab onto a brush or face to create a simple animated surface.

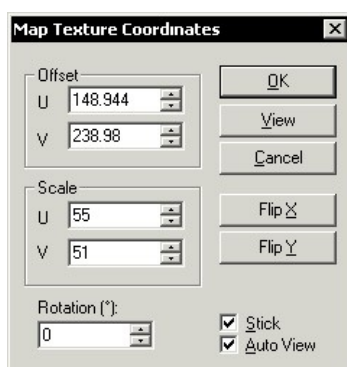
[Top](#)

Adjusting Textures

There are several adjustments you can make to textures: you can map them to the size and shape you want, you can set them to fit a polygon, you can wrap them around the surface of a brush, and you can reset them to their original wrapping. Use the following procedures to adjust the way your texture appears on the surface of any world geometry.

To map a texture

1. In **Brush Edit** or **Geometry Edit** mode, select the brush or polygon to edit, and then press the **A** key. The Map Texture Coordinates dialog box appears:



2. Use the **Offset** area to move the texture horizontally (**U**) and vertically (**V**) across the surface of the selected object.

3. Use the **Scale** area to adjust the width (U) and height (V) of the texture.
4. Use the **Rotation** area to adjust the rotation of the texture on the surface of the selected object.
5. When you have finished your edits, click **OK**.

To adjust a texture with the mouse

1. In any mode, select the texture to adjust.
2. Press and hold the **R** key, then move the mouse to slide the texture over the surface of the selection.
3. Press and hold the **R** key and the **RIGHT MOUSE BUTTON**, then move the mouse to rotate the texture over the surface of the selection.
4. Press and hold the **R** key and the **LEFT MOUSE BUTTON**, then move the mouse to scale the texture over the surface of the selection.

Note: If you hold the **SHIFT** key while using any of the above commands, the texture will snap to precise degrees of motion or rotation instead of moving completely freely.

To fit a texture to a polygon

1. In **Geometry Mode**, select the polygon to fit.
2. Press **SHIFT + C**.

The texture adjusts its coordinates to fill the selected polygon. This command works across multiple faces, but will not work perfectly on faces with more or fewer than four sides (such as triangles) or on an entire brush at once.

To wrap a texture around a brush

Note: To use this command you may need to apply a shortcut key combination to the Texture Wrap action. For instructions on adding the shortcut key assignments, see [Changing or Adding a Shortcut Key](#).

1. In **Brush Mode**, select the desired brush.
2. Press the key combination associated with the **Texture Wrap** action.
3. In any view, click and drag a line across the surface of the brush to designate the direction of the texture.

To reset a texture

1. In any mode, select the texture to reset.
2. Press the **K** key.

When pressed more than once, this command also aligns a texture to each edge of the selected polygons.

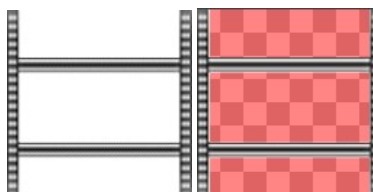
[Top](#)

Using an Alpha Mask

Alpha masks allow you to designate the portions of a texture that are transparent or translucent. Transparent portions of a texture are invisible (not rendered), and translucent portions are ghost-like (partially visible).

To use an alpha mask

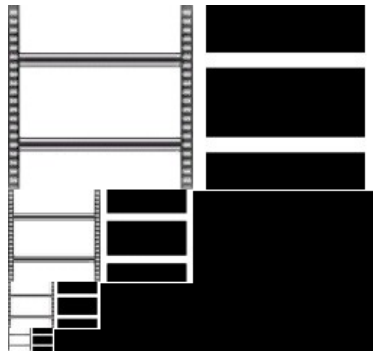
1. Create an image with an alpha mask using an image editor that supports alpha masking (such as Photoshop or PaintShop Pro). The following two images show a ladder created with an alpha mask. The image on the left shows the normal ladder illustration. The image on the right shows the mask for the illustration. The red portions of the mask are painted with black to specify complete transparency.



2. Save the file in .TGA or .DTX format.
3. Import the texture as specified in [Importing a Texture](#).
4. In the **Textures** tab, double click the newly imported texture. A dialog box opens along with a preview of the texture and its alpha mask. The following image shows a preview of a ladder with its alpha mask:

The alpha mask appears directly to the right of the image.

White portions of the mask are completely visible.



Black portions of the mask are completely transparent.

Any **shades of gray** you use in your alpha mask also appear here and create various levels of translucency. The darker the shade of gray, the more translucent that area of the texture.

5. In the dialog box for the texture, you can type **alpharef X** where **X** indicates a value between 0 and 255. This command specifies the level of translucency allowed by the mask (translucency has a range of 256 gray shades). The alpharef command sets the cut-off tolerance of the translucency. The higher the alpharef value, the lighter a part of the alpha mask can be before it stops rendering entirely.

An alpharef of 200, for example, allows forces transparency for all gray shades from absolute black (255) to the dark gray shade specified by 201. All shades of grey from 0 to 200 are considered transparent, so those parts of the texture do not appear in the game. The following image illustrates where to place the alpharef command in the Texture Properties dialog box. Using an alpharef value helps to create a clean edge in transparent textures, and it can also help improve performance for large translucent textures if they have large areas that are almost totally invisible.

* Command String

6. Click **OK** to close the dialog box, and then apply the texture to a brush or polygon as specified in [Applying Textures](#).

Note: Textures with alpha masks do not appear transparent or translucent in DEdit. Objects in your level only appear translucent after you process and run your world.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Advanced Texturing

DEdit supports intricate texture work, both on a brush-by-brush level and on a face-by-face level.

This section explains how to scale, rotate, offset and flip textures to line them up and make them appear correctly. This section also includes information about Texture Properties and Attributes.

This section contains the following advanced texturing topics

- [Aligning Textures](#)
- [Texture Alignment Hotkeys](#)
- [Assigning Properties to Textures](#)
- [Assigning Attributes to Textures](#)

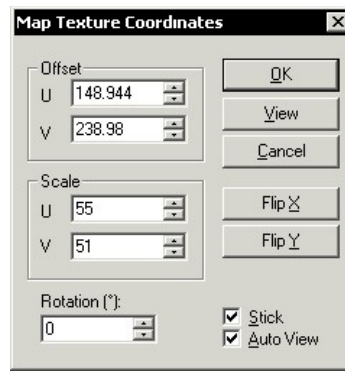
Aligning Textures

Even when you follow all the rules of good brush layout and level design, some textures don't line up where you want them to be when you first apply them. Either the texture's edges don't line up with the edges of the brush, the only texture that has the symbol you want on it is too large, or a crate is rotated off the grid and its textures are not.

To Modify and Align your Texture

1. In **Brush** mode, select the brush with the texture to edit.
2. Get a good view of the brush in the Perspective viewport, then right-click.
3. In the **Context** menu, select **Map Texture Coordinates**.

The following dialog box appears:



You can use this dialog in either Brush or Geometry mode. In Brush mode, the settings affect all the faces on the brush. In Geometry mode, this same dialog can change each face individually. Instead of selecting the face you want, just highlight it in red by hovering the mouse over it in the Perspective port, then right-click.

The dialog's controls are mostly intuitive.

U/V Offset Controls

The U/V offset controls tell you about the alignment of textures on a face. U (horizontal) and V (vertical) offsets are used to move the texture around on the face pixel by pixel. You can apply a negative or positive offset, which allows you to move a texture in any direction along a face. They are especially useful for things like aligning a sign to properly fit the brush you've created for it.

For instance, a texture may perfectly fit the brush but hang a little off-center: say, 4 pixels too low and 8 pixels too far to the left. To correct this, you would add to the U and V offsets. Setting the U offset to 8 would move the texture 8 pixels across the face to the right, and setting the V offset to -4 would move the texture up 4 pixels on the face.

This isn't something you want to use for all of your textures. It's much better to build such that the bulk of your textures fall naturally along brush boundaries so that you don't even need to think about texture alignment on most of your brushes. It's meant to be used for detailed, standout objects like crates, signs and display screens which are often set at odd locations and need to be exactly aligned.

Rotation Control

The next control, the Rotation control is what you use to align a texture on a brush that's been rotated. Perhaps you've made a poster that's hanging on a wall at a slightly weird angle of 17 degrees to make the room look a little unkempt. By applying a 17-degree rotation to the texture of the poster, you can get the poster to look as it should, properly lined up on the brush.

U and V Scaling Controls

The next controls are the U and V scaling controls. By default, these are each set to the horizontal and vertical size of the texture on the current face in pixels. However, by changing these values you can change the size of a texture in the world. By modifying just one of them (say, doubling the value in the U scale) you can stretch the texture. By doubling both of them, you quadruple the size of the texture on the brush. Lastly, you can flip a texture without rotating it by putting a negative value into either of these boxes. There are many cases where mirroring a texture can be useful: aligning trims on a wall, building a fake mirror image of a screen, and giving a clue to a secret door. The dialog also has two Flip buttons that allow you to do the same thing.

Finally, there are two check boxes. When you check the Stick checkbox, DEdit will attempt to hold the textures aligned to the brush's faces as they currently are. This allows you to move the brush around without the texture sliding around on the face. DEdit will try to automatically update the textures' offsets and even rotation to keep the faces. Although DEdit doesn't always make the right choices, for simple moves and simple rotations it can be faster than doing the math yourself.

The XZ Only checkbox is used to force textures to map on the brush from straight above it. It's generally only used for terrain, to make it easier to align textures. You can experiment with it, but it generally causes undesirable results on anything but terrain.

[Top](#)

Texture Alignment Hotkeys

There are also some texture alignment hotkeys that allow you to do some of these operations without going into the dialog.

The **K** key rotates the texture on a face in Geometry mode, attempting to align it accurately with the corners of the brush. It's frequently faster than going into the Map Texture Coordinates dialog for simple problems like our sign problem above.

The other useful shortcut (also used in Geometry Mode) is the **R** key. When this key is held down, moving the mouse will move the texture around on the face of the brush. It is difficult to exactly align a texture using this shortcut, but it can be very helpful for getting the texture roughly aligned. You can then use the Map Texture Coordinates dialog to clean up the result, cleaning up the UV offsets to exactly fit.

You can also hold **R** and click the right mouse button to rotate textures freehand. Holding **SHIFT+R** changes from totally freehand

rotation to rotating in 15-degree increments. Holding **R** while clicking with the left mouse button allows you to scale the texture freehand.

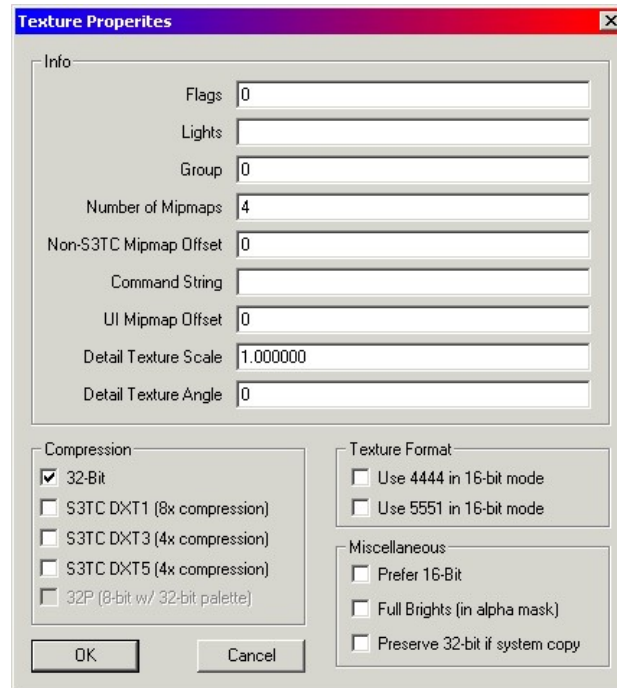
[Top](#)

Assigning Properties to Textures

Each texture in the game has certain properties attached to it. Texture properties are global, and they effect each and every instance of a texture. For example, if you set the `alpharef` Command String for a texture to 100, then all instances of that texture in your game have an `alpharef` of 100.

To assign properties to a texture

1. Click the **Textures** tab.
2. Double-click a texture name. The **Texture Properties** dialog box appears.



3. Enter the appropriate property values and selections, then click **OK**.

[Top](#)

Assigning Attributes to Textures

You can attach attributes to a texture using the **Flags** property in the **Texture Properties** dialog box. To access the **Texture Properties** dialog box, double-click any texture in the **Texture** tab.

The **Flags** property specifies the attributes for a texture. Texture attributes are defined in the `surface.txt` file. For more information about attributes, see [Working With Attributes](#) in the [Programming Guide](#).

Attributes you can associate with a texture using TO2 game code include:

- ShowsMark (Integer)
- ShowBreath (Integer)
- CanSeeThrough (Integer)
- CanShootThrough (Integer)
- MaxShootThroughPerturb (Integer)
- MaxShootThroughThickness (Integer)
 - BulletHoleSpr (String)
 - BulletHoleMinScale (Float)
 - BulletHoleMaxScale (Float)
 - BulletRangeDampen (Float)
 - BulletDamageDampen (Float)
 - BulletImpactSnd1-2 (String)
 - ProjectileImpactSnd1-2 (String)
 - MeleeImpactSnd1-2 (String)

- ShellImpactSnd1 (String)
- ShellSndRadius (Float)
- GrenadeImpactSnd1 (String)
- GrenadeSndRadius (Float)
- RtFootPrintSpr (String)
- LtFootPrintSpr (String)
- FootPrintLifetime (Float)
- FootPrintScale (Vector)
- RtFootSnd1-2 (String)
- LtFootSnd1-2 (String)
- SnowmobileSnd1-2 (String)
 - BodyFallSnd (String)
- BodyFallSndRadius (Float)
- BodyLedgeFallSnd (String)
- BodyLedgeFallSndRadius (Float)
 - ActivationSnd (String)
- ActivationSndRadius (Float)
- DeathNoiseMod (Float)
- MoveNoiseMod (Float)
- ImpactNoiseMod (Float)
 - Hardness (Float)
- SnowmobileVelMult (Float)
 - Magnetic (Integer)
- ImpactFXName (string) (OPTIONAL)
- UWImpactFXName (string) (OPTIONAL)
 - ExitFXName (string) (OPTIONAL)
- UWExitFXName (string) (OPTIONAL)
- SnowmobileImpactFXName (String) (OPTIONAL)

To associate a set of attributes with a texture, place a value in the **Flag** property for that texture. You can specify any Flag number between 0 and 256. Each flag number corresponds to an ID in the surface.txt file. Each ID contains values that specify how a texture functions in the game. The following custom IDs designate textures used in the TO2 game code supplied with Jupiter:

- 8 = Electronics (computers, machines, with activation sound)
- 9 = Chainlink Fence (masked metal, shoot through)
- 10 = Ceramic Tile (Stone tile, cut stone, louder footsteps than cement)
- 11 = Cement (you can walk quietly on it compared to ceramic tile)
- 12 = Plaster & Stucco (flakier and more powderlike debris than other stones)
- 13 = Porcelain (For vases, porcelain lamp bases, toilets, urinals, sinks, etc)
- 20 = Metal
- 21 = Tin (corrugated metal, light metal, can shoot through, different impact)
- 23 = trussing (masked metal like chainlink, but with different impacts/sound)
- 24 = Metal Pipe, Water
- 30 = Wood
- 31 = Drywall (a powdery wood/stone impact, can shoot through)
- 32 = Plywood, thin wood (can shoot through, different sound, impactfx)
- 40 = Glass
- 41 = Bullet-Proof Glass
- 42 = Glass Facade (can't see through, but shows glass impacts)
- 50 = India Spices
- 51 = Sake Jug (water spout)
- 52 = trashbag
- 53 = paper
- 54 = generic debris impact
- 55 = Muck (slime)
- 69 = Terrain: Ice
- 70 = Terrain: Dirt
- 71 = Terrain: Snow
- 72 = Terrain: Mud
- 73 = Terrain: Grass
- 74 = Terrain: Dirt Road
- 75 = Terrain: Rock, gravel
- 76 = Terrain: Slushy, slushy Road
- 77 = Foliage
- 78 = Terrain: Snowy Gravel
- 79 = Terrain: Snowy Dirt
- 80 = Carpet
- 81 = Cloth (can shoot through)
- 90 = Linoleum (like ceramic tile but plastic)
- 91 = Rubber (like linoleum, only more rubbery)
- 100 = Wood Door (with activation sound, can shoot through)
- 101 = Metal Door (with activation sound)
- 102 = stone Door (with activation sound)
- 103 = Metal File Cabinet (with activation sound)
- 205 = Force Field ()

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Using the Camera

This section explains how to use the camera in the 3D viewport.

For information about using the camera in the three 2D viewports, see the [Using the Viewports](#) topic in the [Building Your First World](#) tutorial. Remember, for 2D viewports: press and hold the I key and move the mouse in a 2D viewport to pan the view around the level. Press the O key and use the mouse to zoom the camera in and out.

3D View

The 3D view is more complicated and very useful. For a tour of the 3D view, follow these steps:

1. Hover your mouse over the 3D viewport (top left).
2. Press and hold the I key and move the mouse around slowly. The window's camera moves in the horizontal plane as you move the mouse.
3. To move vertically, press and hold the right mouse button.
4. To move faster, press and hold the left mouse button.
5. Release the I key.
6. Press and hold the O key, then move the mouse slowly. Now, instead of moving, the camera turns. It operates just as the "mouse look" mode does in a first-person game. If you're not careful you can turn the camera completely upside down, so be sure not to move the mouse around too wildly.
7. Also, you can use the left and right mouse buttons to move forward and backward along the camera's sight-path, respectively.

With these controls, you can move the camera anywhere in the map you need to see something. You can also see the object from any angle.

Center View Buttons

There are two other controls that set camera location. On the WorldEdit toolbar, the Center View and Center View On Marker buttons  set the center point of the camera. These buttons are useful for rapidly moving the camera to a new location on the map, and you'll probably find yourself using them a lot in large maps.

The Center View button centers the camera on the selected objects or brushes, or on the middle of a selected group of objects or brushes.

The Center View On Marker button moves the camera so that it's aimed at the green marker.

[Top](#)

View Modes

Right-click on any viewport to display the Context menu. Use the commands on the Context menu to change the view that your camera presents in the Perspective viewport.

The Shade Mode and Move Mode commands both control the way that the camera works in that port. The Shade Mode menu lets you switch between textured, flat-shaded (un-textured), and wireframe views of the level.

Flat shading and wireframe display are useful for seeing how your brushes are laid out, since textures can obscure seams between two brushes, making some tasks difficult. These modes are also faster in some cases, so if DEdit runs especially slowly on your machine you may want to use them to speed things up.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Adding Simulation Objects

Of course, there's more to a map than just the brushes it contains. A map with just brushes is like a movie with no actors: It's a still life. In order to have weapons, enemies, and even lights, you must add objects to the map that tell the game engine where to place these things. A map can't even be started if you don't designate a player start point in the map.

This section deals with adding objects like weapons, enemies, and lights, as well as other types of props, to your maps. Most of the objects mentioned above are referred to as simulation objects (or, more simply just as objects). A simulation object can be defined as an object that is defined not of brushes but by code (either in the game or the engine). Jupiter has certain classes of object that any game built on it can use.

Most games also define their own sets of simulation objects, some of which are brand new, and some of which are subclasses or children of the engine classes. A subclass incorporates most or all of the behaviors and attributes of the parent, but also adds some new behaviors of its own. In the case of most Jupiter games, for example, you can add a Door ("Make this brush move."), you can add a Door-subclass RotatingDoor ("Make this brush move and turn on a point."), or you can add a RotatingDoor-subclass DestructibleRotatingDoor ("Make this brush move, turn and explode when shot."). Each of these subclasses takes the properties and behaviors of the object above them in the tree (their parent) and extends them in new directions. You can still use the root classes in cases where you don't need the added properties of the child objects, and often will.

This tree system is easy to see when you look at the Add Object dialog inside a Jupiter game. For example, each of the AI classes derives from the root class, but each type of enemy, civilian and major character is its own subclass, many of which have their own subclasses for different armies or different versions of the character.

Note: This section assumes you have already created the Big1.LTA file in the [Using Brushes](#) tutorial.

To add a simulation object to your level

1. Switch back to your Big1.LTA file, or open it if you have closed it.
2. Begin by moving the green marker so that it's centered in the archway you built earlier and about 64 units above the floor.
3. Right-click in a viewport and select Add Object from the Context menu. The Add Object dialog appears. This dialog is where you select and add objects to your level.
4. Select the **GameStartPoint** object from the list and click OK. Now, if you look closely at the center of the green marker, there's a small square located there. If a simulation object doesn't have a discrete volume that's specified by the player, this square is how it will be represented in DEdit. Many objects such as models have a volume associated with them and are therefore represented by a box that's the right size for them.

GameStartPoint is the first object you should add to your level, since without a start point your game doesn't know where to start the player off in the level. The reason for raising the start point 64 units off of the floor is to ensure that the player has room to stand when he or she appears. If there's not enough room for the player to appear, the start point is too low to the floor, or if the start point is inside a solid object, the player will just fall out of the bottom of the world.

[Top](#)

Binding Simulation Objects to Brushes

Another way of creating a simulation object is by binding it with a brush or group of brushes.

To bind brushes

1. Select the brush or brushes you want to bind
2. Right-click and select Bind to Object from the Context menu.
3. The object list appears and you can then select the simulation object you want to bind to.

By binding brushes to a simulation object, you tell the simulation object to use the brushes to define its structure or its volume.

Two general classes of bound objects commonly appear in Jupiter: WorldModels and Volume Brushes.

[Top](#)



Creating World Models

WorldModels change the geometry that they're bound with into an object that can be moved, destroyed and manipulated. Usually, brushes can never move or be changed in any way. WorldModels are the basis of things like doors, exploding walls and elevators. Some props may also be constructed out of WorldModels as well.

This section contains the following topics related to the creation of world models:

- [Volume Brushes](#)
- [World Models](#)
 - [Creation](#)
 - [Damage](#)
- [Attachments](#)
 - [States](#)
 - [Messages](#)
 - [Options](#)
 - [Commands](#)
 - [Sounds](#)
- [Animated Lightmaps](#)

Volume Brushes

Volume brushes convert the brushes they're bound to into spaces that affect players standing inside them. They're the basis for things such as water, zero gravity rooms, poison gas and teleporters. In order to fill a pool with water, you would make an empty pool. Then you would create a brush that takes up the space of the water in the pool. Last, you would bind the brush to a volume simulation object with water properties.

There are many different types of objects and they tend to vary dramatically from game to game. It's a very good idea to ask your game's coders to carefully document their work and to write explanations on the use of their custom objects, since they are the ones with the greatest understanding of the specific objects used in your game. DEdit itself can provide help on properties on the fly using a file called Classhlp.BUT.

[Top](#)

WorldModels

There are three basic types of WorldModel objects: **WorldModels**, **Doors**, and **Switches**. Each of these three basic types has different movement types as prefixes describing how they behave.

- **Rotating** means the objects will rotate around a point a specified number of degrees around each axis. For example, a common door.
- **Sliding** objects will move a specified number of units in a specified direction. An example of this is the behavior of a desk drawer.
- **Spinning** objects will spin around a point in a specified number of seconds on each axis. An example of this is the behavior of a ceiling fan.

Each movement type has identical properties and behaves exactly the same so when you are learning how to use a RotatingWorldModel you are also learning the RotatingSwitch and RotatingDoor. However, even though a RotatingWorldModel can be used exactly as a RotatingDoor you should NOT put a RotatingWorldModel where a Door should go, and vice versa. Use a Door only for Doors, use Switches only for Switches, and WorldModels for everything else. Game objects, specifically AI and Players, need to know if they are interacting with a Door, Switch or something else.

Most basic functionality is shared for every new WorldModel object. All of the new objects have common features and properties that you can edit. They are all created the same way, they all move and rotate based on their local coordinate system and they can all receive certain messages. Active WorldModels, that is any Rotating, Sliding or Spinning objects, can also send commands to other objects and play sounds depending on what state they are in.

For information on the WorldModel objects included with Jupiter, see [Appendix E: WorldModel Reference](#).

[Top](#)

Creation

To create an instance of a WorldModel just bind a **WorldModel** object to a brush or a set of brushes. There are several property values you can edit to achieve the object you want.

BlendModes is a property drop down list containing several blend operations for your **WorldModel** object. WorldModels can have Additive blending, they can be Chromakeyed, or they can be Translucent. None is the default BlendMode for newly created WorldModel objects.

You can also edit several other properties such as visibility and solidity. If you have a fairly complex brush that the player will be walking on or moving around you may want to set the **BoxPhysics** flag to FALSE.

You can override the surface flags set by the texture mapped to the brush(es) by simply selecting the desired surface in the **SurfaceOverride** property dropdown list.

Once a WorldModel is positioned and behaves as you would like you can rotate it by selecting both the object and all brushes it is bound to then select Rotate Object in the right click menu in DEdit. You can rotate the WorldModel on all axes and in any direction. The WorldModel will now move or rotate with the same behavior around its own local coordinate system.

[Top](#)

Damage

All WorldModels can handle damage. Each one has a subset property labeled **DamageProperties**. There are several different values and flags that can all be edited to define how the WorldModel behaves when damaged. The following list of **DamageProperties** describe the damage types available for each WorldModel:

Armor—Amount of armor that the WorldModel has when it is created.

CanDamage—Toggles whether the WorldModel can be damaged.

CanHeal—Toggles whether the WorldModel can be healed.

CanRepair—Toggles whether the WorldModel can be repaired.

DamageMessage—Command string that is sent to the object that caused the damage.

DamageTriggerCounter—How many times the object must be damaged before the damage message will be sent.

DamageTriggerMessage—Command string that is sent when the WorldModel receives damage from any source.

DamageTriggerNumSends—Specifies how many times the damage message will be sent.

DamageTriggerTarget—Name of the object that will receive the DamageTriggerMessage.

DeathTriggerMessage—Command string that is sent when the WorldModel has been destroyed.

DeathTriggerTarget—Name of the object that will receive the DeathTriggerMessage.

HitPoints—Number of hit points the WorldModel has when created.

KillerMessage—Command string is sent to the object that destroys this WorldModel.

Mass—Sets the mass of the WorldModel within the game.

MaxArmor—Max amount of armor the WorldModel can ever have.

MaxHitPoints—Max number of hit points the WorldModel can ever have.

NeverDestroy—Toggles whether the WorldModel can be destroyed.

PlayerDeathTriggerMessage—Command string that is sent when the player destroys the WorldModel.

PlayerDeathTriggerTarget—Name of the object that receives the PlayerDeathTriggerMessage.

[Top](#)

Attachments

All WorldModels can have attachments (you can attach models, props, and other WorldModel objects). When attached to a WordlModel, objects will move and rotate with the WorldModel.

To have an object attach to a WorldModel when the game loads

1. Add the object you want to attach and position it in DEdit.
2. In the Attachments property field enter the name of the Object you want to attach.
3. Repeat the previous steps for each object you want to attach. You can have as many objects attached to a WorldModel as you want. List all the objects you want attached separated by a semicolon (for example: Prop1; Prop2; SlidingWorldModel0).

During runtime of the game you can attach and detach objects through messages.

[Top](#)

States

All Active WorldModel objects (Rotating, Sliding, and Spinning) have different states they go through. These states are:

- Off
- On
- PowerOff
- PowerOn

Doors have similar states with different names:

- Closed
- Closing
- Open
- Opening

By default, this can be changed in the **Options** property subset. All objects start in the Off or Closed state.

When positioning and rotating the object in DEdit you are setting its Off position. By entering in values for the **Movement** or **Rotation** properties you are setting how and where the object will move to when fully On or Open.

Sliding WorldModel objects will be moving in the direction you specified while in the PowerOn state and when they have moved to the distance you specified they are considered On. While moving back to their original position and rotation they are in the PowerOff state and when they have reached their original position and are at rest they are considered Off.

Rotating and Sliding objects will stay still in their On positions and rotations, but SpinningWorldModels will continue to spin at the specified rotations per second while On.

[Top](#)

Messages

All WorldModels can receive **ATTACH** and **DETACH** messages sent from other objects.

To send an **ATTACH** just send a message to a WorldModel like you would other objects and enter the name of the object you want to **ATTACH**:

```
msg WorldModelName (ATTACH Prop0);
```

Only one object can be attached at a time using this method. When another **ATTACH** message is sent to the same WorldModel the first attachment is detached and the new object is attached.

To detach the object you attached with an **ATTACH** message, send the object a **DETACH** message. **DETACH** takes no additional parameters.

```
msg WorldModelName DETACH
```

All Active WorldModel objects (Rotating, Sliding, and Spinning) can also receive messages to turn On and Off. Messages that Active WorldModels can receive are:

LOCK—Locks the object. Once locked the object cannot be activated.

ON—If not already in the On or PowerOn states, puts the WorldModel in the PowerOn state.

OFF—If not already in the Off or PowerOff states, puts the WorldModel in the PowerOff state.

TRIGGER—Toggles the state. Basically the same as when a player presses use against this object. If the WorldModel is in the On or PowerOn state it will immediately switch to PowerOff. If it's in the Off or PowerOff state, then it will switch to PowerOn.

TRIGGEROFF—Same as OFF.

TRIGGERON—Same as ON.

UNLOCK—Unlocks the object so it can now be activated by the player and messages.

These messages take no other parameters so an example to toggle a SpinningWorldModels current state would be:

```
msg SpinningWorldModelName TRIGGER
```

[Top](#)

Options

All Active WorldModel objects (Rotating, Sliding, and Spinning) have several options that can be changed in the Options property subset group. These options control how interactive the objects are and set some behavior.

Locked—When TRUE, will start the object off as being Locked. While Locked the object cannot turn on or off until it is unlocked through a message.

PlayerActivate and AIActivate—These let the object know who, if anybody, can turn them On or Close them. If these are FALSE then only other objects can interact with it by sending it messages. If PlayerActivate is TRUE then a player can trigger the object by pressing “use”.

RemainOn—If TRUE, will keep the object in its’ On state until told to turn off, either by the player or another object.

StartOn—When TRUE, puts the object in the PowerOn or Opening state as soon as the game loads.

TriggerOff—Toggles whether or not the object can be turned Off.

Waveform—A dropdown list of movement types that the object will use to rotate or move.

[Top](#)

Commands

All Active WorldModel objects (Rotating, Sliding, and Spinning) can send commands based on what state they are in. There are 5 commands listed in the Commands property subset group:

- LockedCommand
- OffCommand
- OnCommand
- PowerOffCommand
- PowerOnCommand

Doors have similar commands with different names:

- ClosedCommand
- ClosingCommand
- OpenCommand
- OpeningCommand

Each command line can have multiple commands, separated by semicolons, entered and each one will be executed every time the WorldModel object first enters the corresponding state.

To have a command executed on an object as soon as it is activated, either by the player pressing “use” or by receiving an On message, simply enter the command in the PowerOnCommand or OpeningCommand line.

If you would like a command executed at the end of an object’s movement or rotation enter it in the OnCommand line.

The LockedCommand will be executed if the object is locked and a player or message tries to activate it.

[Top](#)

Sounds

All Active WorldModel objects (Rotating, Sliding, and Spinning) can play sounds based on what state they are in. There are 5 sounds listed in the Sounds property subset group:

- LockedSound
 - OffSound
 - OnSound
- PowerOffSound
- PowerOnSound

Doors have similar sounds with different names:

- ClosedSound
- ClosingSound
- OpeningSound
- OpenSound

Other properties that can be edited in this subset group are the relative sound position, the radius the sound fades off to and whether or not the sounds will loop.

The **LockedSound** will never loop.

Each sound is a single .WAV file that is easily picked through a browser. The sounds will start playing every time the WorldModel object first enters the corresponding state.

To have a sound start playing every time the object starts to turn off or close enter it in the **PowerOffSound** or **ClosingSound**. The **LockedSound** will play if the object is locked and the player or a message tries to activate it.

[Top](#)

Animated Lightmaps

All Active WorldModel objects (Rotating, Sliding, and Spinning) can have animated lightmaps associated with their movement or rotation.

To create animated lightmaps

1. Add a **KeyframerLight** object.
2. Set the properties of the **KeyframerLight** as desired.
3. Position it close to the **WorldModel** object you would like to lightmap. If it's a directional light then make sure the light is pointed in the right direction.
4. Setting **ShadowMap** to FALSE typically looks much better.
5. Once the **KeyframerLight** is set just type the name of the **KeyframerLight** object in the **ShadowLights** property. You can enter up to 8 lights in the **ShadowLights** property separated by semicolons.

Enter the number of frames you would like have in the animation, up to 128. Sometimes less frames actually look better but experiment and use what works best. Remember that the more animations and the more frames in the animation is a real memory hog. Too many animated lightmaps is also a real frame rate killer!

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Lighting the Map

Jupiter uses a combination of lightmaps and vertex lighting to light worlds. Lights are represented as a combination: A point that shows the origin of the object and a circle, square or cone representing their area of effect.

Lightmaps are textures generated by the processor at world compile time. Lightmaps contain ray-traced lighting information for an individual polygon. Lightmapping can be used on all world geometry, along with world models. It provides detail, but it also requires much more horsepower (an extra pass) to render. Lightmapped polygons are twice as expensive to render as Gouraud shading, the other way of lighting a brush. In this model, brushes are lit by vertex colors, generated at compile time, and gouraud interpolated across the polygons. You can subdivide Gouraud-shaded brushes to add vertices, thus adding detail to the lighting. In most cases, this mode of lighting is highly effective and efficient, and you should try to use it wherever possible. Models are dynamically lit with normal lighting, attenuated with the equations below.

Note: This section assumes you have already created the Big1.LTA file in the [Using Brushes](#) tutorial.

Adding a Light to Your Level

To add a light to your level

1. Switch back to your Big1.LTA file, or open it if you have closed it.
2. Move the green marker up 64 units above the start location and go back to the Add Object dialog.
3. Select the old one that is surrounded by a large sphere. The sphere is the radius within the level that the light will fill.
4. Go to the Properties tab in the project window and look at the properties of the light. In most cases a simulation object's properties are the most important tool used to control its behavior. In the case of lights, this is where you can change their size, brightness and color.
5. The light's current size (set in the LightRadius property) is 300. That's probably a little small, so change the value to 500 and press the Enter key. In the viewports you'll see the size of the light radius increase as well. Now it's large enough to make a good-sized pool of light around the player.

[Top](#)

Lighting the Entire Map with a Single Source

There is also an additional lighting resource that is very useful with terrains and large outdoor areas. You can also use a StaticSunLight object to light the entire map with a single source. StaticSunLight objects require a working skybox in order to light the map. Most of their properties are self-explanatory and similar to a DirLight, but without FOV (Field Of View).

The light from a StaticSunLight projects in from the level's sky portals. The main different property is Bias, which controls the softness of the light when it falls on terrain. Varying the light's Bias varies the balance between light and shadows from the StaticSunLight.

StaticSunLights will illuminate any brush or model, so a single StaticSunLight can provide all of the main lighting for an entire level if it's largely open. It is still recommended to place individual point sources around your level for highlighting and fine control.

[Top](#)

Interior and Exterior Color Properties

Jupiter is a little unusual in that its lights have both interior (LightColor) and exterior (OuterColor) color properties. By default, the interior color of the light is set to white and the exterior color is set to black. That means that the light starts out as a pure white, then fades out to black at the edge. This becomes useful in areas that have a non-black ambient light, such as a cavern of lava lakes, or an outdoor scene on the savannah in bright daylight. If you wanted to fade a light out more gradually in one of these settings, you could set the exterior color to a mix of black and the ambient light's color for a slower fall-off.

To change the colors

1. Switch back to your Big1.LTA file, or open it if you have closed it.
2. Click on the color sample button next to the property name.
3. Click the white button (LightColor). The dialog that comes up is a standard color picker dialog, and it allows you to pick from a standard palette or create your own custom colors.
4. Choose one of the very light blue colors for something similar to a fluorescent light. You can use the slider on the far right to lighten or darken the color to your taste.
5. When you're done, close the dialog and you'll see your new color in place on the LightColor button.
6. The OuterColor property is changed in exactly the same way.
7. Lastly, you can set the brightness of the light using the BrightScale property. At its default setting of 1.0, the light appears at a normal brightness. If you change the value of BrightScale to 0.5, the light's brightness is halved. However, the radius stays the same. Thus, you can create a very large light that doesn't overwhelm the area closest to its center. Likewise, if you want an intensely bright light, you can increase the BrightScale value to 2.0, which will make the light much brighter. For now, leave BrightScale as it is.

[Top](#)

DirLight

There is another useful type of light with very different behavior from the regular one: The DirLight. While a normal light throws light in a sphere all the way around it, a DirLight acts like a spotlight: It throws light in a cone, the diameter of which you can control.

1. Switch back to your Big1.LTA file, or open it if you have closed it.
2. Switch to the Top viewport and move the marker 256 units (4 lines on a 64-unit grid) down, towards the edge of the platform furthest from your archway.
3. Move the marker 64 units closer to the floor, so that your DirLight will show up better on the floor.
4. Right-click in one of the viewports and select Add Object again.
5. Find the Dirlight class and add one. What you should see is a pyramid-like cone extending from the marker towards the top edge of your platform. The cone represents the area where the light will fall within the game. If you look at the Properties tab, you'll see that the Dirlight has most of the same properties as your regular light from before, but it has an additional property, FOV, which stands for

Field of View. It represents the arc within which the Dirlight actually sheds its light. You can increase this value to widen the pattern or decrease it to reduce the pattern's width.

6. Change the FOV to 60. You should see the base of the light's preview cone shrink down in size.

[Top](#)

Aiming the Dirlight

You can aim a Dirlight using the object's Rotation property, or rotate it freehand using the N key and mouse.

Click the button next to the property's name and you'll see a dialog appear. Within this dialog, you can control every aspect of the object's facing. Each of the three aspects has a range between 0 and 359.

Yaw controls the horizontal facing of the object, and is aligned identically to the Top viewport like a compass needle. You can use the preview display at the top of the dialog to see how the results will appear. A setting of 0 points the object straight "north," towards the top edge of the Top viewport. A setting of 90 would aim "east," a setting of 135 would point "southeast" towards the lower right corner of the viewport and so on. For now, set the yaw to 180 to spin the Dirlight around so it's facing the opposite direction.

Pitch controls the vertical angle of the object. The Pitch preview display shows the result of your changes. Since the Dirlight is pointed at the proper angle already, there's no need to change this.

The Roll control controls the spin of the object. This is useful mainly for props or cameras, where you want the object to bank or its viewpoint to sway. You would change a motorcycle's Roll, for example, if you wanted it to lean as it turned a sharp corner. Again, there's no need to change this value in the case of a Dirlight.

You should also increase the LightRadius of the light to 600 to extend the reach of the light a little further.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Optimizing Your Level

It's actually fairly easy to make a gorgeous Notre Dame-scale cathedral in DEdit. Once you know the basic controls, you can pretty quickly lay out the brushes, apply the textures and add in the lighting necessary to make such a structure. However, it is also extremely easy to build a level that:

- Will not compile
- Will take far too long to compile
- Will run poorly in the game even after a full optimization

This is truly where the science of level design begins to take a back seat to the art. Though there are some basic rules and a lot of guidelines to optimizing levels, the best way to learn good optimization techniques is to experiment and to carefully observe the results you get inside your levels. You must think about optimization right from the moment when you first begin designing your level, and you must watch the performance of your level every step of the way as you build.

Unlike any other computer art product, computer games are rendered in real time, and this restricts you substantially in what you can get away with in your levels. This section contains the following topics related to optimizing your level for the best compiling, rendering, and run times:

- [Polygon Counts](#)
- [Measuring Polygon Counts](#)
 - [Lightmapping](#)
 - [Complex Geometry](#)
 - [Large Levels](#)
 - [Efficient Visibility](#)
 - [Blocking Visibility](#)
- [Visibility Control Strategy](#)
 - [Wireframe](#)
 - [Overdraw](#)
 - [Texture Concerns](#)
- [Simulation Object Concerns](#)
- [Optimization Warning Signs](#)

Polygon Counts

Levels can become bogged down for a number of reasons. The first of these is high polygon counts in a scene. Even a simple empty room that has only walls, floor and ceiling can be made up of several hundred polygons. In order to apply lighting and tune the level's performance, the brushes that you specify inside DEdit are broken down, juggled around and reshaped by the Processor as it builds the in-game .DAT file out of your .LTA file. As mentioned above, this process is one of the reasons why you can't edit a .DAT file in DEdit.

[Top](#)

Measuring Polygony Counts

To measure the polygon counts in your game, you use the ShowPolyCounts console command. As a general guideline, the game designers for TO2 kept multiplayer levels to around 30,000 polygons/scene in general on multiplayer levels and 40,000 or so for single-player levels where frame rate was not as vital. It's important to perform some tests of your own on your target platform to develop your own standards.

[Top](#)

Lightmapping

Lighting a level via lightmapping requires that large brushes be broken down into smaller chunks. It is also useful to subdivide brushes for Gouraud lighting as well, since more vertices lead to greater Gouraud detail. This usually happens along texture boundaries. Thus, if you have a large room with a 64x64 floor texture, the floor may be broken up into dozens of 64x64 chunks when in the editor it is shown as a single brush. The reason this is done is to create the lightmap textures and extra vertices for Gouraud lighting. In some very large rooms it can add significantly to the area's polygon count. It's important to take that overhead into account.

There are ways to reduce this overhead. The LMGridSize parameter listed in the WorldInfo string section describes how you can scale up your lighting splits in order to reduce the number of polygons created, which can lead to a decent savings in poly counts for large rooms.

You can also control the LMGridSize for individual brushes with the LMGridSize property in a brush's LightControl dialog. Gouraud lighting can require fewer extra polygons in areas where the light is relatively uniform, but may require many more in areas of high contrast.

[Top](#)

Complex Geometry

The second cause of high counts is complexity. It's very hard to resist the urge to put every detail into your geometry instead of using textures to create detail. However, it is much more expensive to add detail using geometry than it is to imply detail through tricks of the eye, lighting and texturing. Avoid nonfunctional detail that can be added through textures like molding/trim, curlicues, grooves and bumpy surfaces. A level designer's art is figuring out how to imply detail. Ask your texture artists to build a grooved surface or the face of a curlicued building ledge, and then apply that texture to a simple flat surface. Use as few sides as you can get away with on your columns and carefully consider how to build each shape with the fewest polygons.

[Top](#)

Large Levels

Each polygon in a level must be stored in the .LTA and .DAT file. That means each polygon takes up space, both in memory and on disk. Polygons take up less space than they did in previous versions of LithTech, but since the number of polygons per level has increased more than ten times, your levels must be smaller than in previous SDKs. Alternately, you can reduce the polygon density of your levels back to that of the previous SDKs and create larger levels. Doing this, you could have a level that was larger than levels in previous versions, but the level's polygon counts per scene would be at the level of the older SDKs.

If you wish to take full advantage of the higher polygon densities available in Jupiter, you should expect to make levels about one quarter the size of the levels in older SDKs. This will ensure that your levels will fit within the memory of average machines, and that the data sets will be simple and easy for the engine to traverse.

You should also remove any polygons that won't be visible to the player. In previous versions, LithTech Processor would automatically remove hidden faces via CSG, but this operation was the main reason for the limitations on concave brushes in that version. In Jupiter, you should build more in the way you might build in an external 3D package: Make the level a shell, with only faces that point inward towards the player actually attached. Also, you should try to avoid useless detail that can be better expressed in the texture or do not contribute to the level's functions or lighting. Useless polygons do nothing but reduce performance.

[Top](#)

Efficient Visibility

The computer and the player see the level in different ways. The player can't see anything that's behind a solid wall or any solid surface. However, the computer always looks just a little bit ahead and pre-draws some objects that may not yet be visible to the player. This is because of the sorting mechanism used by the system to generate the list of visible polygons.

A problem area usually comes in the form of other rooms that add to the poly counts in the current areas. Sometimes the increase in counts makes very little sense. You can have a very small room without any real detail such as an entryway with the polygon counts of a much larger, more detailed room. If you look around in a room like this, you can almost always find a hallway or other hole into another more detailed room.

[Top](#)

Blocking Visibility

Visibility in Jupiter is very different from visibility in previous versions of LithTech. Jupiter supports very high numbers of polygons, but you must still do some visibility control work. Whereas in previous engine versions visibility control needed to be extremely tight, in Jupiter, world polygons are one of the cheapest things to render. Although it is still valuable to do visibility control, the preferred method in Jupiter is to control visibility in much larger, simpler chunks with as few visibility blockers as is possible to reduce the overhead of sorting.

By default, nothing in your level will block line of sight in Jupiter. You must manually establish visibility blockers known as Occluders. In Jupiter, an Occluder is a one-sided polygon that blocks the player's line of sight. Models and groups of brushes on the far side of the occluder will not be drawn. Once the player moves around the occluder, they become visible. Skyportals aren't blocked by the occluders, so your player will still have something to look at if you build an occluder outdoors.

You can turn any brush into an occluder by changing its Type property to Occluder. Since occluders are visibility structures, you should keep them as simple as possible. Make your occluders single-sided to eliminate unneeded cuts. The occluder will still block from either side if it's one-sided. You can insert occluders in between the floors of buildings, at the tops of ridges, or in walls of a large building.

Just remember, simpler is often better. Occluders should be large, broad and few in number. Many TO2 levels have only 12 occluders. Some have none at all. If you have a level where the player will only dip behind occluders on rare occasions, it may not be worth having them. In these cases, just control the polygon counts in the area directly by simplifying objects or redesigning areas.

Objects will only be hidden by an occluder if they are completely behind it. In the case of brush polygons, brushes are grouped together by Processor into collections called render blocks. Each block renders as a group: either all its polygons are visible or none of them are. If one corner of the block is visible, then the whole block will draw. Once all the brushes in a block are behind the occluder, it will cease to draw. Another important thing to be aware of is that the player's viewing angle affects what the occluder will block. The occluder and the player's view form a frustum that determines what can be seen behind the occluder. The further away from the occluder the player stands, the less the occluder will block.

The screen capture below shows a scene from TO2 in which wire boxes indicate the render blocks. The white panels are occluders.



If two occluders touch, they operate cumulatively. In other words, if a render block is partly behind Occluder A and partly behind Occluder B, the block will not be rendered as long as A and B are between the player and the render block. This is true even if the two occluders are of different sizes or heights, as long as they're both tall enough to conceal the render block.

You should design your levels with occluders in mind. Make sure that the occluders can be placed where a player won't notice parts of the world flashing in and out of view. Try to make areas where the player will need the highest frame rates (boss battles, multiplayer arenas) lower in detail. Avoid very long fields of view through detailed settings, or make sure that you plan carefully to keep the total polygon counts in these areas under control.

[Top](#)

Visibility Control Strategy

In Jupiter, it's generally better to keep visibility controls simple rather than making them complex. The Jupiter engine does better when it has a shorter list of visibility blockers to run through. Since the occluders work on groups of brushes and checks their corners, it's best to place most of your occluders aligned to the grid. Unlike hullmakers in older LithTech SDKs, which created a portal between areas, occluders simply block. Angling them generally reduces the amount of surface area they present to the player, lessening their effectiveness. Further, if you angle the occluders, they will slice into render blocks in order to ensure that blocks never lie halfway across an occluder. This creates extra render blocks, leading to decreased performance. In some cases, this can create a really large number of extra render blocks.

A render block of world polygons always includes 2,000 polygons. Processor splits the level in half, then checks to see if either half is 2,000 or fewer polygons. If a half is not, it is split again. The splits continue until every block contains only 2,000 polygons. If there are less than 2,000 polygons left for the last block, the remaining polygons will all be stored in the last render block. However, since Worldmodels and models can move independently, each Worldmodel or model object forms its own discrete render block, regardless of its polygon counts.

Keeping the total number of render blocks low is important to good performance, so you should try to avoid creating extra models or worldmodels when you don't need them.

If you need to prototype the visibility structure of a level, you must make sure that you add polygons to the level that approximate the counts and locations of the polygons you expect in the final level. This is again due to the 2,000-polygon threshold for render blocks. If there are too few polygons in the level, it may be reduced down to a single render block. As a result, you will not be able to learn much about how the level's counts will behave in the final level.

[Top](#)

Wireframe

The easiest way to watch this process is to use the Wireframe console command. It switches your view to a wireframe view of the level. Since all brushes are transparent in wireframe view, you can see right through a wall into rooms on the other side. This means you can also see what brushes are being drawn in rooms other than the one you are in. Though it takes some getting used to, this mode makes it possible to figure out exactly when and where the view of a room or object is cut off, making it extremely useful when you want to check your visibility blocks.

[Top](#)

Overdraw

Another concern is a factor called overdraw, which is the number of polygons drawn on screen that are actually out of the player's view behind other polygons. This is related closely to the visibility optimizations referred to previously. You can often find high levels of overdraw in rooms with lots of detail: columns, low walls, complex fixtures and cracks can all lead to overdrawing. Overdraw in a scene is measured using the ShowFillInfo console command. In the development of the game Shogo, for instance, values of 2-3 here were considered acceptable and values of 4-6 were danger signs. Once again, testing on your target platform to obtain figures for your own game would be wise.

[Top](#)

Texture Concerns

Besides geometry-related factors that may slow down your game when it's running, you can also get significant slowdowns if you use too many textures or several overlarge textures in a scene. Even in the age of AGP, if you use a 512x512 texture for everything from tiles on the bathroom floor to murals to the nameplate on a door, the game will run more slowly. As with many problems, this is best addressed in the design stages rather than in the testing stage.

You should try to resist the urge to use the largest texture possible and instead work to find a balance between a texture that looks good and is small in size. Objects that won't get much scrutiny like floor/ceiling tiles, wall-to-wall carpets and grass should usually be given small textures. Objects that the player will focus on (view screens with data, pictures, signs) can use higher-resolution textures.

[Top](#)

Simulation Object Concerns

Simulation objects can also affect game performance. If you build a huge particle fountain that creates 1000 particles every second with a decay time of 10 seconds on each particle, the engine is going to be very busy keeping track of the particles. Likewise, large, moving objects such as props made from WorldModels can slow the engine down as it tries to manage their physics.

Character models and AI can also affect frame rate. A level that runs fine with just the player on it can bog down seriously when two very intelligent AI enemies are added to it. Likewise, a Boss character with thousands of polies in her character model will slow the engine much more than a minor character with just a few hundred.

Be careful how you code, design and place your AI. Also, be sure to test the results to make sure you haven't stressed the system too much. The ShowTickCounts command gives useful information for figuring out these sorts of generalized issues by listing what portion of the engine's time is spent in what area, be it rendering, sound or game code.

[Top](#)

Optimization Warning Signs

As mentioned before, optimization is more art than science, and it takes practice. The warning signs of trouble you should always heed are low frame rates, high polygon counts in a scene, and sudden jumps in the length of time it takes to process your level. Any of these should make you take notice. Immediately backtrack and figure out what has caused the issue.

Even if you don't do anything to correct it at the time, performance issues are almost always easier to fix if they're identified early. It is heartbreaking to have to discard a level that is completed but completely unworkable, and you will do so if you don't pay careful attention to optimization throughout the construction process.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



—TUTORIAL— Building Your First World

This tutorial walks you through the process of creating a new world using Jupiter and DEdit. This tutorial contains the following world creation topics:

- [Creating a New World](#)
- [Making a Simple Room](#)
 - [Adding Objects](#)
- [Processing the World](#)
- [Testing the World](#)

Creating a New World

The first step we'll want to follow in building our new world is to pick a project. A project specifies all the resources you'll have access to in your world, from the game code on up. Generally, you'll only create or use one project per game. Projects can be modified over time very easily, since they simply consist of a collection of files laid out on disk in a structure that DEdit can recognize.

To create a world

1. In DEdit's File menu select the Open Project command, or press CTRL+O.
2. Navigate to the No One Lives Forever 2 game resource directory in the following location:

\<installation directory>\Development\TO2\Game\

3. Open the TO2.dep file.

TO2.dep is the project's root file, and contains a variety of different data about the project. .dep files are the only type of file directly opened with DEdit.

4. On the File menu, click New World.
5. When DEdit asks you what to name the world, type Simple1. This is the name your world will have when you open it in DEdit.

Note: The name you provide is also the name of the world's file on your hard drive (for example, Simple1.LTA).

[Top](#)

Making a Simple Room

Our first world is going to be very, very simple. We're going to make a box with a light. If you're a programmer, this may be as complex of a level as you need to create, since you'll mainly be making levels only to test your code and not for inclusion in the game itself.

Choosing a Texture

Even though this level is extremely simple, we'll still want to texture it. When you select a texture, it will be applied to all brushes that you create in the world. It's like choosing a font in a word processor. When you select the Arial font, all the text that you enter is Arial until you select a new font.

To select a texture

1. Select the Textures tab in the Project Window. This tab is where you select textures to apply to your brushes.
2. In the top section of the Textures tab, there's a box with folders listed in it. Click on the folder that is called Tex. When you do so, the list box in the middle of the tab should fill up with subfolders that break the textures up into categories and types to make it easier for designers to find the right texture quickly.
3. Navigate to the Japan subfolder and then click on the folder to select it.
4. In the list of textures choose a texture named SFJp006.dtx. Notice that there's a preview of SFJp006.dtx in the bottom window of the tab. From now on (until you select a different texture) this texture will be applied to every brush you create.

Using the Viewports

Move your mouse pointer over the upper-right-hand viewport (top view). The green crosshair (also called the marker or the insertion point) operates like the cursor in a word processor: it shows your insertion point. If the viewports are your eyes, then the marker is your hand. We can leave the marker where it is for now, but you'll learn how to move it around shortly.



Camera control is done in DEdit using the I, O and C keys in conjunction with the mouse.

The following table lists a few of the most important key and mouse combinations for moving the camera view.

Press	Result
I + Mouse	In the Perspective View, hold the I key and move the mouse pointer to scroll in the X and Y plane. In all other views, this combination scrolls in all available directions.
I + Left Mouse	Same as above, but faster.
I + Right Mouse	Scroll along the Y-axis in the Perspective View.
O + Mouse	In the Perspective View, hold the O key and move the mouse to pan the camera in all directions. In all other views, this combination zooms in and out of the displayed geometry.
O + Left Mouse Button	In the Perspective View, hold the O key and the left mouse button to move or "fly" forward through the level.
O + Right Mouse Button	In the Perspective View, hold the O key and the right mouse button to move or "fly" backward through the level.
C + Mouse Button	In the Perspective View, hold the C key and move the mouse pointer to adjust the draw distance.
C + Left Mouse Button	Same as above, but faster.

Since we're zoomed in fairly close to the grid, we should zoom the camera out a little in the viewport so that we can see more of the grid at once.

- Hold down the O key and move your mouse pointer slowly up. You should see the grid in the upper right viewport getting smaller. We're actually zooming out to see more of our map, not changing the scale of the grid.
- Don't zoom out farther than you need to in order to see a few more grid squares on the screen (perhaps four or six squares across the screen).

- Your screen may be at a high enough resolution that you can already see six or so grid squares. If that's the case, play with the camera a little but make sure you come back to a scale of about 4-6 squares across.

Creating a Floor

First, we'll place a brush to represent the floor of our room.

1. Press CTRL+B to switch into Brush mode. This is the mode reserved for moving brushes, as well as adding them to the world.
2. From the center of the marker, count up two gridlines and then over to the right by two gridlines. Move your mouse pointer to that intersection in the gridlines.
3. Once you have the mouse over the intersection, press the SPACE key once and move the mouse away. You should now have a line following your mouse pointer. The line you see is the first side of the brush you're about to create.
4. To continue the brush, move your pointer until it's over the grid two gridlines below and to the right of the marker and press SPACE again. DEdit drops the vertex on that point and adds a new vertex and side to the brush.
5. Now move the mouse and place another new vertex four grid units to the left of the current vertex.
6. Add your next vertex four units straight up (two units above the green marker).
7. Last, move the mouse pointer back to the vertex you started at.
8. When you press SPACE over the original vertex, DEdit displays a dialog box that asks you to enter the thickness of your brush. Enter a value of 64 and click OK. This completes the creation of the brush.

Your room should now look something like the following image:



In the 3D Perspective viewport, you can see that your square has been created and given a height of 64 units. This square will form the floor of our simple room. Each time you create a brush in the editor you will be prompted to give it a thickness. The 64-unit thickness was chosen to make the floor brush easy to find in the editor by eye.

Using relatively large units where possible also helps in other ways: You can more often let the editor align your textures for you if you build to the grid, since typical texture sizes fall along the grid as well. If your textures are mostly 32x32, 16x16, 32x64 and so on, laying textures out becomes a very simple task when you build to the grid. You'll need to scale things to fit much less often, you can more easily make use of textures that tile together and you can realign a texture much more simply. We'll talk about that in a later section as well. For now, assume that building to a large grid where possible is a good rule of level design practice.

Creating a Ceiling

Next, we're going to use the Copy and Paste feature to create a ceiling for our room.

1. In the Edit menu select the Copy (CTRL+C) command. Now there is a copy of the floor brush on the clipboard.
2. In the Edit menu select Paste (CTRL+V) to paste. If you look in the perspective viewport (the one in the upper left), you should now see the new brush near your original brush.
3. Since the brush isn't lined up quite as you'd want it for a ceiling, you'll want to move it around a little. In the Top viewport (upper right), press the O key and click the mouse to zoom out until you can see both brushes.
4. Move the mouse to the black drag handle in the center of the brush. Drag handles can be used to move or to resize the objects they're attached to. In this case, click the left mouse button and drag the center handle towards the center of our original brush (which should still be the center of the marker as well).
5. When you have the center handle over the marker, your selected brush should be lined up exactly on top of your first brush.
6. Drop the brush there and move your mouse to the bottom right viewport (Front).
7. Use the O key again to zoom the viewport out until you have a good view of the selected brush.
8. Click and drag the center handle of the brush until you've moved the brush straight up 4 grid units. Don't worry if that means dragging the brush out of the visible area. The viewport will scroll up for you when you reach its edge.

Your ceiling is now a good height above your floor, at 192 units. That's about the height of a tall conference room ceiling in most games.

About the FALSE Detail Brush

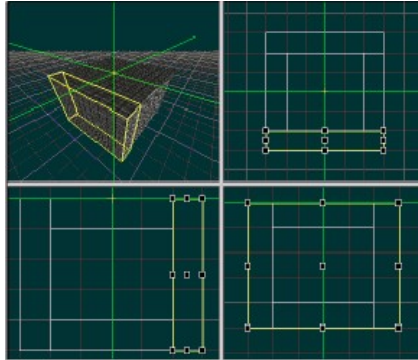
For reasons that are unimportant, one brush in each of your levels must have its detail property set to FALSE. It should be a brush with a simple shape, and it cannot be concave. Usually, if there's not a brush in your level that's cube-shaped, you can just add a 64x64x64 cube somewhere out of sight. We will use the ceiling brush we just created as our non-detail brush.

1. Click on the Properties tab in the Project window.
2. Find the Detail property.
3. Click the Detail property to change it from True to False.

Creating Walls

Now we need to create four walls.

1. In the Front viewport, hold down the X key and move the mouse upwards a few grid lines. What you should see is that the green marker moves wherever you move your mouse. Since the marker represents the insertion location where new brushes will appear, we'll need to move it upwards until it reaches our ceiling in order to add our walls. Move the marker until it's at the same grid line as the top edge of the ceiling brush.
2. Switch back to the Top viewport.



3. Use the O key to zoom out a little further until you can see a few rows of grid around the selected brush.
 4. Use the SPACE bar to draw a rectangular brush that's 1x4 grid squares and runs along the whole right edge of your ceiling brush.
 5. When you're prompted for a thickness, give it a value of 320. That's a lot thicker than you used for the floor, but there's a reason. We want this wall to reach all the way from the ceiling to the floor. If you'll look in the Front view, you'll see that our new brush does just that, neatly sealing off one side of our room.
 6. To make the next wall, copy and paste your new brush.
 7. Then use the center drag handle on the pasted brush to move it until it seals the west side of the ceiling brush the same way that your first brush seals the east side.
 8. Now we have to make two more walls. Using the SPACE bar to draw a 6x1 320 unit tall rectangle on the north edge of the ceiling brush. By making it 6 grid squares wide, you'll make a good seal against both the floor/ceiling brushes and also against the two existing wall brushes.
- Note:** Sealing a level is another topic that we'll cover later. For now, accept that it is important.
9. The last step you'll need to take making brushes for your level will be to copy your latest brush, paste it and move it to cover the south edge (in the top viewport) of the ceiling brush.

Now would be a good time to save your level again.

Your room now has solid walls, a floor and a ceiling. You've created 75 percent of your level. Follow the next set of instructions to add some important objects.

[Top](#)

Adding Objects

There are only three objects left that you need to add in order to see your level:

1. A GameStartPoint (which specifies the location where your player will first spawn into the world).
2. A light (without which you won't be able to see your level).
3. A WorldProperties object (which specifies some of the rudimentary information necessary to create your level).

Perform the following three procedures to add your start point, lighting, and world properties.

To Place the Start Point

1. Switch to the Front viewport so that you can move the marker vertically.
2. Use the X key to move the marker down until it is one grid square above the floor. Since 1 grid square is 64 game units, this ensures that your player won't accidentally appear halfway into the floor.
3. You should also check the Top viewport and make sure that the marker (if it isn't still at the very center of the level) is at least 1 grid square from any of the walls.
4. Press CTRL+H to enter Object mode. Object mode is the main mode you use for creating, moving, and setting the properties of objects.
5. While in Object mode, select Add Object from the World menu.
6. In the Objects dialog, the left pane of the dialog lists all the objects that your game code supports. Scroll down the list until you see a GameStartPoint object. Select it and click OK.

Once you've done that, look at the center of the marker. The small white square is the newly created start point. Your player can now actually enter the world.

To Place the Light

1. Move the marker vertically upward one grid square, using the X key in the Front viewport. Although you could add your light right at the location of the GameStartPoint object, it will be easier to select later if you add it at a different location.
2. In the World menu select Add Object again. This time, go through the list of objects until you find one called Light. Select it and click OK.

Your start point will become unselected, and a small new cube will appear at the center of the marker. You should also notice a bluish circle that surrounds your whole level. That circle represents the radius of the light in the level. The light fades out as it gets further from its center point, and it ends completely at the edge of the blue line. Since you have a small level, one light will illuminate it all without trouble.

To Add a WorldProperties Object

1. Use the X key to move your marker forward one grid square in the Top viewport.
2. As before, select Add Object from the World menu, scroll down, select WorldProperties, and select OK.
3. Save your world.

[Top](#)

Processing the World

The last step you'll need to take in preparing your level is to process it.

To process your world

1. In the World menu, select Process and you'll see a large dialog box appear.
2. Make sure that the Import Geometry, Apply Lighting and Full optimization checkboxes are checked. If they aren't, then check them. You can leave the rest of the checkboxes as they are by default.

Note: Don't worry if the Import Geometry box is grayed out. DEdit is just assuring that the option's turned on, since the level has never been processed before.

3. Under Project Directory, enter the full path to the No One Lives Forever 2 game resource directory (\Development\TO2\Game\, for example).
4. Click OK to process the level.

The dialog box will change into a progress monitor and a log of what the Processor's doing. Once an OK button appears in the dialog, your level has been processed. Your world's now ready for you to explore.

[Top](#)

Testing the World

1. To actually load your level in the game, follow these steps:
2. Click on Edit, and the Options, to bring up the DEdit Properties menu.
3. Click on the Run tab.
4. In the Executable field browse to LithTech.exe.
5. In the Working directory enter the location of the root TO2 directory (\Development\TO2\Game\).
6. In the Program arguments field enter the command string -rez Game +runworld %WorldName%. Note that the WorldName parameter is case-sensitive, so you must capitalize the W and N.
7. Launch your world by clicking World and then Run. Alternately, you can launch the world by pressing CTRL+ALT+R, or by clicking the Run World button on the tool bar.
8. Examine your new level.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



—TUTORIAL— Using Brushes

A brush is the basic unit of world geometry. A brush is made up of planes that define its faces, lines that define its edges, and vertices that define its corners. All brushes must be convex. You can manipulate either an entire brush at once or any of the vertices, lines, or planes in the brush.

This section contains the following brush tutorial topics:

- [Starting the Tutorial](#)
- [Adjusting the Grid](#)
- [Creating the Floor](#)
- [Making Complex Brushes](#)
 - [Setting Markers](#)
 - [Selecting Brushes](#)
 - [Manipulating Brushes](#)
 - [Using Brush Power Tools](#)
- [Understanding Invalid Brush Types](#)

Starting the Tutorial

This tutorial explains how to select and manipulate brushes, as well as default brushes called primitives, powerful brush tools, and the origins, problems, and solutions for invalid brushes. First, however, you'll see how brushes can make it quick and easy for you to create complex items in your world.

With DEdit for Jupiter you can quickly create custom brushes of any shape or size. For this tutorial, we must first create a new world, and call it Big1. Create the new world just as you did in the [Building Your First World](#) tutorial.

Also, we need an appropriate texture for our floor. In DEdit, go to the Textures tab, select the Tex\Japan container, and then select the texture SFJp003.dtx. This is a good-sized tiling texture that will make a good floor.

[Top](#)

Adjusting the Grid

The grid is a way of organizing the placement of objects, similar to a ruler. It keeps you from laying out points totally at random and it allows you to measure the size of the objects you're creating and their relative positions.

Using the grid makes for much more efficient level layout. When you build to the grid using standard units, your walls are more likely to line up properly and others can more easily understand your level just by looking at it.

To Adjust the Grid

1. Go to the Top viewport (the upper right one).
2. Look in the Status Bar for the Grid box, which shows the size of the grid blocks in the current window. Make sure that it says Grid: 64, indicating a grid block size of 64 units.
3. If your grid setting shows something other than 64, you'll want to adjust the grid scaling.
 4. First, single-click in the Top viewport to make sure that the focus is in that port.
5. Then, either use the Expand/Shrink Grid toolbar buttons or the PLUS SIGN (+) and MINUS SIGN (-) keys on the numeric keypad to scale the grid up or down until it's at 64 units.

Note: Each viewport can have a different grid setting, according to the user's preference.

While 64 units is a reasonable grid block size for laying out floors, walls and ceilings in indoor spaces, if you were working on a large, open outdoor level, 256 units might be a more manageable grid scale. Your goal is always to use a large enough grid to allow you to tell how big your current brush is going to be, but still small enough to create whatever detail you're going to create.

[Top](#)

Creating the Floor

You'll want a floor that's about 512 units on a side, centered on the marker in the Top viewport. That means you'll want to make a square that's eight grid squares on a side. Create one now, and you should get a brush that looks like this:


[Top](#)

Making Complex Brushes

Now that you have a floor, the next step will be to use DEdit to make complex brushes, beyond the simple squares that you've used so far. You'll also learn tips and techniques for good brush construction.

The simplest technique is to use DEdit's vertex-by-vertex brush creation method to build a brush that is close to what you want right from when you lay the brush down. Unlike many other editors that force you to start with simply shaped primitives such as squares or pyramids, DEdit allows you to create a brush by placing its separate vertices wherever you want them. While this doesn't make it possible to instantly create anything, it speeds things up a lot.

To create a complex brush

1. On the Textures tab, select the texture called COJp002.dtx in the same container. It's different enough from the floor texture to make a good contrast.
2. Go into one of the horizontal views (Left or Front viewports).
3. Press and hold the X key to move the marker up 4 grid squares above the floor, where it is now.
4. You can press the O key to zoom the view out further if you can't see enough of the grid to complete this task. Your marker is now 256 units above the floor, about the right height for a tall and imposing support structure.
5. Now, switch back to the Top view.



6. In the center of the map, create a pillar that's eight-sided and shaped like a rectangle with the corners cut off. Make it 256 units thick/tall.
7. Save your level as Big1.LTA.
8. This is an example of an object that can be created quickly using DEdit but would require several steps in most other editors. However, the ability to create brushes this way does allow you to create brushes that DEdit and Jupiter cannot properly use in games.

[Top](#)

Setting Markers

The Marker controls all aspects of where you place things, such as lights, start points, brushes, or pasted objects, in your world. When you first worked with the marker, all your work was done in the top viewport, but it works exactly the same in any of the ports. It even works in the Perspective view, although most often it's not useful to work in that view.

Object placement in ports other than the top view is simple. The object being added always appears at the center point of the marker, and the viewports do not affect this functionality. However, the viewports do affect the placement of brushes.

In your Big1 level, move the marker in the Top view so that it's one grid square south of the center of the column you created earlier, as in the image below.



If you were going to add a support to your column, this is probably where you'd want to add it.

Now, switch to the Front viewport. Zoom out a little so that you have some space to work and add a brush that is a triangular wedge with a thickness of 128 from the top of the column and looks similar to the figure below.



Look in the top view and see what's been done. What you should see is that the brush lines up exactly with the green marker, and its thickness of 128 units extends along the front/back (X axis). The marker's location in each view defines where the brush starts and the thickness you specify determines how far from the marker it extrudes.

By placing the marker carefully, you can lay brushes out exactly where you want them. Although this system of viewports and marker is somewhat different from many of the traditional editing tools, it's also very powerful once you get used to it. Remember always to be aware of where the marker is and where you need it to be.

[Top](#)

Selecting Brushes

To select a brush, left-click on it. In any of the viewports, move your mouse over the brush you want to select. If there are brushes stacked on top of each other, DEdit tries to pick the one it thinks you want. If a big brush is in front of a smaller brush, clicking inside the smaller one will select it even though the bigger one is "closer" to you.

You can select brushes in order of their depth in the current viewport by pressing the T key when you click. In other words, if two identical brushes are stacked on top of each other in the current view, the T+left-click combination would select the one closest to the "front" of the view, and subsequent clicks would select the brushes behind that one.

One way of speeding up your work is to select multiple objects or brushes at once. You can do this in many ways, including multi-select mode, the CTRL key, selecting by name, and the Selection menu.

Multi-Select Mode

In the Mode menu, select the Multiple Node Selection command. While you're in this mode, selecting an object no longer de-selects the objects you have previously selected.

With Multiple Node Selection chosen, when you select a new object it's added to the selection. If you mistakenly add a brush to your selection, hold down the CTRL button and left-click on it again to remove it from the selection.

It can be difficult sometimes to select the brush you want, and you can accidentally select the wrong brush. Pressing the U key de-selects all brushes, after which you can select the single brush you want to work with.

Be sure to use the camera in Perspective view and press the U key frequently so that you don't grab the wrong brushes.

Adding Brushes to a Selection

You can press and hold the CTRL button while clicking on brushes to add them to your selection. While the CTRL key is pressed, you can click any brush again to remove it from the selection.

Block Selection

Another way to select a group of brushes at once is by using a block selection, which can be used in any mode.

1. Move the mouse into the top view, then left-click and drag out a rectangle that covers the brushes you want to select. You will see a white highlight that outlines the area you're selecting.
2. Once you release the mouse button, all the objects under the highlight are selected.

In the case of brushes, a brush only becomes selected when the highlight touches one of its corners, so you can select a brush that's on top of a larger brush without selecting the brush below it.

Select By Name

You can use the Nodes tab in the Project window to select and find objects by name instead of having to find them visually.

Click the Nodes tab to switch to it and examine what looks like a tree diagram of your whole level.



The list you see shows all of the brushes and objects in your level. Each of the names has a box and a symbol next to it.

Move the camera in one of your views out so that you can see the whole level.

Then, click on the box next to the object called Brush1. Two things happen. First, a check mark appears in the box and its name is highlighted. Second, one of the brushes (your pillar) is selected.

This way of selecting things is useful on larger levels, where you may know what brush you'd like to select, but can't easily jump to it to select it. In that case, you can then use the Center View button (described earlier) to quickly center your views on the brush.

You can do many other things with the Nodes tab to organize your level as well.

Selection Menu

The last major method for selecting brushes and other objects is via the Selection menu. The menu provides useful commands for working with several brushes at once, as well as a few commands for organizing your brushes.

The first two commands, **All** and **None**, are straightforward. Clicking None behaves exactly the same as pressing the U key: it clears all selections. If you choose All, then DEdit selects everything in the level: all objects and all brushes.

The **Inverse** command inverts the selection so that everything selected is unselected and vice versa.

The **Container** command tells DEdit to select an object's parent object in the Nodes tree. You can use container objects to organize the objects in your maps without affecting the way the map works. You can read about using containers in the section on using and understanding nodes.

The **Advanced** command displays a dialog that will help you find objects and brushes based on their name or type. This is another feature designed to help you find objects in large levels where you may not be able to locate something right away visually, or to find large groups of objects quickly.

The **Hide** and **Unhide Selected** commands let you hide selected parts of the level from view if you're not currently working on them. This will help your editor run a bit faster, and can help you to see parts of the level that would otherwise be hard to view with your camera. You can also use these commands to accomplish some complex tasks with carving and geometry mode (which you'll learn about later). Since a hidden object isn't affected by most actions while it's hidden, hiding brushes can be used (like masking tape in painting) to protect certain

brushes from work you're doing on other nearby brushes. This is another command that you'll read more about in the Nodes section.

The **Hide Inverse** and **Unhide Inverse** commands are just like the above commands, but they operate on anything that you have not selected. They are commonly used for hiding items other than the ones you're currently working on. For example, in cases where you're working on only a small area of a large level, you can speed things up by hiding the rest of the level. It can also be useful to hide other brushes when you need to do work on the geometry of a brush that's too tightly packed in among others to easily find its vertices.

The **Mirror** command allows you to flip a shape or group of objects in a level. You can use the mirroring commands on any brush, object, or group of brushes and objects. We'll discuss the Mirror command later.

The **Save as Prefab** command takes the currently selected objects and brushes and stores them as a Prefab in their own file so that you can use them in other levels, or in other places in the current levels. Once you've saved a Prefab into its own file, you can then import the prefab again by double-clicking it in the Prefabs tab. Prefabs can be as simple as a vending machine or as complex as an entire room complete with lights and characters.

The **Generate Unique Names** command creates a unique name for objects that have identical names. In cases such as AI paths and scripting, this can be important, since some objects use the names of other objects to select them as a target, to move them, or to send them messages. Unless you're working on such tasks, you shouldn't have to use this command.

The **Group Selection** command (related to the Container command) adds a new container object and groups the selected objects together in it. This command is very helpful for sorting large levels or rooms into smaller parts. See the Nodes section of this document to learn more about containers and their use.

[Top](#)

Manipulating Brushes

In addition to the grab handle at the center of each brush, there are several other ways to rearrange your brushes in the editor.

Use the M key to move selected objects, including brushes. Press and hold the M key, and then click and drag with the mouse to move the objects wherever you drag them.

Rotating Brushes

You can also rotate your brushes, which is very useful when building symmetrical areas. As an example, assume you want to make a second copy of your pillar and its support but facing the opposite way. The slow way would be to lay out each brush again by hand, which would be tedious. The faster way would be to copy the current brush, then just rotate it into the new position.

1. Switch back to your Big1.LTA file, or open it if you have closed it.
2. Switch to multi-select mode (using either the toolbar buttons or Mode menu Multiple Node Selection command).
 3. Select the pillar brush, and then select the support brush.
 4. Press CTRL+C to copy them.
5. Since you don't want to insert the brushes at the green marker when you paste them, from the Edit menu use the Alternate Paste command.

Note: When you perform an Alternate Paste, DEdit pastes the object into the current world at the coordinates where it was copied out instead of at the marker. This is most useful in two cases. First, when you want to rotate or realign the brush slightly in relation to the original. Second, in cases where the new brush will be in a new world, but be in the same exact position there as the old brush was in the old world. For example, you might want to copy or paste parts of your world into a test world, just to try a few ideas without modifying the original world's design. You would not want to align rooms, lights, and so on by adjusting the green marker for each paste operation.

6. Press CTRL+ALT+V to paste in the new brush. The selection will flicker, but since you pasted into exactly the same place that you cut from, nothing will appear to change.
7. Go into the top view and move the marker so that it's right at the tip of the support brush:



8. Right-click in the top view and select Rotate Selection.
9. Since we want to make an archway out of the two pillars, enter 180 in the Degrees box that appears.
10. When you click OK, you'll see your selection move. If you use the camera in the Perspective viewport to look at what's happened, you'll see that there's now a mirror image of the pillar on the opposite side of the original. Notice that the marker defines the center around which the brush is rotated.

Note: You can also hold down the N key to rotate the selected objects, but since this rotates the brush in very small increments you must be careful to rotate neatly. It's difficult to rotate an object by exactly 90 degrees by hand; using the menu command is generally faster and more accurate. Additionally, you can hold down SHIFT+N to rotate in 15-degree increments. The most important thing to remember about rotations is that if your rotation operation leaves brush vertices off the grid, then precision errors can occur during processing.

Mirroring Brushes

Another way to flip objects is to use Mirror commands in the Selection menu. This method works well for brushes that are not symmetrical enough to rotate, such as a brush that's beveled on one edge in order to meet up neatly at a corner. Each command flips the brush along a different axis relative to the world. It doesn't matter which viewport you use the mirroring commands in, since the flip axis is relative to the world, not the viewport. A top/bottom mirror inverts the brush along its Y-axis, a front/back mirror along the X-axis, and so on.

Resizing Brushes

Resizing a brush (which you'll want to do now, since your level's floor is too small) is done using the drag handles on the edges and corners of a brush.

1. Switch back to your Big1.LTA file, or open it if you have closed it.
2. Press the U key to unselect everything, then select the brush that makes the floor of your level. You will see filled boxes appear at each corner and on each edge that look just like the one in the middle for moving the brush. The ones on the edges are for stretching the brush in the direction of that axis. The handles on the corners allow for resizing in both directions at once.
3. Go to the top view and resize your floor brush with the drag handles until it is four times its current size on each of the edges and it is underneath both of your pillars so that players can walk underneath them. You should get something like the picture below.
4. It may also be useful to make the grid size larger (128 or 256) to simplify resizing the brush properly.



Having the two types of drag handles helps when you want to either quickly resize a brush in multiple dimensions like your floor (corner) or resize it in only one direction as you'd do if you extended a hallway (edge). These handles appear around any group of things in the world, including objects. You can use the drag handles to widen a circle of lights or spread out a group of start points as well as using it to increase the size of a brush. You can even mirror a brush by dragging it inside out. DEdit is smart enough to flip the face normals of the brush so that they still end up pointing outward as they should.

However, one thing that can happen when resizing a group of brushes is that you can re-shape some of the brushes in ways that you didn't intend. For example, select both the floor brush and the right hand pillar in your level. Drag the right edge resize handle to the right by one square. What you should see is that not only does the pillar grow, it also moves to the right. This is because DEdit treats the entire group selection as one scalable object/brush.

That sort of movement, if you're not aware of it, can cause leaks when you use the handles to resize a hallway whose floors and ceilings aren't all the same length. For the same reason, you don't want to use these handles to move a pair of walls to narrow a hallway. The walls thicken or shrink as they're moved, and not usually by nice, neat increments. A pair of walls that began as 32 units thick each and 128 units apart might end up 196 units apart as you intend, but be 45 units thick each. That's not a unit that will be easy to fit textures to or align with your other walls and floors. Conversely, if you scale a lamp post that has a lens flare sprite attached at just the right location, resizing the lamp post and the sprite as a group will ensure that the sprite stays at the proper location.

This brings up the last possible problem with resizing objects. If a brush has vertices that fall on points that wouldn't line up with even a 1x1 grid spacing (the smallest DEdit and Jupiter can handle without possibly producing precision errors), the Processor may have some trouble getting the corners of the brush's faces to line up when the level is processed. This can cause gaps to appear between faces and may also cause level leaks.

Note: You should undo your resize by hitting CTRL+Z in order to return your column to its proper size now.

If you do use the drag handles to resize a regularly-shaped object, you should generally verify that all of the moved vertices wind up on the grid. If you can't, correct errors by shifting vertices back onto the grid in Geometry Mode as described in the Brush Power Tools section.

Note that for rotated objects, irregular shapes and terrains, this step is not usually required.

Primitives

One way of creating a new brush is to insert a primitive into the map. Primitives are pre-defined shapes that can be used instead of placing vertices. The main purpose of this is to quickly add brushes that would otherwise be very complex to build. You can add five shapes: Cubes, Cylinders, Pyramids, Spheres, or Domes.

To add a primitive

1. Choose Add Primitive from the Brush menu.
2. Select one of the primitive types.
3. You'll get a dialog that asks for a few options to set up the brush, and then the primitive appears at the marker.
4. One difference between creating a brush this way and making it using Brush mode is that primitives come in un-textured. Be sure to apply the proper texture to your primitive once you've inserted it. To do so, select the new primitive and press CTRL+T.

Note: You may be tempted to fill your level with lots of domes and 20-sided columns, but be sure to read about optimizing performance in your levels before you do this. These objects can create great beauty where they're used, but used in the wrong place they can create big problems with your level's performance.

[Top](#)

Using Brush Power Tools

Even with DEdit's primitives and its vertex-by-vertex brush layout tool, you still can't create all of the brushes you want in a single step. DEdit has four main tools for changing the actual shape of a brush: Hollowing, Carving, Splitting and Geometry Mode.

While they are very useful tools, they are also very effective for destroying brushes, making your levels slower to process and slower to run, and making bad designs worse. As in carpentry, think twice before you cut. Be sure of what you will get before using these tools.

Hollowing

Using hollowing, you can empty the inside of a brush, like hollowing out a pumpkin. For a demonstration of how to use this technique, you can very quickly build a box around your entire level to contain it for processing.

First, raise the marker about 128 units above the tops of your columns. Then, zoom out your Top view until you can see all of your level's brushes and a little space around it. Re-select the texture you used for the floor of the level, since the texture of your pillars and supports is a bit dark.

Draw a square brush that surrounds the whole level in the top view and make it 1024 units tall. That'll make it large enough to contain the whole level with some space left over. Now, with the big new brush (and nothing else) selected, select Hollow from the Brush menu. When prompted, enter 64 for the new brush thickness. This is the thickness of the walls of the brush, which will be created around the brush. Then the original brush is deleted.

You're probably annoyed that you've been taught to create rooms using the slower method of laying out each of your wall brushes and ceilings by hand instead of just creating and hollowing cubes. However, if you look at the created brushes, you'll see that DEdit created brushes whose edges are beveled instead of straight. This is to fit them together neatly with the minimum number of faces visible, although in some cases hollowing can drive up the number of faces on your brushes, leading to higher polygon counts inside your game. Also, because of the beveled edges, hollowed rooms must be resized by moving vertices, instead of by scaling whole brushes.

Hollowing can be useful, especially when the bevels fit in with the design of your room, or are in areas that won't affect the rest of the layout. If you're using it in an area of the map that you plan on changing a great deal, consider building your brushes by hand instead. It's usually neater even if it is slower.

Carving

Your next tool is carving, also known as CSG Subtraction. It's similar to hollowing and the two can even be used interchangeably for some purposes. However, as with hollowing, it can often produce unintentional results.

In order to carve a brush, first you need to create the brush to be carved. Next, you need to create a second brush that will serve as a tool to carve with. As you may guess from the name of the operation, the volume of the second brush will be subtracted from the volume of the first brush.

As an example, you can bevel an edge off of the top of your two pillars to give them a little more of an interesting look. Select one of your triangular supports. Copy (CTRL+C) and Alternate Paste (CTRL+ALT+V) it, then move (M) it so that it overlaps the outside edge of the support.



Resize the copied support so that its edge is a 45-degree slope in the Front view, then resize it in the top view so that it overhangs the north and south edges of the pillar a little. You should get a brush that looks like this:



The purpose of the overhang is to make sure that the brush cleanly carves the surface of the pillar. It's not necessary, but it's a good idea. DEdit may occasionally create a bad edge or face when using carving, and overhanging a little helps to minimize the chances of this. If it does happen to you, use undo to back up and then try resizing your carving brush to overhang the area where the problem appeared if it's possible to do so.

If you can't overlap the carving brush for some reason, see the section on fixing damaged brushes later in this chapter. It includes techniques that might help you fix any damaged brushes that result. You may also be able to use splitting planes (which you'll learn about next) to do what's needed.

Notice that your carving brush only touches the pillar and not any of the other brushes in the level. That's because carving with a brush cuts its shape out of any other brushes it's touching. Make sure that your carving brush isn't touching anything you don't want to carve. You can select both the column and the carving brush, then use the Hide Inverse command on the Selection menu to hide brushes that are in the way.

Then select any brush and use the Unhide Inverse command to bring them all back. Alternately, you can move the carving brush and the brush to be carved to an area of the level that's open enough to avoid the problem and move them back when you're done.

Once you have the carving brush in place (make sure only the carving brush is selected), press H or use the Carve command on the Brush menu. Move the carving brush away from the column and look at the result. What you should see is that a new edge has been carved off of the column, leaving a large beveled edge. As with resizing brushes, it's a good idea to line your carve up so that any new vertices stay on the grid, although this is most often a problem for brushes you plan on doing more to after the carve. If you're building chunks of rubble, you may not care as much about the results being on the grid.

This technique can be used with more complex shapes as well. If you stick a cylinder-shaped brush into a wall to make a doorway or into a ceiling to make a window, you can create some very complex effects quite quickly. However, as with hollowing, your decisions on how to lay out brushes are very often better than the computer's, and your cuts are probably going to be neater and easier to understand. The splitting planes tool, your next power tool, is designed with exactly that in mind.

Note: You might want to create some brushes outside the current map and try playing with carving for a little bit to better familiarize yourself with how it tends to cut before moving on. Try more complex carving shapes and carving multiple brushes at once. Be sure to watch the brushes that you get as a result. To more easily view the splits on your brush faces, turn off the textures. Right-click on the Perspective viewport, choose Shade Mode and then click Flat Polies.

Splitting

The splitting plane tool is like carving, except that instead of using a full brush to remove a section from other brushes, you use a single plane as a saw to cut brushes into pieces. Although this tool only cuts along a single plane, it's a very useful and refined device for making complicated brushes for two reasons. First, it doesn't require the creation of a cutting brush. Second, it allows you to pick the exact position and order of the cuts as they're being applied instead of letting DEdit make the decisions.

To see the tool in action, select the pillar whose edge hasn't been beveled off yet. Splitting planes only splits through the brushes that are currently selected, so they can be used even in crowded parts of your level without the problems inherent in carving.

Next, you need to draw a splitting plane that's located where you want your cut to be. Since we're trying to be symmetrical with the other side, our cut should cut off the outside corner from two grid lines in at the top to two squares down on the side, a 45-degree angle cut. To define your cutting plane, you do as you'd do if you were starting to create a new brush along the line to be carved: Go to the Front viewport and press SPACE to define a line between the two points where you want the cut:



Make sure you only define two points on the line, since splitting only uses the first segment of the line to split on and will ignore any later segments. Use SPACE to place the first spot, then move the mouse to your second spot and press SPACE again to define the second point.

Note: Splitting occurs along an infinite plane defined by the two points you place. You do not need to ensure that you overlap all selected brushes or make the line the full length of the split you're creating. Just make sure that the two points you choose are on the exact line that you want.

Once you have your split line defined, press S to perform the split. Your selection will be cleared, and instead of one brush you now have two: a pillar with a beveled edge and a wedge of pillar that's been cut cleanly off. If you select and delete the wedge, you'll have a pillar that's a mirror of the pillar on the opposite side.

For a more complex case, imagine that you want the floor of your level to float in space. Now, imagine that you want to blow off a corner of the platform to make it appear damaged. What is the best and cleanest way to do it?



First, select the floor brush. Now, go into the top view and center in on the 4 squares in the lower left hand corner of the brush. Once you have a good view of them, split the brush vertically at the edge of the second square in from the left.

Select the smaller of the two resulting brushes (the left-hand one) and split the bottom two squares off so that you have a rectangle above a small square that forms the lower left corner of your floor. What we're doing is isolating the rest of the brush from the area we're going to work on. This is important, since one of the things we want to add is a splintered, irregular edge. Always choose carefully where you make your cuts. Otherwise, in this case, the whole floor here would be splintered and irregular which would drive up polygon counts and look ugly.

Now, select the square that you created. Zoom in on the square and lower your working grid size in the top viewport to 32 units so you can create smaller cuts. Now, using the splitting tool, carve up the brush and delete any excess bits until you have a nice jagged corner like this one.



You may want to use the flat shade view in your Perspective viewport again to do your carving, since it'll be easier to select items that way. Note that you can now create a pretty complex shape without destroying the rest of the floor. Imagine what the floor would have looked like if you had used carving instead of splitting.

Geometry Mode

DEdit's last brush power tool, which you'll use to fine-tune the raggedness of your destroyed corner, is called Geometry Mode. Unlike

Brush Mode, where each operation works on a whole brush at a time, Geometry Mode works on a single face, edge or vertex of a brush at a time.

Press U to unselect your brushes, then press CTRL+G to switch to Geometry mode. Move your mouse around in the Perspective viewport and watch what happens. What you should see is red highlights around the brush faces as you pass over them. You will also see green highlights on nearby edges and vertices. For the most part, you don't make normal selections in Geometry mode the way you do in Brush or Object mode. The red highlight is DEdit's way of indicating where the focus is instead. If you have several brushes selected, Geometry Mode filters out all unselected brushes so you can only manipulate parts of the selected brushes. This helps you to make changes in tight areas.

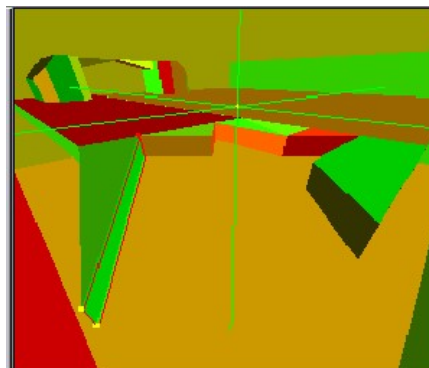
In the Top viewport, move the mouse over the end of the triangle on the upper left edge of the jagged area. Get the mouse over the tip of the triangle that juts out of the corner into space. You'll want to move that point so that the jagged edge sticks out further into space and looks more torn. Press and hold the M key, then click and drag the point of the triangle a grid line to the left.



What you've done is move the two vertices that make up the point of the brush over to the right. Something you may also notice is that you actually moved two vertices, not just one. You moved the vertex on the top face and the vertex on the bottom face, which was directly below it in the top view. DEdit knows that since the vertex on top had another vertex directly connected to it that was below it, you probably wanted to move the whole edge. DEdit only makes this assumption when the vertices are both on the same brush and are directly connected by an edge. In order to move vertices that aren't linked this way in a single step, you have to select them a different way.

The right corner of the triangle you just resized butts up against the corner of another brush. Get that corner in view in the top pane. Then hold down the left mouse button and drag out a rectangle that surrounds that corner. Be careful to only select the one corner, and not any other vertices. When you release the key, you'll see a yellow dot indicating that the vertex is now selected. If you use the other viewports, you'll see that all the vertices at that point are selected, whether they're on the same brush or not.

The nice thing about this is that you can now move all of them as a single unit. Press and hold the S key and drag the yellow dot to the right by one grid space. You should see that both brushes change shape when you do this. It's important to know what vertices you've selected when using this method, since it's easy to grab the wrong ones. If you have trouble, you can always select the brush you want to work on or hide the surrounding brushes to make it easier. Remember, M moves the vertex you click on (and those below it if connected via an edge) while S moves the vertices you have selected.



Now, press U to unselect the vertices you currently have selected. Go back to the triangle that you re-shaped before. Use the same technique to select its tip. However, instead of moving it left/right, you'll be moving it downwards. Go into the Front viewport. Press the S key to drag the two vertices down three grid lines. In the Perspective view, use the camera to look at the result.

What you've done is bend your original brush downwards to make it droop. This is where the real power of Geometry mode comes in: You have the ability to totally re-shape a brush down to its smallest features. You can use Geometry mode to fix brushes when their vertices don't line up with the grid, to resize irregular objects and to repair all kinds of problems.

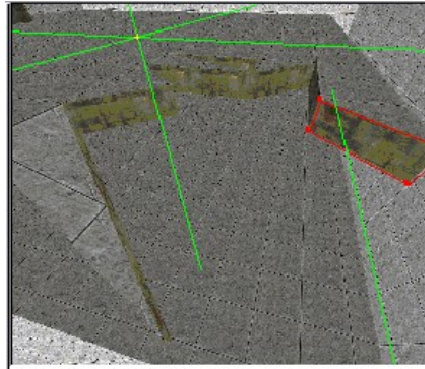
One other way to select objects in Geometry Mode is to use the Y key to highlight all the vertices on a single face. Move the mouse over a face so that it's highlighted, then press Y. What you see is that all the vertices on that face are now selected. This is the method most commonly used in DEdit to skew a brush: you select the face and then use the S key to move it around, skewing the brush.

The last area where Geometry Mode is highly useful is face-by-face texture management. There are cases where applying a single texture to a whole brush won't get the result you want. For example, perhaps you want to darken the edges of your ragged corner.

In the Perspective viewport, highlight the side of one of your pillars. Next, right-click and choose Select Texture from the Context menu. If

you now go to the Textures tab, you'll see that the texture on the pillar is now the active one. Using this tool helps you to quickly grab a texture without needing to know where it's located in your texture tree.

Move the camera so that you can see the edges of your ragged corner.



Hover the mouse over one of the edge faces of your ragged brushes and press CTRL+T. The highlighted edge now has the dark texture on it. By moving the mouse and camera you can quickly re-texture all of the brushes to the new, dark texture.

Since Geometry mode is such a powerful tool, it also allows you to damage brushes and create objects that the LithTech engine can't run. DEdit assumes that users in Geometry mode are experts doing things for a reason, so it allows them to do actions that might cause problems because it thinks that is what they want to do. You can probably easily imagine ways to make concave brushes using Geometry mode, but there are more subtle problems you can cause using this mode as well.

Do not use Geometry mode to mirror brushes. If you select a face or group of vertices and drag them so that the brush becomes inverted, the brush will literally turn inside out, with the normals of each face towards the center, resulting in a bad brush when compiled.



Do not twist the faces of a brush. This is a hard concept to demonstrate. Say you have a face that's rectangular, and you pull one of the vertices straight downwards, as in Fig. A. What results is a twisted face.

Since faces must be totally flat, you have effectively split this one face into two without creating an edge to separate the new faces. To DEdit this will register as a concave brush, and Processor will not be able to properly compile it. Twisted faces are probably the most common kind of Geometry mode problem, and the hardest to spot in many cases. Use the Triangulate Brush (CTRL-E) command to double-check your work occasionally if you're working heavily in Geometry mode. Any folded faces will be triangulated into good faces when you use the command. If DEdit complains about a brush and you can't see anything wrong with it, chances are good that you have twisted a face.

Geometry mode is very effective for doing many things that otherwise simply couldn't be done, but it's also very easy to do serious damage with it. It's the most powerful of the power tools, and as such should be used with plenty of care and attention.

[Top](#)

Understanding Invalid Brush Types

This section provides more detail on the how and why of bad brushes and a few ways to repair them. Not all problem brushes cause terminal problems. However, you should always be sure to avoid creating invalid or problem brush types.

One Sided Brush

The first type is often created accidentally, and here's how to do it:

1. Move to the Top viewport and press CTRL+G to switch to Geometry mode. You're now in the mode DEdit reserves for editing on the level of faces, vertices, and edges, rather than brushes.
2. What happens if you create a brush in this mode? Try it: Use the SPACE bar to create a square brush in the Top viewport. Notice that when you close the brush by placing the last vertex, you are not prompted for a brush thickness the way that you are when you build a brush in Brush mode.
3. Press CTRL+B to switch back to Brush mode and look at the new brush you've created.
4. Use the I and O keys in the Left or Front viewport to bring it into view.

What you'll probably notice is that the brush is completely flat. In fact, not only is it flat, it has only one side. What that means is that from one side it's solid and has a visible surface, and from the other it's completely transparent.

These brushes are used in building your level as Occluders, which are brushes with a very specific and special purpose. They can also be used in WorldModels or as regular world geometry. They can only be seen in the game from the side which has a face. Since Jupiter does not perform hidden surface removal on your brushes, you will often create objects out of one-sided polygons. However, there are times when you intend to create a multi-sided object and get a single-sided polygon instead.

If you create a brush in Object or Geometry mode, or if you ever notice you haven't been prompted for a brush thickness, you have created a single-sided brush. If that was not what you intended, you should make sure you're in Brush mode, then delete and recreate the brush.

Concave Brush

Concave brushes were invalid in previous versions of the LithTech SDK. That has changed in Jupiter. Jupiter's Processor can understand concave brushes, and most world geometry in Jupiter is concave. However, you must still be very careful to avoid making concave or folded faces. Just as in Max or Maya, a concave face will create rendering issues and may cause collision problems as well. If you are going to create concave shapes, always check them carefully and triangulate them if needed to remove folds and concave faces.

Loose Faces

Brushes also sometimes develop a loose face, or you may build an object out of separate faces and want to connect them into a single unit for ease of editing.

If you try to move one face on a brush in Geometry mode and the rest of the brush doesn't follow it, the face has probably been detached from the rest of the brush. This is easily fixed.

1. Undo the move so that the face goes back where it was.
2. Then use click-and-drag selection technique to carefully select the vertices at one of the corners.
3. Press CTRL+J to join the tagged vertices.
4. Keep doing this for each corner until you've reattached each of the corners.

This technique can also be used to correct problems from a carve where one plane of the brush protrudes too far, another rare problem. Just move any misplaced vertices to where they should be attached and then join and clean them up as described above.

Off-Grid Vertices

Another problem that occurs rarely is with brushes whose vertices don't all land on the grid.

Most of the time, vertices off the grid are fine and normal, especially for complex terrain or rotated objects. However, sometimes a vertex that's off of the grid slightly will be miscompiled due to rounding errors in the floating-point math involved in processing the level for the engine. As a result, you may see gaps between brushes where two faces don't line up as they should. Care when stretching, carving or splitting brushes can prevent most of these problems, and you can correct them after the fact using Geometry mode or the Triangulate command. This problem rarely happens in Jupiter, since concave shapes are now supported.

Overlapping Brushes

Another type of error occurs when two faces of different brushes occupy the same space. In such cases (a handrail sticking through a wall, two walls whose corners overlap), if the overlapping planes are both visible to players, the engine may not know which to remove. In these cases, it will leave them both. This results in a weird flickering display of both surfaces at once, which is pretty unpleasant. You'll recognize the effect when you see it.



The way to avoid this is to make sure that where surfaces meet, they don't overlap on surfaces where the player can see. Don't overlap two walls at the corner; just neatly join them.

Don't run a handrail all the way through the wall. Just end it where it touches the wall. The second diagram presents the proper way to connect two brushes at a corner. Luckily, you only need to worry about z-fighting in cases where both surfaces are visible to the player.

However, surfaces that the player can never see should always be removed to save on file size and memory. Processor will no longer remove them automatically.

T—Junctions

A t-junction is anywhere that one polygon has a vertex touching the edge of another polygon without a vertex at the same location, like the one circled in the figure below.



As you can see in the figure below, the red and yellow polygons each have a vertex in the middle of the adjoining edge of the green one, which does not have a vertex there.



To remove the t-junction, you'd add a vertex there, as indicated in the figure below by the purple arrow.



Without fixing the t-junction, what happens is that any time there's a rounding difference when the polygons are rendered, the red and yellow polygons stick together because of their vertices being at the same spot. As a result, there's never a gap drawn between the red and yellow polygons. The green guy, however, can't bend there due to the lack of a vert, and you get a gap as shown in the following figure.



If you add a vertex at that spot (as is done in the figure below) or split the green polygon at that point, then all three polygons can flex around and stay together.



The way that you solve this is by avoiding long brushes with smaller brushes stacked on top of them. In the case of terrain, make sure you insert extra vertices and re-triangulate or otherwise make sure that the face stays planar. Fix T-junctions and the console command are fallbacks for cases where you either couldn't or didn't know how to build without Ts when you first made the level; they can't always stop the problem, unfortunately. The best method is prevention.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



—TUTORIAL— Adding Sky to Your Level

This tutorial describes how to add sky to your Jupiter level, either using the prefabs or creating a sky using TO2 resources (textures, prefabs). It includes a sample level which you can study in order to see a complicated working skybox. You can locate the Tut_Skybox.LTC sample level in the following directory:

`\Development\TO2\Game\Worlds\RetailSinglePlayer\`

This section contains the following topics related to adding sky to a level:

- [Adding SkyPortals](#)
- [Adding SkyPortal Brushes](#)
 - [Adding Clouds](#)
 - [Moving Clouds](#)
- [Index Ordering Multiple Sky Objects](#)
 - [Adding Fog](#)

Adding SkyPortals

Before you can add a sky to your level, you must first add some SkyPortals. SkyPortals are brushes that tell the renderer where the player will see the sky. These are simple to set up.

1. Add brushes in your level where the sky is to be rendered.
2. Assign the Sky texture to the brushes to make them easy to recognize in DEdit.
3. Set the Type property of the brushes to SkyPortal.
4. Make sure to turn off lighting features such as lightmapping on the brushes, since they will not be lit.

In the case of a level that is largely or completely open and outdoors, you can just build your skyportals as a box that surrounds your entire level. Make sure that the box's faces point inward so that the sky is visible inside the level. If you are making a level that is mostly indoors, where the sky is only visible in a few places, then you may want to make your SkyPortal brushes out of several smaller brushes fitted into the windows or other areas where your players will want to see the sky.



Adding Skybox Brushes

The part of your level's sky that your players will actually see is called the Skybox. There are three ways you can create a skybox: using a prefab sky from TO2, creating a new sky using TO2 textures, or creating and applying your own textures. The following topics describe how to create a Skybox using each of these methods.

Using a Prefab Sky from TO2

With the TO2 prefabs (located in the `\Development\TO2\Game\Prefabs\` folder) you can use a sky from TO2 by simply double-clicking on the description of the Prefab type in the Prefabs tab.

Under the Japan directory in the Prefab tab's list you will find a selection of prefabs for the Japanese sections of TO2. This includes two sky prefabs.

These simple skies are composed of either a hollow sphere or a cube whose inside is textured to look like a sky-filled room. You can also do this with a cube by using six textures, one for each face. Such sphere- or cube-shaped skies are the most common types of sky in use. When using a simple sky it is best to check your WorldProperties object's FogInfo properties and make sure SkyFogEnable is set to FALSE. This property turns on and off the z-fog in the skybox.

Complex sky types often require SkyFogEnable set to TRUE in order for the sky to look right. Most skies work well with a SkyFogNearZ value between 0 and -256 and a SkyFogFarZ value between 256 and 800, depending on the sky used. Experiment with the SkyFog color values to find an appropriate color to the sky. Note that the SkyFog variables are all console variables, which means that you can change them in the game until they look correct, then enter those same values in DEdit. Refer to the sample maps for reference.

Once you place your sky prefab, make sure to move the prefab to the outside of your level. A good place for the sky prefab is underneath the level, so that the actual geometry won't show through the skyports like a box hanging in the sky. (See the screen capture on the following page.)



Creating a New Sky Using TO2 Textures

You can create the same skies manually without using the prefabs. Here's how to create the simple skybox (See the screen capture on the following page.):

1. Build a hollow box or sphere out of the view of the level. A good place is underneath the level, so that the actual geometry won't show through the skyportals as mentioned before. Make sure it is large enough to give you good working room (512x512x512 is a decent size).
2. Set the AmbientLight value in the Light property to white.
3. Set the lighting features such as lightmapping on the brushes to FALSE, since these brushes will be lit by their Ambient value.



4. With the box geometry ready, place the crosshairs in the center of the box and bind all the skybox brushes to a DemoSkyWorldModel.
5. In the Properties tab, set each of the DemoSkyWorldModel's InnerPercent fields to "0.001" to create a static sky. InnerPercent is used to make your sky move around in proportion to the player movement (parallaxing), which isn't useful in a simple skybox.
 6. Set the SkyDims of the object so that the inner walls of the box are touched by it in each dimension.
Example: for a 512x512x512 cube, set the dimensions to "256 256 256."
7. Texture the insides of the sky with the appropriate set of sky textures. In the Textures tab, browse for the Sky directory under TexFX. Usually a set of skybox textures for a cube will consist of six different textures, one for each side of the box's interior. The name's extension depicts the position of the texture like so:

blue_bk goes on the back face of the cube.
 blue_dn is the bottom face of the cube.
 blue_fr is the front.
 blue_lf is the left.
 blue_rt is to the right.
 blue_up is the top face of the cube.

You can create the textures for a cube skybox in Max or Maya as described below and import them into DEdit, or you can use the simple texture JpSky001, a cloud texture, for your test sky.

In the case of a sphere, you usually only need one texture, stretched across the sphere. You can use the texture JpSky001 for the inside of your sphere.

If your level only needs a simple skybox, then you are finished at this point.

Creating Your Own Skybox Textures

If you plan to use a single tiling texture for your skybox, you can create it as you'd create any other texture. If you want to create cubic skybox textures, typically you will need a 3D package such as Bryce3D, 3DStudio Max or any other 3D package that will allow you to render a 3D scene into the six images needed to accomplish the effect.

As an alternative to 3D, you can use a paint program such as Painter or Photoshop to paint a sky utilizing a shareware plug-in named Skypaint. Skypaint is a handy tool that aids in setting the correct field of view of the painting and assures that the cubic form of the geometry will not be visible in the perspective of the textures you create.

- For more information on SkyPaint, visit wasabisoft.com

Creating a New Complex Sky Using TO2 Textures

A complex sky goes beyond a static skybox. Your sky can contain more than the skybox object: panning clouds, a sunflare, or sometimes even a keyframed object. Refer to the included sample level (**Tut_Skybox**) for an example of a complex sky.

In order to create a good complex sky, it's best to start with the simple skybox first. Keeping a sealed skybox ensures that players will never see the void of 3D space outside the level, which causes a mirror/shearing effect. Also, the DemoSkyWorldModel object's center is the sky camera's reference point. Be aware that any objects added to the sky scene will use the DemoSkyWorldModel's center as a reference, so it's important to create this object at the center of your sky.

Enlarging

If you created your own textures for a cube skybox, you will want to re-texture the cube now. Since you'll be adding objects for visual

effects, you can replace the six larger textures with a smaller tiling texture, which is more memory efficient. For this example use the simple texture JpSky001, a tiling cloud texture.

1. Open the Texture tab and select JpSky001 from the `TexFX\Sky` directory.
2. If you need to, resize the skybox to allow more working room now. For a cube, 4096x512x4096 will do well. For a sphere, try 2048x2048x2048.
3. When you resize the geometry make sure the DemoSkyWorldModel object stays in the center, and reset its SkyDims to fit the new sky's size.

[Top](#)

Adding Clouds

Now the stage is ready for other objects to be added to the scene. The first object to make is the clouds object.

1. Add a 4096x64x4096 one-sided brush facing downward about 64 units above the center of the DemoSkyWorldModel object.
2. In Geometry Mode, texture the brush face with a clouds texture with an appropriate alpha mask. For this example, use the texture Clouds2 in the `TexFX\Sky` directory.
3. Right-click on the textured face and select Map Texture Coordinates and set the U and V fields to 1024. Increasing the scale of the texture will prevent players from noticing that the texture repeats.
4. Bind this brush to a Worldmodel object and set the following brush properties under LightControl:
 - AmbientLight = White
 - ReceiveLight = False
5. Make sure the Worldmodel's FogDisable property is set to FALSE, since you want the SkyFog to fade the cloud layer to the horizon.
6. Set the BlendMode property to Translucent. This will make the brushes in the WorldModel translucent according to their texture's alpha channel.
7. If you wish to increase the translucency of the layer, you can set the Alpha property to any number less than 1, such as 0.99. The smaller the value, the more translucent the clouds will be.
8. In the WorldModel object's Name field, enter Clouds1.

Note: When working with more than just a DemoSkyWorldModel in your sky scene, it is best to name each object with a descriptive term, such as changing the DemoSkyWorldModel's name to skybox and the WorldModel to Clouds1. Careful naming makes the next steps a lot easier to deal with.

Moving Clouds

To make the clouds to appear to move

1. Select your cloud brush and open the brush Properties tab.
2. In the TextureEffect field, click the Browse button.
3. In the Select Effect Group dialog, click New.
4. In the Edit Effect Group dialog, enter Pan as the name of your effect.
5. Since we want to pan the cloud texture, In the Stage 1 group, set Channel to Base Texture.
6. Click the ... button on the Script field to find a texture script. Browse to the `\TextureScripts` folder in your `\TO2\Game` directory. Find the file WSPosXZPan.tfs and select it, then click OK. You should now see some additional properties listed. These are the parameters for the texture script that's going to move your clouds.
7. Set the XSpeed and YSpeed to 0.01.
8. Set the XScale and ZScale to 0.001.
9. Click OK, then click OK again to save the settings and apply the texture effect to your cloud brush.

Now that the skybox and cloud objects are created, you need to set them up to render in the sky.

[Top](#)

Index Ordering Multiple Sky Objects

Setting the rendering order of all the sky objects is the most important step in creating a complex sky. This is done using the Index field. Because the DemoSkyWorldmodel is the only WorldModel type with the built-in ability to render in the sky, you must add a SkyPointer object to your sky scene for every other WorldModel object you add after your skybox. If you don't do this, these objects will not show up in the sky. The SkyPointer object points to the WorldModel and provides the Index value for that WorldModel.

1. Add a SkyPointer for the clouds layer. It is recommended that you place all objects without geometry at the middle of the skybox.
2. Fill in the SkyPointer's name field with a descriptive term such as Clouds1 Skypointer.
3. Enter the name of the Worldmodel you made for your cloud layer (Clouds1) in the SkyObjectName field. This tells the SkyPointer which object to render.
4. Continue adding SkyPointers for each worldmodel object in your scene as described above, with the exception of the DemoSkyWorldmodel bound to your skybox.

Now that there is an Index field provided for each object via the new SkyPointers, you will set the drawing order of all the objects for the

sky. Each DemoSkyWorldModel or SkyPointer has an Index property. The objects in the sky are drawn in order of Index, so objects with low values are drawn first, followed by objects with higher values, which are drawn on top of (effectively "nearer to the camera" than) the earlier objects.

1. Open the properties of the DemoSkyWorldModel (your skybox) and check that the Index field is set to 0. This tells Jupiter to render the skybox first.
2. Select the Clouds1SkyPointer and set its Index to 1. Now the game engine knows to draw the Skybox object first, then the clouds layer next, drawing the clouds in front of the sky.
3. Continue adding more objects in your sky as needed, while adding SkyPointers and updating the Index for all the objects. Once you've done so, your skybox is complete.

Typical Index settings for a scene are:

- Skybox = 0
- Sun = 1
- Clouds1 = 2
- Clouds 2 = 3
- Trees = 5

[Top](#)

Adding Fog

Once the sky is fully functional, you can add fog to hide the edges of the clouds.

To Add Fog

1. Go into the WorldProperties object and make sure that FogEnable is set to TRUE.
2. In the FogInfo, set the SkyFogEnable to TRUE.
3. Select white for the FogColor.
4. Set the SkyFogNearZ to 0.
5. Set the SkyFogFarZ to 800.

As an alternative to using fog to fade in the clouds, you can add other objects around the horizon in front of the clouds (such as trees or buildings), just as you added the clouds. If you don't like the appearance of your fog, you can change the fog in the game using the SkyFogNearZ/SkyFogFarZ console commands at runtime. Once you find values for both that you prefer, write them down and change your WorldProperties when you return to DEdit to make the change permanent.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Processing Levels

Overview

What is processing?

Processing is the term used for taking a level that you have created in DEdit and transforming it into a playable map. Processing your level is a relatively simple and painless task, but doing it incorrectly can have highly adverse effects on both the look and performance of the game.

When should I process?

Processing takes place after you have created a level in DEdit. You must have at least one brush placed in your level before the processor will work. Similarly, you must have both a WorldProperties object and a GameStartPoint object in your level before you can run it in the game. Each time you make a change to your level in DEdit, you must reprocess your level for the change to take effect.

How do I start the processor?

To start the processor, load your world in DEdit, and then select Process World from the World menu.

How can I run my level after it's processed?

When the processor is finished, you can run your level directly from DEdit by selecting “Run” from the world menu. You must have DEdit properly configured to run files before this will work. See the DEdit tutorial document for more information on this topic.

What are .TBW Files?

When you save a world in DEdit, it is saved with the .TBW extension. These files are only readable by DEdit and cannot be run directly by the game. When you distribute your completed maps, you may choose to include this file so that others can further modify your work.

What are .DAT Files?

Once your level has been processed, it will be saved with the same name as your .TBW file, but with the .DAT file extension. So if you built a level called MyLevel.TBW and compiled it, you would get MyLevel.DAT as a result. This is the file that you will need to include in your .rez files for distribution if you want other to be able to play your level.

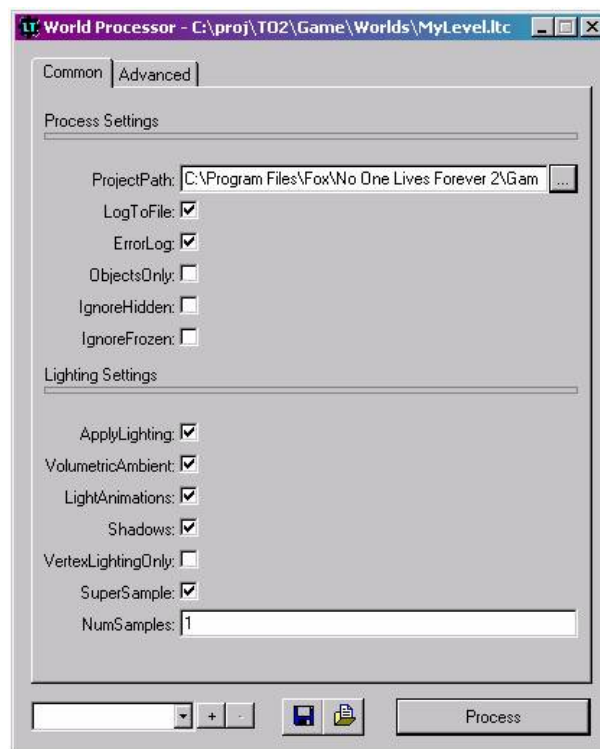
Processor Options

The default settings for the processor (with the possible exception of the project path) are the settings that were used for the retail release of Nolf 2. These are the settings that you will want to use when you process your level for the final time prior to release. However, a full process on a large level using a slower machine can take up to an hour or more. During the course of development when you are likely to check your changes and additions often by actually playing the level, this can be a problem.

The following sections will describe the various settings available in the processor application, and show you which options can be used to minimize the time it will take to covert your levels into playable maps. Options that are not specifically covered should be kept at their default values to avoid unexpected results. Note that any changes you make to any of these options will remain after you have closed the processor; so be careful to check your settings often.

Common Options

The processor application is running, the following dialog will appear on your screen:



Process Settings

ProjectPath: This is the path to the directory where your DEdit project file (.dep) is located. This must be set correctly or the processor will not be able to locate the resources needed to compile your level.

LogToFile: This option will write out all of the text from the processor window to a log file, including error messages. The completed log files will be written out to the directory where processor.exe is located. The filename format for log files is:

(level name) (log file #).log

ErrorLog: This option will write out any error messages that occur during the compilation processes to a separate log file for easy reference. The filename format for error log files is:

(level name)Error_(log file #).log

ObjectsOnly: Once you have completed a full compile of your level and a .dat file already exists, you can use this option to do a quick, objects only compile of your level. This option will only process the objects in your level, leaving the existing geometry and lighting information contained within the .dat file intact. This option is intended to help you tweak your objects and then check them in the game without going through the hassle of a full install. Please note that you should always do a full compile before releasing any level to the public.

IgnoreHidden: In large, complex levels, you may find that it's easier to hide any nodes that you are not currently working on to increase performance and avoid mistakes. This option will determine whether or not the hidden pieces of your level should be processed.

IgnoreFrozen: Similar to IgnoreHidden, this option will determine if nodes you have set to frozen (i.e. not modifiable) will be processed.

Lighting Settings

The following options are provided so that you can turn off lighting features to streamline compile time during development. However, you should always make sure to do a full compile with all lighting options set to their default settings before you distribute your level!

Apply Lighting: Unchecking this box will automatically disable all of the lighting options. If you have not made any lighting changes to your level since the last time it was processed, you can use this to significantly improve processing times.

Volumetric Ambient: This determines if volumetric lighting should be calculated from the ambient setting on brushes. This is on by default.

LightAnimations: This determines if lightmap animations should be created by the code and is on by default.

Shadows: This determines if shadows will be generated from stationary objects in the level. This is also on by default.

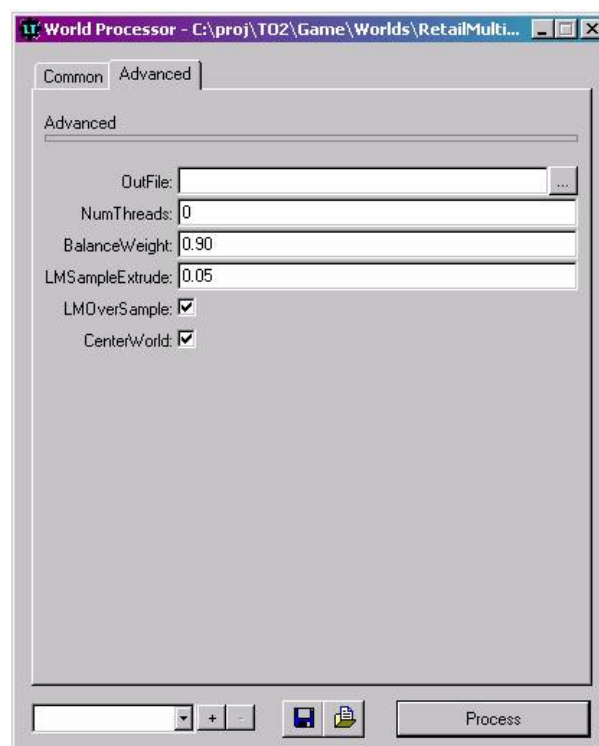
Vertex Lighting Only: For quick compiles, this option can be used to approximate lighting based on vertices. The results will not look as good as they would with a complete lighting pass. This is off by default.

SuperSample: Specifies that lighting should be done by supersample, causing multiple samples to be taken per lightmap level. This is on by default.

Numsamples: Specifies the number of samples that should be taken per lightmap texel. This defaults to 1.

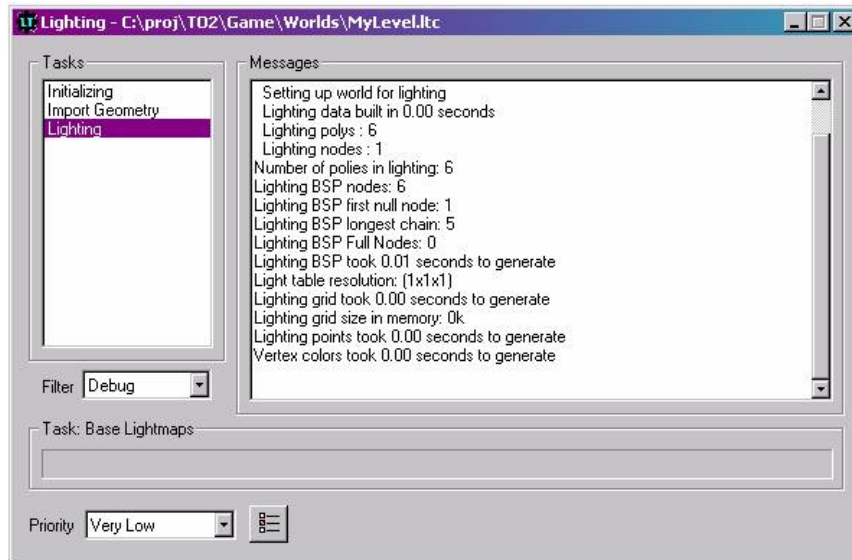
Advanced Settings

The options continued under the advanced settings tab were used in Nolf2 development and should not be changed for NOLF2 maps. If you happen to change anything in this section accidentally, please refer to the image below for the default settings and change them back before processing your level to avoid potential problems with the game. Feel free to play with these values in your own project.



Finishing up

Once you actually run the Processor, the dialog will change to a log window and a display of progress bars showing compile progress. If you read the log window you can learn a lot about how the level is being handled. Error messages will also appear here.



While your level is being processed, you may need to use your computer for other tasks. Use the priority option to determine how much of your system's resources will be used to process your level. On the low and very low settings, the processor shouldn't have too much of an impact on your other applications, but the tradeoff is that your level won't be finished as quickly.

Once the processor has finished, a bell will sound and a "close" button will appear at the bottom of the dialogue window. Click on this to close the processor. Your level is now ready to run.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Using ModelEdit



ModelEdit is the tool used to integrate models into the game. Typically, game designers use 3D applications such as 3D Studio Max or Maya to create and animate models, but ModelEdit is where you finish preparing them for use in Jupiter. This section describes the most frequently used and important features of ModelEdit.

Read the following sections to understand how ModelEdit works, and how you can use ModelEdit to prepare your models for use with the Jupiter engine.

ModelEdit Topics

[Understanding ModelEdit](#)

[Controlling Level of Detail](#)

[Adding Animations](#)

[Editing Animations](#)

Description

Describes the ModelEdit interface

Describes how to adjust the level of detail (LOD)

Describes how to add animations to your model

Describes how to add and edit animation frames

Animation Blending and Weighting	Provides instructions for blending and weighting animations
Adding Sockets	Describes how to add socket locations to models
Jupiter Model Issues	Discusses the issues involved with creating a model in Jupiter
Creating a Character	Discusses the issues involved with creating a character in Jupiter
Setting up a Player View Weapon	Describes how to set up a player weapon
ModelEdit UI Reference	Describes the commands in the ModelEdit menus

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Understanding ModelEdit

The ModelEdit UI is made up of several windows, which are covered briefly in this section, and then described with greater detail in successive topics. The image below shows the ModelEdit UI. Move your mouse over the image to identify the individual ModelEdit interface components.



- The **Model View** window previews the model mesh and its animations.
 - The **Player** buttons control playback to preview the animation.
 - The **Nodes** list selects the model's skeletal nodes.
 - The **Pieces** tree lists all of the individual meshes that make up the current model.
 - The **Animations** panel lists all animations that are loaded in the current model.
 - The **Child Models** list contains all model files whose animations are referenced by this model. An animation that comes from a child model cannot be edited and will have an empty box next to its name in the Animations list. An animation that is part of the file you are currently editing will have a check mark beside its name and can be modified.
 - The **Sockets list** contains the model's sockets.

Each of the ModelEdit controls and windows are discussed further in successive sections. The following lists some general uses of the controls:

- Use the **Level of Detail** (LOD) slider to select the level of detail you want to view.
 - Use the **Animation Edit** control to manipulate animation data.
 - Use the **Transform Edit** control to manipulate socket or node positions.
 - At the bottom of the display is the animation **Time** bar. The **Time** bar has a red mark for every key frame in the currently selected animation. Each animation or model has at least one key frame, but only those with more than one will show red marks. Frame Strings are a way to tie an event or message to a key frame so that it will be available to the game programmer. For example, if your

character has a swiping claw attack, you can add a message to its swipe animation so that each time it swipes at something, a "whipping claw" sound plays.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Controlling Level of Detail

One of the best ways to optimize the rendering of models is to draw fewer of the model's polygons where appropriate. ModelEdit has a tool to create multiple levels of detail dependent upon a distance value. At lower levels of detail, ModelEdit creates a version of the model that uses fewer polygons to draw the same object. The game engine will take the distance from the rendering camera (either from the player's point of view or a cut scene camera) to the object seen and use the triangulation set for that distance.

To create an LOD for a model piece

1. In the Piece menu, choose the Build LODs option. The following Level of Detail Generation dialog will appear.



2. Enter a value in the Distance box. The distance is the threshold for the LOD being created. In the game when the view camera is that distance in game units from the object, your new detail level (the lower-polygon mesh) is used to render the model.
3. In the % Triangles box, enter the percentage of model triangles from the current mesh you want to keep in the new LOD mesh at this distance.
4. Click OK.



LOD 0

Reduced Triangle LOD

The images above are the result of an LOD operation. The image on the left (LOD 0) is the base model; the image on the right is the reduced triangle model. In the game, the renderer will automatically switch to the next LOD level when the distance associated with the LOD level has been reached.

Model LOD Differentiation

The LOD operation affects the whole model, but the user can change how LOD affects different pieces of the model. Artists can make some important pieces stay detailed while other parts of the model become lower resolution.

To differentiate the pieces for the LOD operation

1. In the Level of Detail Generation dialog, pick a piece from the Piece Weights list.
2. Enter a value greater than one into the Piece Weights value box. The value represents the LOD priority for that piece. The larger the number, the later the piece will have LOD operations done on it. For example, if the value of one piece is three, it will take three

vertex collapses before that piece's triangles start to collapse. This is useful for objects like heads or shields, where a small loss in detail can be very noticeable.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Adding Animations

Animations are part of the model's database. Animations can be added directly to a model's own data, or they can be added by reference.

Note: All animations that you import or add as child models must have the same hierarchy as the model being worked on. ModelEdit will warn you when there is a hierarchy mismatch.

Adding an Animation Directly

To add an animation directly

1. In the File menu and select Import. A file dialog appears with check boxes at the bottom listing the import options.
2. Make sure the Animations box is checked.
3. Browse to the animation file you wish to add to the model and select it.
4. Click Open.

The new animation's name appears in the Animations list box when loading is done.

Adding animations directly to a model is useful if you know that no other models will need the animation. For example, an octopus's animations are probably only needed by the octopus model.

Adding an Animation by Reference

A referenced animation's data is not included in the model file. Instead, the model file refers to another file that contains the actual animation data. This is useful when several models are going to share the same set of animations, because once the referenced model, the child model, is loaded into memory, all models that share animations from that model refer to the copy that's already loaded. This saves significant memory in your game when it's running.

To add an animation by reference

1. In the Child Model menu select Add Child Model. A browsing dialog appears.
2. Browse for and select the model or models you would like to load as the animation.
3. Click OK.
4. ModelEdit checks the .LTA file to see if the two models' hierarchies are the same. If they are, the animations will now be available to your current model.

When the model is saved, a reference to the child file is stored in the model data.

About Child Models

The term "Child Model" is used to describe animations that are shared across multiple models. A child model is a reference to another data file that contains animations to be used. The only constraint to this construct is that the child model's skeleton has to be organized exactly like the skeleton of the currently loaded model. ModelEdit will signify that an error has occurred if the child model skeleton is not the same as the base model.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Editing Animations

Below the Model View panel, there are CD player-style buttons that control animation playback.

Below that is the animation bar. Red vertical lines are key frames.

The four buttons in the Animation Edit area manipulate the ordering of the animations in the Animations list box. Clicking the button with a red X on it deletes the current animation from the model. The # button rennumbers the currently selected animation. The arrows move the animation up and down in the list box.

The radio buttons labelled User Dimensions and Translation control the six thumbwheel buttons below them. Depending which is selected, you can adjust either the size of the bounding box of the animation or the base position of the animation in space.

Note: To see the User Dimensions, in the Options menu select Show User Dimensions. In Jupiter, you cannot change the User Dimensions for child model animations without editing them directly in the child model.



In the Frame String text box, you can associate a text string with each key frame in an animation. This association is used as a message to indicate to the game code that a certain event has happened. This is most often used to associate a sound with an event. For example, you could associate footsteps with a "foot down" frame, gunshots with a model's "fire" frame, or a body hitting the floor with the last frames of a death animation.

To add a Frame String

1. Select a key frame by clicking on it
2. Type a word in the Frame String text field.

When the animation runs, the frame string is sent to the game code as the tagged key frame passes.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Animation Blending and Weighting

This section discusses some specific details about animation, and provides instructions on blending, weighting, and interpolating your animations.

Animation Blending

The Jupiter animation system supports two kinds of animation blending operations, consecutive and concurrent. The engine automatically blends between all consecutive animations. Additionally, you can create a custom blend between two concurrent animations using Animation Blending Weights.

Consecutive Animation Blending—Jupiter blends consecutive animations together by default. At the end of each animation the engine interpolates to the next animation in 200 milliseconds. This value can be changed per animation in ModelEdit.

Concurrent Weighted Animation Blending—Use ModelEdit to blend from two to four animations concurrently by specifying Weighted Animation Blending values for nodes in the animations. When the game programmer sets up concurrent animations for the models in the game code (using `ILTModel::AddTracker`), the Weighted Animation Blending can be swapped dynamically to produce new custom animations from existing animations. Be sure there is an Anim Blending Weight set associated with the second animation added in this fashion.

Animation Weight Sets

All nodes belong to the weight sets, and for each weight set the nodes can be weighted differently. For example, an upperbody weight set might have all the nodes belonging to the hips and above set to have a weight of one, while the nodes in a lowerbody weight set would have all the node's weights below the hips set to one and all the nodes above the hip node set to zero.

To Create a Weight

1. In the Model menu, select the Edit Weights option.
2. In the Weight Set dialog box, click Add Set.
3. In the New Weight Set dialog box, type a name for the new weight set you are creating and then click OK.
4. In the Weight Set dialog box, select the new weight set you created.
5. In the Nodes list, select one or more nodes and then enter a new value in the Current Node Weight text box.
6. When you finish assigning weights to the nodes for the weight set, click Close.

To Create a Weighted Animation Blending

1. In the Animations list, click to select the first animation. Notice that the selected animation appears in the Animation Keyframes area.
2. Press and hold CTRL while you select a second animation. The second animation appears in the Animation Keyframes area, below the first animation.
3. In the Animation Keyframes area, click the button that displays the name of the first animation.
4. In the Select Animation Blending Weight Set dialog box, click the name of the weight set you created and then click OK.
5. Repeat steps 3 and 4 for each animation.
6. Click the Play button to see the results of your Weighted Animation Blending.

[Top](#)

Weight Values

The value of weight can be any numerical value. Typically, you will use values between 0 and 1 since 0 means no change and 1 gives full value to the second node.

If the weight set value is 2, the algorithm will simply add the two animations together. This can also be used to great effect. Generally the added animation should be designed intentionally to modify the first. A walk and a run added together would result in a very strange animation.

[Top](#)

Linear Interpolation

The order in which the animations are selected makes a difference. The animations linearly interpolate (LERP) per frame from the first to the next using the node's weight value. The formula of the resulting animation equals the first animation plus the difference between the first and the second animation times the second animation's weight value.

$$RES = FIRST + ((SECOND - FIRST) * SECOND's WEIGHT)$$

Be aware that when you blend more than two animations, this algorithm is recursive: the result from the previous evaluation becomes the FIRST value, and the SECOND parameter is the next animation in the list that has yet to be evaluated. For example,

$$LERP (LERP (first, second, second-weight), third, third-weight)$$

Because of the nature of the algorithm, getting things to look the way you want could be tricky, so as you experiment with various animations, try to make concurrent animations as similar to each other as possible. For example if you want to accentuate a walk, make sure the foot plants occur on the same key frame. If you are doing upper-lower body animations, have the weights at the fusion points ramp from one animation weight set to the other. With enough experimentation, you will develop good intuition as to what works.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Adding Sockets

Sockets are places in models where you can add attachments. Attachments are sub-models that do not deform with the main model. Also, attachments can be changed dynamically in the game. Usually attachments are guns, hats, and sometimes even armor pieces. The socket or attachment sites are normal transformations that are manipulated in the Transform Edit control or from the Sockets list.

To add a socket

1. In the Nodes list select a node where you want to create a socket.
2. In the Sockets menu select Add Socket. Enter a name for the new socket.
3. Click OK.
4. A socket appears in the Sockets list. Double-click its name to open the Socket Properties dialog.
5. In the Attachment box, either browse for or type the name of the .LTA file to add as an attachment.
6. You can also change the socket's name in the Name box, or explicitly change its Position and Orientation in the respective X, Y, and Z boxes.
7. Click Apply to see your attachment in the model view window. You can make changes in the Socket Properties dialog and click Apply to refine your attachment.
8. Click OK to close the Socket Properties dialog.

The attachment .LTA file will be kept in the model's data as a reference that can be changed during the game. This allows you to switch models on a socket while the game is going. For example, if a player picks up new power-ups, armor, or even clothing, you can add to it and modify it during the game.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Jupiter Model Issues

Creating models for Jupiter is conceptually straightforward:

1. Use a modeling package to create a polygonal mesh model.
2. "Skin" the mesh model over a bone hierarchy. Skinning involves weighting each vertex of the mesh to one or more bones.
3. Select a texture for the model.
4. Apply UV texture coordinates to it.
5. Animate the model's bones.

The resulting model is ready to export.

The following issues relate to creating models for the Jupiter system:

Geometric Center

The geometric center of most models should be at the origin. For human character models, for example, this would be the pelvis.

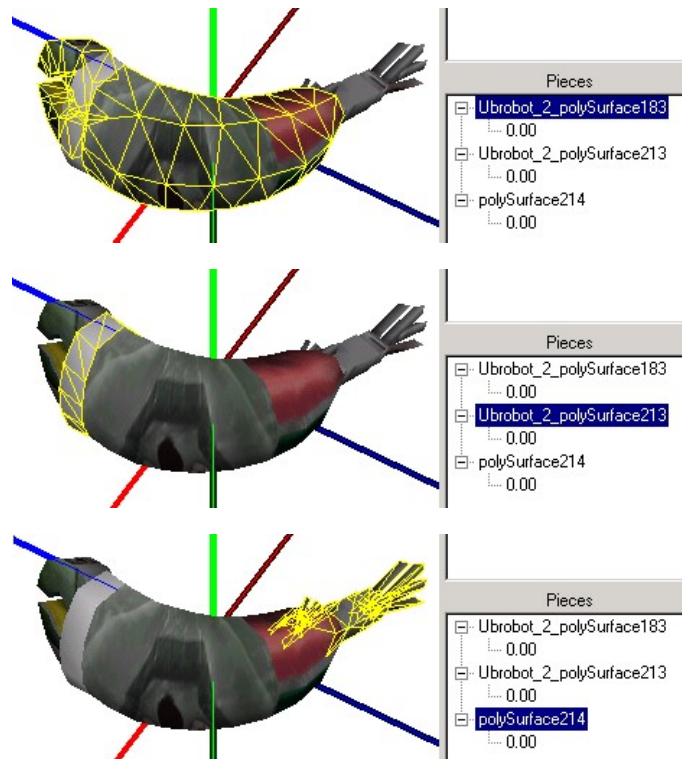
Textures

Except for UVW Mapping, textures and material values are not explicitly assigned in the modeler. Instead, this information is specified in ModelEdit and DEdit. Each piece of a model can have up to four textures applied to it using RenderStyles. These four textures are applied one on top of the other, blended as the RenderStyle specifies. Each layer can have its own index value, which is used to specify which texture loaded for the model belongs on which piece/layer.

All textures in Jupiter are 32-bit by default and their height and width must each be a power of 2.

Meshes

The model can be made up of one or more separate meshes. Generally, splitting up models makes UV mapping easier and can create reusable parts such as hands, feet and so on. The UV mapping of a model has to be done in the modeling package. The following images illustrate a model composed of three separate meshes.



Texture indexes correspond to the indexes on your model's different meshes. This way, the game programmer or the level designer controls explicit texture assignment for each mesh in a model. There can be only one UV mapping per piece, with up to four textures in layers.

Surface Material Properties

Except for UVW Mapping, all surface material properties defined in the modeling package are ignored in the exporter. Vertex colors, lighting and so on are lost in the conversion.

Image Conversion

Use DEdit to convert images into the DTX format required for the game. DEdit can understand and import .PCX or .TGA files, though .TGA is definitely the preferred format.

The advantage to using a .TGA file is that since Targa files have a built-in alpha channel, DEdit can automatically generate an alpha mask for the texture without the need to import a second file as your alpha mask.

.PCX files require a second file for their alpha mask, adding a second step and a second file to keep track of.

Using Animation Sharing

If you want models to share animations via the child model system, then all models sharing a set of animations should have the exact same hierarchy. This is very important. Differently organized hierarchies cannot share animations. The skeleton offsets should be similar, but do not have to be exact.

There are problems using animation from a short character on a tall character, but most things like walks, runs, and general hand movement work fine. The biggest problem is two characters with different shoulder/arm proportions sharing an animation that requires using two hands together, like holding a rifle, or even something like clapping hands. This will almost never work right, but Jupiter supports overriding a shared animation on a per-character basis, so you can make a model-specific version of any animation when needed. Remember that you cannot scale bones.

.LTA File Format

An important concept to remember about the .LTA file format is that it is a database that contains the model's geometry, bone hierarchy, level of detail information, and mode. The model database can refer to other files, especially if many models share the same set of animations.

All of the models in TO2 were created to be exported from Maya 4.0 using our exporter.

[Top](#)



Creating a Character

The sections below explain the steps required to create a character in Jupiter. All the characters in TO2 were created in Maya, so steps will vary from package to package. Also, be aware that most of the naming conventions and skeletal layout requirements can be changed by modifying the TO2 game code.

Skeletons

All the human characters in TO2 have the same skeletal hierarchy. This allows animations to be universally shared.

To share animations across models, keep their height within a range of about 15%. Having extra tall or short characters will cause strange floating when using a shared animation. Exported skeletons will still need to match the existing TO2 skeleton in order to share animations with TO2 models. This may require some experimentation. If you'd rather use a different skeleton (for example, a ponytail) then you will need to redo all of the animations used by that character or use attachments to add extra objects.

TO2 uses a node called Eyelid to control vertices that allow blinking. It is important to name this bone correctly since the TO2 game code uses this node to tell what a character is looking at.

Polygon Counts

All characters used in TO2 were modeled using 1000-3000 triangles.

Textures

All texture mapping for TO2 was done in Maya. Each character uses two base texture maps: one for the body and one for the head. These should be saved as 32 bit .TGA files. They are converted in DEdit to .DTX files by going to the Textures tab and importing the .TGA files to the `\Development\TO2\Game\ Chars\Skins` folder.

Naming

Any model that is going to use the Jupiter AI code needs to be named in the same form as the characters for TO2, such as `ninja.LTA`. There is no comprehensive list of appropriate character names, but looking at the `modelbutes.txt` file (in the `\Development\TO2\Game\attributes\` folder) should point you in the right direction.

Textures should be named in the same form as the models. You can use the `modelbutes.txt` file to define what textures are used where.

Export Location

Characters should be exported to the `\Development\TO2\game\chars\models\` folder.

ModelEdit Attributes

After exporting a character, you use ModelEdit to add several attributes including sockets, weight sets, LODs, user dims (used for physics), and internal dims (used for in-game clipping). Most of these can be imported from a similar existing model if there is one. Just select File, then Import and select a similar character file, and then check the items you want to import.

Weight Sets

Jupiter characters can use weight sets to allow morphing animations for up and down aims with guns and also for blending upper and lower body animations. These can be created individually, but it's a lot easier to just import them from an existing character if you have one. If your new model has a different skeleton, weight sets will not import properly and will need to be created from scratch.

Levels of Detail

Most TO2 characters have three levels of detail (LODs). These can be imported from existing characters, but only from a character that has the same number of pieces. If your new character has more or less pieces than any of the existing TO2 characters, you can still use the LOD values of the existing characters by manually copying the values onto the new character.

Child Model Translation Information

Another thing that must usually be done is to turn off the translation information from child models on most of the bones.

In the case of TO2 characters, study an existing character and check the box next to each node name in the Nodes list that is checked off in your own character. Not doing this may result in strangely stretched or squished characters. If you create your own skeletons, you should decide for yourself which nodes are used for translation.

Multiplayer Animations

TO2 Multiplayer characters use different animations than the single player ones. Therefore, duplicates must be created at this point.

Place the copy in the \Development\TO2\Game\Chars\Models\Multi\ folder. The child model animations are in this directory already.

Internal Radius

After adding these animations, set the model's internal radius. This is most easily done by going selecting Calc Internal Radius from the Model menu, but you can also set this value by hand on using Set Internal Radius on the Model menu. This value determines when the model appears and disappears from view. If you do not set an internal radius, your model will disappear as soon as its origin is out of view of the active camera. The internal radius should be exactly large enough to contain the model in any of its most extreme movements.

[Top](#)

Attachments

Any model can be used as an attachment, but won't animate without specifically being called by the game code.

To use an attachment in the game

1. Update the attachments.txt file in the \Development\TO2\Game\attributes folder. The attachments.txt file contains comments that will help you to understand how it works.
2. Copy and paste any attachment section to the end of the attachment list.
3. Change the number and name to be appropriate for your new attachment.
4. Also change the file paths to point to your attachment file and texture. These can be named however you want, and it's best to keep them in the \Game\Models\Attachments and skins\ folder.
5. The attachment must now be added to the character in DEdit.

Frame Strings

Frame strings are added to TO2 models to provide animation based cues to the game code. For characters, the ones used are **bute_sounds** for death sounds and **FOOTSTEP_KEY** for footsteps.

These are entered by going to the frame of animation where the event occurs and typing the appropriate string in the Frame String box. These can be placed on the child models when possible so many characters can use them.

Animation Editing Features

The simple animation editing features of ModelEdit, found in the Animation menu, can only be used on animations which are native to a model (not from a child model).

Use Set Animation Length to set the time the selected animation takes to play back to speed up or slow down an animation.

Use Set Animation Rate to get rid of frames in a really dense animation. This is mostly useful when dealing with motion capture which is at 30 frames/second. Most animations look fine at 8-10 frames/second using the linear interpolation of the engine.

Make Continuous is an easy way to make a simple cycling animation. Select an animation and delete the last couple of frames making a note of how much time is being cut by doing this. Then go to Make Continuous and enter the time that was noted above. This may need to be tweaked a little, but on animations that don't have a lot of action, usually this will give a good cycle.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Setting up a Player View Weapon

The player view (PV) hands, guns and vehicles for TO2 were modeled, UV textured and animated in Maya. Each weapon PV model was exported from Maya to an .LTA file and could then be opened in ModelEdit for final additions needed for the game code as well as tweaks to the animations. Finally, to get player view weapons to work properly in the game, outline the attributes for each weapon in the weapons.txt and the effects defined in the fx.txt.

Modeling

The PV weapon models in TO2 are somewhere between 1400 - 1800 triangles including the hand models.

Animating

Model exporters keep translation and rotation information but not scaling information. You should keyframe at least one piece or null object for every frame of the entire animation because fcurve information or similar non-keyframe interpolation information is not retained when exporting. The engine interpolates between keyframes linearly. The exporters only export keys for every keyframe in the scene.

Note: Some types of animations, such as character or vehicle turning, are implemented in the game code rather than being built into the models.

Exporting

In order for all of the geometry to be exported properly, all the pieces of the model and skeleton need to be part of the same hierarchy. Models should be exported to the `\Game\Guns\models_pv` directory.

Animation Naming

The animations for the weapons need to be very specific for the weapons to function properly in the game. You can use the TO2 models in the SDK to figure out what animations the game code refers to for different behaviors. If you prefer to use your own strategy for organizing weapon animations, you can change the game code and .BUT files to work however you wish.

PV Items in ModelEdit

After you export a PV (player-view) weapon or gadget or vehicle, use ModelEdit to add attributes, such as user dims, internal dims, sockets, command strings, and frame strings.

User Dims

User dims for all weapons in TO2 are the same: **X: 3.0, Y: 3.0, Z: 3.0**. Because the models are so small, these dimensions do not need to be exact. However, the default user dimensions for models are intentionally large to remind you that they need to be set.

Internal Dims

The model's internal radius must be set in order for it to draw properly. This is most easily done by selecting Calc Internal Radius from the Model menu, but you can also set this value by hand on the **Model>Set Internal Radius** menu. This value determines when the model appears and disappears from view. If you do not set an internal radius, your model will disappear as soon as its origin is out of view of the active camera. The internal radius should be exactly large enough to contain the model in any of its most extreme movements.

Muzzle Flashes

Muzzle flashes on weapons in TO2 are set up in the fx.txt file. The location of each weapon's muzzle is specified in its section in the weapons.txt attribute file. Both these files are found in `\Development\TO2\Game\Attributes`.

Command Strings

Models can have a command string that is executed when it's loaded. If you want to set per-model properties in your game code that aren't defined in ModelEdit or the TO2 game code, you can add code to process any message you want to put in the model's command string.

Frame Strings

Frame strings are added to provide animation based cues to the game code. These are entered by going to the frame of animation where their event occurs and typing the appropriate string in the Frame String box.

WEAPONS.TXT

The weapons.txt file found in the attributes folder defines all weapon and ammo properties: interface artwork, model, skin, position, and much more. For a better explanation, look at the weapons.txt for examples. The weapons.txt also has a brief description for what everything does.

FX.TXT

The fx.txt file found in the Attributes folder of your TO2 project defines all weapon effect properties: impact effects, muzzle flashes, and other effects like the flamethrower's pilot light. For a better explanation, look at the fx.txt for examples. The weapons.txt also has a brief description for what everything does.

Positioning PV Models

The PV models are positioned in the engine by using the mpwpos command when you hit the key that has been mapped for 'talk.' Hit the key (T by default in TO2) to switch to manipulation mode with the currently selected weapon or gadget. From there you can manipulate the model position by using the character movement keys. When the model is in the desired position enter the mpwpos command again and the position for that specific model will be saved to the weapons.txt file.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



ModelEdit UI Reference

This section describes the following menu commands available on the ModelEdit interface:

- [File Menu](#)
- [Model Menu](#)
- [Animation Menu](#)
- [Child Model Menu](#)
- [Sockets Menu](#)
- [Piece Menu](#)
- [Options Menu](#)
- [Model View Window](#)
- [The Animation Bar](#)
 - [Animation Edit](#)
 - [Transform Edit](#)
 - [Nodes List](#)
 - [Animations List](#)
- [Command String](#)

File Menu

Open—Opens a model (.LTA).

Import—Reads data from a model (.LTA, .ltc) into the current open model, such as new animations.

Save—Saves the model.

Save As—Saves the model under a different filename.

Exit—Quits ModelEdit.

[Top](#)

Model Menu

Model Info—Shows information about the model such as the amount of memory it uses and the number of triangles it contains.

Command String—Allows control over attributes of the model; See below for details.

Build LOD—Generates LOD information .

Rename Node—Renames a node.

Generate Vertex Normals—Recalculates the vertex normals based on neighboring polygons.

Remove Node—Deletes the selected node.

Set Texture—Changes the texture to be displayed on the model in the display window in ModelEdit. To display multiple textures, select a piece with the lowest texture index and click Set Texture to select its texture. Then select a piece with the next index and click Set texture to set its texture.

Set Internal Radius—Sets the radius of the visibility-culling sphere.

Calc Internal Radius—Automatically sets internal radius based on all the animations used by the model. Using this command instead of 'Set Internal Radius usually gives greater accuracy and is quicker.

Edit Weights—Dialog where you create weight sets on the model's nodes for animation blending.

[Top](#)

Animation Menu

Rename—Renames the selected animation.

Duplicate—Makes a copy of the selected animation.

Dimensions—Changes the user dimensions of the selected animation to determine its collision information.

Reverse—Reverses the time direction of the selected animation.

Make Continuous—Causes the selected animation to loop. This adds a length of time at the end of an animation so that it can interpolate back to the first keyframe.

Set Animation Rate—Changes the overall frame rate of the selected animation (or all animations) This does not directly set the animation's frame rate, but instead sets the number of stored keyframes per second. The engine then linearly interpolates to fill in the gaps.

Set Animation Length—Scales the length of the entire animation, making it play back faster or slower.

Set Animation Interpolation—Creates a dialog to set the interpolation time (in micro seconds) between two animations.

Create Single Frame—Creates a single-frame animation containing the current frame.

Translation—Adds a spatial translation from the model's origin to the selected animation.

Rotate—Rotates the selected animation.

Remove Extra Frames—Opens a dialog that attempts to clean out any redundant frames in the current animation. Best used to reduce the amount of unneeded data in a motion-capture file, but can also find extra keyframes you may have added by mistake.

[Top](#)

Child Model Menu

Add Child Model—Adds a new child model. Any .LTA file with an identical hierarchy to the current model can be used as its child model.

Remove Child Model—Removes the selected child model.

Rebuild Tree—Attempts to reconstruct and verify the current model's childmodel references.

[Top](#)

Sockets Menu

Sockets are transforms for attachments.

Add Socket—Adds a new socket.

Remove Socket—Removes the selected socket.

Rename Socket—Renames the selected socket.

[Top](#)

Piece Menu

Piece Info—Changes material attributes of the piece. The texture index can be changed here, as well as the specular attributes of the piece. You can use the texture index here to modify the indices you set when you exported the model, or use this value instead of setting it when exported. The specular power represents the size of any reflected light on the surface of the model. The larger the power the larger the light reflection will be. The scale attributes represent how matte the surface is. With a value close 0.5 the surface will not reflect very well, but with a value of 1.0 the surface will look very smooth and shiny. ModelEdit will not show the specular highlight. The engine uses the value when running.

Merge Pieces—Select a group of pieces in the piece window and use this to combine them into one piece, based on texture map. If you select all the pieces, the result will be 1 piece for each texture map the model uses. Do not use this on models with LODs, since the LOD pieces will combine incorrectly. This is mostly very useful for prop objects which contain many pieces.

Set Preview Texture—Select a piece in the piece window and use this menu command to apply a texture to it. When you save the model, the preview texture information will be saved as well. Note that this texture information is used only by ModelEdit. You will still need to set the character's/item's texture as usual in its attribute file.

[Top](#)

Options Menu

Lights Follow Camera—When toggled, the red/blue preview in ModelEdit lights move along with the camera.

Wireframe—Toggles wireframe rendering mode.

Show User Dimensions—Draws a box that shows the size of the selected animation's dimensions graphically. User dimensions define the bounding shape around models for physics.

Show Internal Dimensions—Draws a box that shows the size of the model's internal dimensions, which are used to determine the visibility clipping of a model. The internal dimensions should always *contain* all of the model geometry in any of its animations as *efficiently* as possible.

Show Skeleton—Draws the location of the nodes in the model.

Show Sockets—Draws the position and orientation of the sockets.

Show Attachments—Draws any attached models in relation to their socket (See Command String/SocketModel).

Solid Attachments—Toggles Solid/wireframe mode for any attached models.

Show Normals—Draws the vertex normals.

Show NormalRef—If supported by the model, displays the vertex normal frame of reference.

Profile—Shows performance information in the draw window. Note: this information doesn't determine the final performance of the model in your game.

New Background Color—Change the color of the model view window.

Change Field of View—Change the camera angle in the model view window.

[Top](#)

Model View Window

Mouse motion changes the camera position.

Left mouse rotates the camera around the model.

Right mouse dollies the camera in and out.

Right and left mouse combined pans the camera.

Shift left mouse restores the view.

[Top](#)

The Animation Bar

Click to move to the nearest key frame, represented by red lines. Double-click to tag a key frame. Double-click on the frame again to un-tag it. Right-click on a frame to see a context menu for the frame.

Frame Time—Allows skipping to a given time in the animation.

Frame String—Assigns a control string to the specified time in the animation.

LOD slider—Changes the displayed LOD of the model.

[Top](#)

Animation Edit

User Dimensions radio button—When this button is selected, the +/- buttons below it alter the size of the current animation's dimensions.

Translation radio button—When selected, the +/- buttons change the translation of the current animation

X—Deletes the current animation.

#—Moves the animation to the specified location in the model's list of animations. Game code often refers to model animations by index, so their order may be important.

Up arrow—Moves the animation up in the model's animation list.

Down arrow—Moves the animation down in the list.

[Top](#)

Transform Edit

Global—Toggles whether adjustments are node-relative or global.

Socket—Specifies that the Rotation and Translation color boxes will affect the model's sockets.

Relation—Specifies that nodes will be adjusted.

Rotation—These buttons each represent an axis of the currently selected socket or node (Red = X, Green = Y, Blue = Z). Click on the color of the axis you want to change and drag left/right to rotate around that axis.

Translation—Click on the color and drag left/right to move along that axis.

[Top](#)

Nodes List

If a node name has a check in its check box, the node will be placed relative to its parent and only rotation information from the animation data will be used. If you don't check off a node here, its bone length will be transferred from the child model to the parent along with the rotational data, causing potentially strange-looking scaling of a character.

[Top](#)

Animations List

A checkbox here indicates that an animation is from the current model and may therefore be changed/moved/etc.

[Top](#)

Command String

The command string is a semicolon-delimited string used to define extra control settings for a model in ModelEdit, the engine, and in the final game. Most commands are game specific, but some Command String commands are used by ModelEdit.

A model containing SocketModel commands will store its attachment model filenames in your machine's Windows registry so that you no longer need to re-specify an attachment each time you load the file in ModelEdit to preview it.

[Top](#)



FxED

Overview

This document details the new system for creating special effects for The Operative 2. It covers how to use the FxED application to create a group of FX and details the properties of each individual FX. I will go over some common uses for each FX and which FX should be used to create a specific special effect. This paper is intended as a reference tool for anyone with a creative mind who would like to create some cool looking special effects but all are welcome to read and comment on its contents. As changes are made to FxED, the individual FX and the system as a whole this document will be updated to reflect any changes or additions.

Creating special effects for previous games meant having an engineer and an artist sit down together to create and FX in code and then expose a few properties for tweaking in a text file. This severely limited the types of special effects we could have and limited the behavior of the FX we did have. For TO2 we decided to use the FxED application to create and edit all of our special effects. With FxED you can specify the length of a whole group of FX, the length of an individual FX, the scale, color and alpha at any point in time for an individual FX and you can even create a motion path for each individual FX to follow without setting up lots of key framers in DEdit.

General

You will see the word FX and GroupFX used in this document. FX refers to the individual objects that are considered special effects. A FX can be a ParticleSystemFX, SpriteFX, or even a NullFX. A GroupFX are a collection of FX. A GroupFX may contain one or many FX. When working with FxED you are creating GroupFX by adding FX to them.

Each FX has a set of properties that you can edit. Some of these properties are common between all the FX but most are specific to the individual FX. I will explain how to use each property of each FX later on.

Getting Started

FxED is the application used to create the GroupFX that will be used in the game. When FxED runs for the first time on your computer you will have to fill out a dialog box of information telling FxED where to look for resources and the file containing the FX. If you wish to run the game from within FxED you will need to edit the “App to execute...” to be the full path name of your Lithtech.exe file. You can also enter in optional information into the command line if you wish to run the game in a window or load a specific level. You must fill out the “Location of resource file...” box. This tells FxED where to look for the resources such as .ltb’s and .spr’s, this should be the full path of your Game directory. The “Location and name of clientfx.fxd...” box also needs to be filled out in order to get the list of FX you can use. This should be the full path from the “Location of resource file...” box with “\clientfx.fxd” added to the end. FxED is now ready for you to work with and will never ask you for this information again. If you made a mistake in entering in the info or would like to change some options for the command line you can do so by selecting “Edit Game Info” from the File menu.

You will now want to open the file that contains all the GroupFX for the game. Click on the open icon and open the clientfx.fxf file found on the ClientFX directory of your Game directory. You will now have a list of GroupFX that you can open and edit.

Creating A GroupFX

To create a new GroupFX simply click on the “new” icon and a blank GroupFX will appear. By default all GroupFX start off with a Phase Length of 10000. Phase Length is the lifetime of the GroupFX in milliseconds so each GroupFX is 10 seconds long by default. To change this, just click on the button that says 10000 and enter in a new time. To start adding FX to this GroupFX you will first need to add a Track. Tracks are holders for Keys. Keys are the specific individual FX. Each Key needs to be placed on a Track, you can have multiple Keys per Track or you can have each Track contain only one Key. Once you have added a Track by right clicking in the gray area of the GroupFX and selecting Add Track you will need to add a Key. Right click on an empty space in a Track and select Add Key, a popup box will appear with a list of FX you can add for this Key. Select the type of FX you would like to add and it will then appear on your Track. You can now move this Key around your Track so you can set when you would like this FX to begin playing. You can also lengthen the FX by grabbing its edges and growing or shrinking the Key to the desired length. To have the FX play for the whole duration of the GroupFX right click on the Key and select Expand Key from the menu. If you have many FX that are of the same type you may want to differentiate them from each other by giving them each a unique name. To do this right click on the key and select “Name Key” from the popup menu

and enter in a custom name for this Key.

Now that you have your Key set to the desired length and start time you can edit the properties of the FX by right clicking on the Key and selecting Edit Key. A popup dialog box appears with several different properties listed for you to edit. From the same right click menu you can select options to edit the scale of the FX, the color of the FX and create motion for the FX. Not all FX types expose the ability to edit all of these options. A list of FX and properties that you can edit are listed below.

Color Keys

For FX that allow you to create and edit color keys you access this interface by right clicking on a Key in the GroupFX and select “Edit Color Keys” from the menu. This will bring up a dialog box with a large colored bar numbered from 0 to the lifetime of the Key. The full time of the FX is represented in the bar along the bottom the, the alpha of the FX at a particular time is represented on the left side. From here you can create new color keys or edit the two preexisting keys. There are always at least two color keys per FX, one for the very beginning of the FX and one for the end of the FX. By default these two keys are set to the color white and are fully opaque. To create a new key, simply click in the color bar at the time you wish the new key to be and at the alpha you want the new key to be. You are then prompted to choose a color for the key. After a key is created you can move it around to the time and alpha desired by left clicking the key dragging it to the desired values. You can change the color of a color key by right clicking on one and choosing a color from the color menu. To delete a key simply select it by left clicking and then click the Delete Key button. You cannot delete the two preexisting beginning and end keys. If you create a set of color keys that you wish to use for other FX you can save them by clicking on the “Add To Favorites” button, you can select from a list of favorite color keys by selecting the choose button.

Scale Keys

For FX that allow you to create and edit scale keys you access this interface by right clicking on a Key in the GroupFX and select “Edit Scale Keys” from the menu. This will bring up a dialog box with a large white bar numbered from 0 to the lifetime of the FX. The full lifetime of the FX is represented in the bar at the bottom and the Scale of the FX at a particular time is represented on the left side, ranging from Min Scale to Max Scale. From here you can create new scale keys or edit the two preexisting keys. There are always at least two scale keys per FX, one for the very beginning of the FX and one for the end of the FX. By default these two keys are set to a scale of 1. To change the Min and Max scale for this FX click on the numbered buttons for Min Scale and Max Scale and then enter a new value. To create new scale keys, click in the white bar at the time and scale you desire, you may move the key to the desired time and scale after creating by clicking and dragging the key. To delete a key, right click on it and then select “delete key” from the menu. You cannot delete the two preexisting beginning and end keys. If you wish to start over again click the “Reset” button. This will delete all but the two beginning and end keys and set them to be at 10% of the Max Scale value. If you create a set of scale keys that you wish to use for other FX you can save them by clicking on the “Add To Favorites” button, you can select from a list of favorite scale keys by selecting the choose button.

FX

ParticleSystem

Features

This is probably the most used FX in the list. The particle system emits individual particles based on the properties that you fill out. This FX allows you to edit it’s Color Keys, Scale Keys and Motion Keys. The particles are always facing the camera and will never rotate. You can specify the particles to use a texture or a sprite. The Color Keys and Scale Keys for this FX are per particle, not per system. This means the particle will interpolate between its scale and color keys based on its own lifetime.

Use

The ParticleSystems can be used almost anywhere for anything. They can be used as smoke from a steam pipe, fire, sparks and smoke from a gun and many other possibilities. You can set them up to be an impact FX and emit a puff of smoke when a bullet hits a wall, or as bubbles in the water.

Color Keys

This FX allows you to edit its color keys. The Color Keys for a Particle System are per particle. So every particle will use the color keys based on their own lifespan.

Scale Keys

This FX allows you to edit its Scale Keys. The Scale Keys for a Particle System are per particle. So every particle will use the scale keys based on their own lifespan.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Sprite

The ParticleSystem can use an .spr file as its particles and each particle will loop through the textures in the sprite. To select a .spr file click the button and browse through the Resource Browser and select the desired file.

Texture

If no Sprite is given, the ParticleSystem can use a single .dtx as its file for the particles to display. To select a .dtx file click the button and select the file through the Resource Browser.

EmissionInterval

This controls the length of time, in seconds, between emissions of the system. Typically this is set low so the system is constantly emitting particles, however if you want particle to be emitted in bursts raise the value.

ParticlesPerEmission

Every time the system emits particles, based on the “EmissionInterval”, it will emit this many particles.

GravityAcceleration

This controls the speed and direction the particles will float or sink. A negative value will make the particles sink to the ground and a positive value will float the particles up towards the sky. Gravity is always in relation to the world, no matter what the systems rotation is.

MinParticleLifeSpan

Each particle will last for at least this amount of time, in seconds.

MaxParticleLifeSpan

Each particle will last no longer than this amount of time, in seconds. The life span of each particle is a random number between the Min/MaxParticleLifeSpan. If you want every particle to last the same amount of time, making these exactly the same.

MinRadius

This is the minimum distance from the center of the system that the particles will be emitted.

MaxRadius

This is the maximum distance from the center of the system that the particles will be emitted.

EmissionPlane

This is the normal vector of the plane that the particles are emitted from. By default this is a plane parallel to the systems Up vector. If you are using a ParticleSystem as an impact FX this should be set to 0.0 0.0 1.0.

MinParticleVelocity

This controls the minimum speed and direction that each particle will have when they are emitted. The velocity is relative to the system so if you never want particles to be emitted in the downward direction of the system, make the Y component positive. For impact FX the Z component should be positive so particles never get emitted into the surface they hit.

MaxParticleVelocity

This controls the maximum speed and direction that each particle will have when they are emitted.

Type

A dropdown list box of predefined emission types for the system:

- Sphere – The particles will emit in all directions around the center of the system a distance no less than MinRadius from the center and no more than MaxRadius from the center.
- Point - The particles will emit from the same random point on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius from the center. If you wish to have the particles emit from the direct center of the system make Min/MaxRadius 0.0
- Circle - The particles will emit in a circle on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius From the center. Playing with Min/MaxRadius can yield an emission pattern of a circle, doughnut, or ring.

Bounce

If set to Yes, each particle will collide with and bounce off geometry and solid objects in a realistic manner. NOTE: This can dramatically decrease frames and slow the game. This should be used sparingly and only with systems that have few active particles.

FlipRenderingOrder

By default the newest particle particles are drawn first, over the older particles in the system. You can change the order so older particles are drawn over newer ones by setting this to yes.

BlendMode

Depending on the texture or sprite used you may wish to set the system to use Additive or Multiply blending.

MoveParticlesWithSystem

If a particle system is moving or rotating, the particles will move and rotate with the system. You can override this behavior by setting this property to No. For speed reasons, if a system is staying still (ie. Not moving or rotating), keep this value set to Yes.

Sprite**Features**

The Sprite FX displays a sprite specified through a .spr file. The sprite can rotate, be flat against a wall, or always face the camera.

Use

There are many uses for a sprite, one of the most common is to put a rotation on it and use it as a smoke or gas cloud. You can also use it as a smoke puff coming from the barrel of a gun.

Color Keys

This FX allows you to edit its color keys. The Sprite's color and alpha will change over time by interpolating between the color keys.

Scale Keys

This FX allows you to edit its Scale Keys. The Sprite's scale will change over time by interpolating between the scale keys.

Properties**UpdatePos**

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Sprite

This is the sprite, .spr, file used for the Sprite FX. To select a .spr file click the button and browse through the Resource Browser and select the desired file.

Normal

This is the normal the sprite will be facing when set to face along the normal. By default this is set to be the forward vector of the sprite.

Facing

A dropdown list of presets that tell how the sprite will be oriented.

- CameraFacing – The sprite will always be oriented so its forward is facing the camera.
- AlongNormal – The sprite will face in the direction specified by the Normal Property.
- ParentAlign – If the FX has a parent then the sprite will have the same rotation as the parent object.

BlendMode

Depending on the sprite used you may wish to set it to use Additive or Multiply blending.

DisableZ

When this is set to Yes the sprite will look as though it is being drawn over everything else.

DisableLight

This controls whether or not dynamic lights affect the color of the sprite.

Null

Features

This FX is used solely to give some random motion to other FX within a GroupFX. Nothing gets drawn for this FX; its only purpose is to keep track of a position for other FX to use. Other FX, a single FX or multiple FX, should motion link to this FX to give them a position to follow.

Use

Other FX should motion link to this FX. It should not be used as a stand alone FX. You can motion link any FX to this Null FX to give them a random motion. PolyTrails should motion link to this FX to create sparks and trails. You can link multiple FX within a GroupFX to this FX so it gives them a since of motion together.

Color Keys

This FX does not allow you to edit its color keys.

Scale Keys

This FX does not allow you to edit its Scale Keys.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

GravityAcceleration

This controls the speed and direction the FX will float or sink. A negative value will make the FX sink to the ground and a positive value will float the FX up towards the sky. Gravity is always in relation to the world, no matter what the FX rotation is.

MinVelocity

This controls the minimum speed and direction the FX will have in relation to its self and not the world.

MaxVelocity

This controls the maximum speed and direction the FX will have. The velocity is determined by getting a random value between the MinVelocity and MaxVelocity so each time the FX is played the motion will be different.

Bounce

If set to Yes, the FX will collide and bounce off geometry and solid objects in a realistic manner.

LTBModel

Features

This FX allows you to display a model that can fade in and out, scale, rotate and change the default animation length.

Use

The LTBModel can be used as a chunk of something flying through the air, a dead body or you can set it up so many LTBModels are used as a gib FX.

Color Keys

This FX allows you to edit its color keys. The Model's color and alpha will change over time by interpolating between the color keys. Alpha will only work on models with an additive RenderStyle associated with it.

Scale Keys

This FX allows you to edit its Scale Keys. The Model's scale will change over time by interpolating between the scale keys.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Model

This is the model, .ltb, file used for the LTBModel FX. To select an .ltb file click the button and browse through the Resource Browser and select the desired file.

Skin 0-4

The skin properties are used to texture the model. Each skin should correspond to the texture index of the models textures set in model edit. To select a skin, click the button and select the .dtx or .spr you wish to use.

Normal

This is the normal the model will be facing when set to face along the normal. By default this is set to be the forward vector of the model.

Facing

A dropdown list of presets that tell how the sprite will be oriented.

- CameraFacing – The model will always be oriented so its forward is facing the camera.
- AlongNormal – The model will face in the direction specified by the Normal Property.
- ParentAlign – If the FX has a parent then the model will have the same rotation as the parent object.

OverrideAniLength

You can manually set the length of the animation or let the length of the FX determine the length by setting this to Yes.

AniLength

If OverrideAniLength is set to Yes and this is non-zero then this length, in seconds, will be used to set the length of the animation. If this is 0.0 and OverrideAniLength is set to Yes, then the lifespan of the FX is used to set the length of the animation.

RenderStyle 0-3

The renderstyle properties are used to set renderstyles on the model. Each renderstyle should correspond to the renderstyle index of the model set in model edit.

DynaLight

Features

This FX creates a Dynamic light with the color and scale specified. The color, scale and Intensity of the light can change over time.

Use

The DynaLight FX can be used to show something is bright and emitting light. A good example of this is for a MuzzleFlash or for a fire or explosion.

Color Keys

This FX allows you to edit its color keys. Intensity of the light is controlled through the color keys by setting the Alpha. Fully Opaque is fully bright and Fully Translucent is No light. The Light's color and intensity will change over time by interpolating between the color keys.

Scale Keys

This FX allows you to edit its Scale Keys. The Light's scale will change over time by interpolating between the scale keys. The scale is in DEdit units so for this FX the scale should be at least 100 to even show up. Because changing the scale of a dynamic light forces the engine to recalculate the lightmaps, it is better to fade a DynaLight in and out using the intensity scale. And it looks better to ☺

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Flicker

This will give the light a slight flicker feel to it by slightly changing the color and intensity.

PlaySound**Features**

This FX allows you to play a sound at anytime during the GroupFX. You can control the radius, volume and whether or not the sound loops.

Use

Any time you want a specific sound to play like a fire crackling, or a steam vent you can use this to play a sound.

Color Keys

This FX doesn't allow you to edit its color keys.

Scale Keys

This FX doesn't allow you to edit its scale keys.

Properties**UpdatePos**

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach

to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Sound

This is the file, .wav, used as the sound to play. To select a sound, click the button and select the .wav you wish to use.

Loop

If this is set to Yes, then the sound will loop for the duration of the GroupFX.

PlayLocal

If set to Yes, the sound will feel as though its playing in the players head.

InnerRadius

If the player is within this distance from the center of the FX they will hear the sound at full volume.

OuterRadius

If the player is between the OuterRadius and the InnerRadius they will hear an interpolated volume for the sound. If outside this distance the player will not hear the sound.

Volume

This is the full volume of the sound heard within the InnerRadius.

Priority

How important is this sound. If there are too many sounds playing at once the lowest priority sounds are not played.

PitchShift

This is a multiplier to the playback frequency of the sound, which will shift its pitch. If you had a sound recorded at 22,000 Hz and set its pitch shift to 1.1f, then the output sound would be 24,200 Hz and make the sound have a higher pitch.

CamJitter

Features

This FX will shake the camera around as though the player is in an earthquake.

Use

This FX is good for simulating a shaky ground. When an explosion occurs it is a nice effect to shake the camera.

Color Keys

This FX does not allow you to edit its color keys.

Scale Keys

This FX allows you to edit its Scale Keys. The Scale Keys for a CamJitter FX controls how intense the shaking is. This allows you to fade the shaking off rather than just abruptly stopping it.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

InnerRadius

If the player is within this distance from the center of the FX they will feel the shaking at full force.

OuterRadius

If the player is between the OuterRadius and the InnerRadius they will feel an interpolated shake. If outside this distance the player will not feel the shake.

WonkyFX

Features

This FX plays with the FOV of the camera to create a wavy, watery effect. The cameras X, and Y FOV increases and decreases.

Use

The WonkyFX probably wouldn't be used very much and might only be useful for underwater type effects, or hallucination, type effects.

Color Keys

This FX does not allow you to edit its Color keys.

Scale Keys

This FX does not allow you to edit its Scale keys.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

xMultiplier

This controls the amount of fluctuation in the xFOV. Increasing this value increases how far the xFOV stretches.

yMultiplier

This controls the amount of fluctuation in the yFOV. Increasing this value increases how far the yFOV stretches.

InnerRadius

If the player is within this distance from the center of the FX they will feel the wobble effect at full force.

OuterRadius

If the player is between the OuterRadius and the InnerRadius they will feel an interpolated wobble. If outside this distance the player will not feel the wobble.

PolyTrail

Features

This FX will draw a series of polygons following a path to create a motion trail behind the position the FX is following. This FX should be motion linked to another FX to give it more of a moving feel or attached to a node or socket of a model to give it a moving feel.

Use

Motion link a PolyTrail FX to a NullFX and you can create sparks from a bullet hitting metal, or electrical sparks from the end of a downed power line. Attach this FX to a grenade or rocket to produce a motion trail or even a smoke trail behind it. Attach this FX to a socket on a model, like a sword, to show the path of motion.

Color Keys

This FX allows you to edit its color keys. The Color Keys for a PolyTrail are per trail section and not the trail as a whole. So the section closest to the head will be the color of the first key and the tail section will be the color of the last key.

Scale Keys

This FX does not allow you to edit its Scale keys.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach – If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Texture

The PolyTrail can use a .dtx as its file for the trail to display. To select a .dtx file click the button and select the file through the Resource Browser.

TrailWidth

This is the width of the trail. Make this small for effects like sparks.

WidthStyle

A dropdown list how the trail is drawn:

- Constant – The trail is always at the constant width set by TrailWidth.
 - SmallToBig – The trail starts small and ends big.
- SmallToSmall – The trail starts small, is big in the middle, and is small at the end again.
 - BigToSmall – The trail starts big and ends small.

TrailLen

This is the Length of the trail given by number of segments. For a straight line trail this should be 1 but for a very curvy trail this should be much higher.

SectionLifespan

This represents the lifetime of each trail segment in seconds.

UAdd

An offset for the texture coordinates.

SectionInterval

The time spans between trail segments, in seconds. Every time this interval is hit a new trail segment is used.

BlendMode

A drop down list of various blend mode operations that can be performed to get the desired effect.

AlphaTest

A drop down list of various alpha tests to perform when determining weather or not to draw the trail.

ColorOp

A drop down list of various color operations such as Additive and Multiply to perform when determining how to draw the trail.

FillMode

A drop down list with two options for filling the trail. Fill just draws the trail normally and Wireframe draws lines to outline the polygons of the trail.

PlayRandomSound

Features

Just like the PlaySound FX except this FX will let you play a random sound from a selected set of sounds. You provide the base sound and then it randomly grabs sound with that base and a number appended at the end.

Use

If there are several sounds available for an effect and you don't want to use just one for fear of being redundant this is what you should use. This FX is good to use if a single GroupFX is used and played quite often to give it a more random and non set feel.

Color Keys

This FX doesn't allow you to edit its color keys.

Scale Keys

This FX doesn't allow you to edit its scale keys.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Sound

This is the file, .wav, used as the base sound to play. To select a sound, click the button and select the .wav you wish to use.

NumRand

This is how many sounds there are to use and the sound will be played by getting a random number between 0 and NumRand.

Loop

If this is set to Yes, then the sound will loop for the duration of the GroupFX.

PlayLocal

If set to Yes, the sound will feel as though its playing in the players head.

InnerRadius

If the player is within this distance from the center of the FX they will hear the sound at full volume.

OuterRadius

If the player is between the OuterRadius and the InnerRadius they will hear an interpolated volume for the sound. If outside this distance the player will not hear the sound.

Volume

This is the full volume of the sound heard within the InnerRadius.

Priority

How important is this sound. If there are too many sounds playing at once the lowest priority sounds are not played.

PitchShift

This is a multiplier to the playback frequency of the sound, which will shift its pitch. If you had a sound recorded at 22,000 Hz and set its pitch shift to 1.1f, then the output sound would be 24,200 Hz and make the sound have a higher pitch.

SpriteSystem

Features

This FX is almost identical to the ParticleSystem FX except that the Sprites can rotate about their own axis. The Sprite system emits individual sprites based on the properties that you fill out. This FX allows you to edit its Color Keys, Scale Keys and Motion Keys. The Color Keys and Scale Keys for this FX are per sprite, not per system. This means the sprite will interpolate between its scale and color keys based on its own lifetime.

Use

The SpriteSystem can be used almost anywhere that a Particle system can be used for and more. The major difference of the Sprite system is that the individual sprites can rotate which can create a sense of movement in smoke and gas clouds. Typically ParticleSystems look and better and give better performance but for things like gas clouds and smoke puffs, adding a rotation makes all the difference in the world.

Color Keys

This FX allows you to edit its color keys. The Color Keys for a Sprite System are per sprite. So every sprite will use the color keys based on their own lifespan.

Scale Keys

This FX allows you to edit its Scale Keys. The Scale Keys for a Sprite System are per Sprite. So every sprite will use the scale keys based on their own lifespan.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.

- **SocketAttach** - If this is selected the FX will attach to the specified socket.
- **PV_NodeAttach** – This will attach the FX to a node in a PlayerView model.
- **PV_SocketAttach** – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Sprite

The SpriteSystem can use a .spr file as its sprite and each sprite will loop through the textures in the sprite. To select a .spr file click the button and browse through the Resource Browser and select the desired file.

EmissionInterval

This controls the length of time, in seconds, between emissions of the system. Typically this is set low so the system is constantly emitting sprites; however if you want sprites to be emitted in bursts, raise the value.

ParticlesPerEmission

Every time the system emits sprites, based on the “EmissionInterval”, it will emit this many sprites.

MinSpriteRotation

The minimum rotations a sprite will rotate around each axis.

MaxSpriteRotation

The maximum rotation a sprite will rotate around each axis. The sprites will get its rotation by randomizing between Min/MaxSpriteRotation.

GravityAcceleration

This controls the speed and direction the sprites will float or sink. A negative value will make the sprites sink to the ground and a positive value will float the sprites up towards the sky. Gravity is always in relation to the world, no matter what the systems rotation is.

MinSpriteLifeSpan

Each sprite will last for at least this amount of time, in seconds.

MaxSpriteLifeSpan

Each sprite will last no longer than this amount of time, in seconds. The life span of each sprite is a random number between the Min/MaxSpriteLifeSpan. If you want every Sprite to last the same amount of time, make these exactly the same.

MinRadius

This is the minimum distance from the center of the system that the sprites will be emitted.

MaxRadius

This is the maximum distance from the center of the system that the sprites will be emitted.

EmissionPlane

This is the normal vector of the plane that the sprites are emitted from. By default this is a plane parallel to the systems Up vector. If you are using a SpriteSystem as an impact FX this should be set to 0.0 0.0 1.0.

MinSpriteVelocity

This controls the minimum speed and direction that each sprite will have when they are emitted. The velocity is relative to the system so if you never want sprites to be emitted in the downward direction of the system, make the Y component positive. For impact FX the Z component should be positive so sprites never get emitted into the surface they hit.

MaxParticleVelocity

This controls the maximum speed and direction that each sprite will have when they are emitted.

EmissionType

A dropdown list box of predefined emission types for the system:

Sphere – The sprites will emit in all directions around the center of the system a distance no less than MinRadius from the center and no more than MaxRadius from the center.

Point - The sprites will emit from the same random point on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius from the center. If you wish to have the sprites emit from the direct center of the system make Min/MaxRadius 0.0

Circle - The sprites will emit in a circle on the emission plane at a distance of no less than MinRadius from the center and no more than MaxRadius From the center. Playing with Min/MaxRadius can yield an emission pattern of a circle, doughnut, or ring.

AType

A dropdown list for determining the additive type to use for the sprites. Norm uses no additive blending and Add uses additive to render the sprites

StretchU

An offset to use to stretch the textures of the sprite.

StretchV

An offset to use to stretch the textures of the sprite.

CreateFX

Features

This is a unique FX in that it doesn't render anything or really do anything but start another premade GroupFX. You can specify the name of a GroupFX and whether or not the new FX will loop so you can combine different GroupFX together quickly and easily. You have the option to use a blinding flare by editing the Blind properties. A blinding flare will disable the Z and be drawn over everything else if there is a clear path to the camera.

Use

Whenever you want to start a preexisting GroupFX at a certain time within another GroupFX you can use this FX to set it up. The new GroupFX will begin playing at the time specified by this FX's Key and will run independently of the GroupFX that contains it. This could mainly be used if you have an explosion that you want to play and then a looping cloud of smoke in the middle of the explosion to keep playing after the explosion finishes.

Color Keys

This FX does not allow you to edit its Color keys.

Scale Keys

This FX does not allow you to edit its Scale keys.

Properties

FXName

This is the name of the preexisting GroupFX that you wish to play.

Loop

If set to Yes, the newly created GroupFX will loop and continue to play after the GroupFX that contains it is finished.

FlareSprite

Features

The FlareSprite FX displays a sprite specified through a .spr file and will scale and fade the sprite based on angles of the camera and sprite. The sprite can rotate, be flat against a wall, or always face the camera.

Use

The most common use for a FlareSprite is as a "halo" of sorts for a light.

Color Keys

This FX allows you to edit its color keys. The Sprite's color and alpha will change over time by interpolating between the color keys.

Scale Keys

This FX doesn't allow you to edit its Scale Keys.

Properties

UpdatePos

This property is a drop down list of predefined ways to update the FX position

- Fixed – This is the most common option and will keep the FX at a fixed position.
- Follow – If the FX is motion linked to another FX UpdatePos must be set to follow
 - PlayerView – If you are using this FX as a MuzzleFlash, select PlayerView.
 - NodeAttach – If this is selected the FX will attach to the specified node.
 - SocketAttach - If this is selected the FX will attach to the specified socket.
 - PV_NodeAttach – This will attach the FX to a node in a PlayerView model.
 - PV_SocketAttach – This will attach the FX to a socket on a PlayerView model.

UpdateInterval

This is the length, in seconds, between updates for this FX. This is normally set very low so the FX can update every frame. If constant updating is not needed then the UpdateInterval should be set higher.

AttachName

If any of the “Attach” options were selected for UpdatePos this property must be filled out so the FX knows which socket or node to attach to.

Offset

If the value of this vector property is anything besides zero the FX will be placed at this position offset from its center.

RotateAdd

If the value of this property is anything besides zero the FX will rotate this amount around the axis every update.

Sprite

This is the sprite, .spr, file used for the Sprite FX. To select a .spr file click the button and browse through the Resource Browser and select the desired file.

Normal

This is the normal the sprite will be facing when set to face along the normal. By default this is set to be the forward vector of the sprite.

Facing

A dropdown list of presets that tell how the sprite will be oriented.

- CameraFacing – The sprite will always be oriented so its forward is facing the camera.
- AlongNormal – The sprite will face in the direction specified by the Normal Property.
- ParentAlign – If the FX has a parent then the sprite will have the same rotation as the parent object.

BlendMode

Depending on the sprite used you may wish to set it to use Additive or Multiply blending.

DisableZ

When this is set to Yes the sprite will look as though it is being drawn over everything else.

DisableLight

This controls whether or not dynamic lights affect the color of the sprite.

MinAngle

This is the minimum angle that the FX will use to calculate the scale, color and alpha of the Sprite. If the angle is less than this the sprite will interpolate its color, scale and alpha from 0, being the max, and MinAngle being the minimum. If the angle is greater than MinAngle, the FlareSprite is invisible.

ObjectAngle

This is a dropdown list to choose which object to use to calculate the angle. If Sprite is chosen the MinAngle is compared against the angle from the sprites forward to the camera. If Camera is chosen then the angle used is from the forward direction of the camera to the center of the sprite.

MinAlpha

When the angle is between 0 and the MinAngle the alpha will go no lower than MinAlpha.

MaxAlpha

When the angle is between 0 and MinAngle the alpha will go no larger than MaxAlpha.

MinScale

When the angle is between 0 and the MinAngle the scale will go no lower than MinScale.

MaxScale

When the angle is between 0 and MinAngle the scale will go no larger than MaxScale.

BlindObjectAngle

You can choose which object to use for the angle, sprite or camera, to calculate the scale.

BlindSpriteAngle

This is the angle from the Sprite to the camera that is compared against for creating the blinding flare. If the angle is greater the BlindSpriteAngle then the blinding flare is not created.

BlindCameraAngle

This is the angle from the Camera to the Sprite that is compared against for creating the blinding flare. If the angle is greater than BlindCameraAngle then the blinding flare is not created.

BlindMaxScale

If the angle from the Sprite's forward to the Camera is less than BlindSpriteAngle and if the angle from the Camera's forward to the Sprite is less than BlindCameraAngle then the angle, specified by BlindObjectAngle, is used to calculate the scale of the sprite from MaxScale to BlindMaxScale.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



3D Studio Max Brush Exporter

This exporter plug-in for 3D Studio MAX provides a full-featured approach to generating terrain for use within LithTech. The user can model terrain using any MAX geometry modeling techniques. Textures can be applied and mapped using MAX tools. The end result is an .LTA file that retains all the Max texture mapping and model hierarchy information.

This section contains the following topics related to exporting brushes created in 3D Studio Max:

- [Installation](#)
- [Texture Setup](#)
- [Creating Level Geometry](#)
- [Applying Textures to Brushes](#)
 - [Export Options](#)

Installation

The `%install%\tools\plugins\MAXWorldExport.dle` plug-in file must be placed within a plug-in path used by 3D Studio Max. After that, the plug-in is accessed through the **File>Export** and **File>Export Selected** menu commands within 3D Studio.

Texture Setup

The textures you apply in 3D Studio must be the same resolution as the textures you use within your DEdit project. They must also exist in a directory tree that has exactly the same layout as your DEdit texture directory tree. The following example path shows what a sample DEdit resource directory might look like:

- c:\coolgame\resource
 - \worlds
 - \textures
 - \wood
 - \wood1.dtx (a 256x256 .dtx file)
 - \metal
 - \brushedalum.dtx (a 128x128 .dtx file)

The corresponding MAX texture tree for the previous DEdit resource directory would look as follows:

- d:\artresources\maxstuff
 - \textures
 - \wood
 - \wood1.tga (a 256x256 .tga file)
 - \metal

- \brushedalum.jpg (a 128x128 .jpg file)

When exporting the terrain, you would enter `d:\artresources\maxstuff` as your base texture path. This way, textures in the .LTA file would be referred to as `textures\wood\wood1.dtx` and `textures\metal\brushedalum.dtx` and DEdit would correctly find the textures that have been applied within MAX.

Note: you only need corresponding MAX textures for those that you will be applying to your terrain.

[Top](#)

Creating Level Geometry

Any geometry you can see within the MAX viewports should export well. The exporter will not handle render-specific geometry, such as displacement maps.

The hierarchy of objects is retained in the .LTA file. If you group several objects together, they will show up in DEdit as Container nodes for each of the objects, along with a parent container grouping them together. This is true of normal linked objects as well.

[Top](#)

Applying Textures to Brushes

Once you have the corresponding texture directory set up, you can begin texturing your brushes within MAX. Apply texture coordinates using whatever tools you want. Only the first UV channel is used by the exporter.

Texture coordinates alone aren't enough information for the exporter to generate correct mappings within the LTA. You must also apply textures of the correct resolution through the material editor. All materials assigned to the geometry must be "Standard" materials, or a "Multi/Sub-Object" materials with only Standard materials in the sub-material slots. Each of the materials that is referred to by a face in the scene must have a single "Bitmap" texture within its diffuse map slot. The Bitmap texture must use a texture in the texture tree that you set up following the above instructions.

If any of the exported faces doesn't have the materials assigned correctly to it, you will be warned once the export has completed. The .LTA file will still contain all the correct geometry, but texture information will use defaults for faces without valid materials.

[Top](#)

Export Options

Use MAX Texture Information

If this option is checked, the exporter will attempt to generate LithTech mapping coordinate information from the UV coordinates and materials assigned to the objects. If it isn't checked, default mapping coordinates and textures will be applied to the LTA. Defaults to on.

Pivot Point to Origin

If this option is checked, all exported objects will be placed at the origin in the LTA file, ignoring their relative positioning within MAX. Defaults to off.

Base Texture Path

This path is stripped off the front of the full path of all textures that are exported. This allows the user to have their MAX texture files in a directory other than their LithTech resources directory, but still have DEdit read the textures from the resources directory. Clicking on the button to the right of the edit field will bring up the standard MAX directory selection dialog.

Export Animated WorldModels

Animated objects in MAX are saved to the .LTA file as animated worldmodels. KeyFramer and Key objects will be placed as needed in the .LTA file. Any object with set keys in MAX will become its own WorldModel, along with its children. If any of the children are also animated, they will become separate WorldModels, and so on. Time settings in MAX will be used for determining the time values for Key objects in the .LTA file.

The Exporter will not delete any vertices when merging triangles into polygons. For example, when exporting a cylinder created in MAX, an extra triangle will be added to the end-caps of the created brush. This is because MAX adds a vertex in the center of the end-caps. The designer should remove any vertices that are in the middle of polygons and not connected to any visible edges.

Texture support on brushes works the same as it does for the terrain export. Only planar projections will work for brush faces that have more than 3 vertices. Texture coordinate tools can still be used, but each face must have a uniform mapping across the entire face in order to export correctly.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Content Guide



Max .LTA World Importer

The Max World Importer plug-in complements the Max Terrain Exporter but is more versatile in that it will import any full .LTA level file into Max. Every brush and terrain object in the level will be imported into Max as an independent mesh, with textures aligned exactly as they were in DEdit.

This section contains the following topics related to importing worlds created in 3D Studio Max:

- [Installation](#)
- [Texture Setup](#)
- [Importer Options](#)

Installation

Place the plug-in file `%install%\tools\plugins\MaxWorldImport.dli` within a plug-in path used by 3D Studio Max. After that, the plug-in is accessed through the File menu Import command within 3D Studio Max.

Texture Setup

When an .LTA file is imported into Max, all textured surfaces show as green (until textured viewing is enabled in Max), and all untextured surfaces show as gray. The Max .LTA Importer assumes that for every .DTX texture file used in a level, there is an identically named texture stored in the same location in the .TGA format. If it can't find the appropriate .TGA file for a surface, that surface will remain plain green even when rendering and displaying textures in the Viewport.

To batch convert all the .DTX textures in your project to .TGA files, copy dtxutil to your main resource directory and run it with the following command line switches:

```
dtxutil -r_dtx2tga [the texture directory] copy
```

Note: The texture directory must be the actual directory name, and not the path of the directory. Currently, this converter will usually display various warnings and errors while it runs, but these can be safely ignored.

[Top](#)

Importer Options

Merge Objects with the Current Scene

If this option is checked, the importer retains the contents of the current scene being edited in Max, and inserts the imported .LTA level in it.

Completely Replace the Current Scene

If this option is checked, the importer erases the contents of the current scene being edited in Max, and replaces it with the imported .LTA level.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Content Guide



Maya Brush Exporter

This exporter plug-in for Maya allows creation of brushes and terrain within Maya. The plug-in consists of two parts. There is the UI front-

end, which consists of a Maya file translator plug-in and the associated UI MEL script. Optionally, a user can directly call the MEL command `lithtechWorldExport` to manually export a scene.

This section contains the following topics related to exporting a brush created in Maya:

- [Installation](#)
- [Texture Setup](#)
- [Creating Geometry](#)
- [Applying Textures to the Geometry](#)
- [Export Options](#)

Installation

The `\Development\tools\plugins\MayaWorldExport.mll` plug-in file must be placed within a plug-in path used by Maya. The `\Development\tools\plugins\LithTechWorldExportOptions.mel` exporter interface MEL script must be placed within a script path used by Maya. Finally, the plug-in must be loaded using the Maya Plug-in Manager dialog.

Texture Setup

This procedure is the same as in the 3D Studio MAX world exporter. See the [3D Studio MAX Brush Exporter](#) section for more information.

[Top](#)

Creating Geometry

The exporter only supports polygon geometry. NURBS and other procedural geometry will not export at all. If you want to model using other geometry types, be sure to convert them to polygonal meshes before exporting them. Maya scene hierarchy and grouping will be correctly represented in the exported world.

[Top](#)

Applying Textures to the Geometry

Once you have the corresponding texture directory set up, you can begin texturing your terrain in Maya. Apply texture coordinates to the vertices using whatever tools you want.

Texture coordinates alone aren't enough information for the exporter to generate correct mappings in the LTA. You must also apply textures of the correct resolution to the faces of the mesh. This should be done by creating a material with a File texture assigned to the color channel. Place one of the converted textures into the file texture. Any faces without correct mapping information will generate simple default mappings.

[Top](#)

Export Options

The easiest way to use the exporter is to use the **File>Export All** and **File>Export Selection** menu options in Maya. These launch a dialog with all the options supported by the exporter.

Scale

The exported scene will be scaled by the amount entered in this edit box. Defaults to 1.0.

Export as Detail Brushes

All brushes in the exported scene will have their `DetailLevel` property set or not, according to the value in this checkbox. Defaults to ON.

Triangulate Brushes

Forces triangulation of all faces on export. Best used as a debugging option, when you encounter concave faces. You can triangulate concave faces you encounter using Maya's tools.

Export Animated WorldModels

Animated objects in MAX are saved to the .LTA file as animated worldmodels. KeyFramer and Key objects will be placed as needed in the .LTA file. Any object with set keys in MAX will become it's own WorldModel, along with its children. If any of the children are also animated, they will become separate WorldModels, and so on. Time settings in MAX will be used for determining the time values for Key objects in the .LTA file.

Base Texture Path

This path is stripped off the front of the full path of all textures that are exported. For example, if your LithTech world has a texture `d:\project\d3drez\textures\wood\pine1.dtx` and you have an equivalent Maya texture at `c:\Maya Scenes\textures\wood\pine1.tga`, you would enter `c:\Maya Scenes` as the base texture path. This will result in `textures\wood\pine1.dtx` being stored in the .LTA file.

Exported brushes must follow the same rules for valid brushes as always. Concave faces are not valid and will cause the same processing issues as invalid brushes created in DEdit.

Texture support on brushes works the same as it does in DEdit. Only planar projections will work for brush faces with more than 3 vertices. Texture coordinate editing tools can still be used, but each face must have a planar mapping across the entire face in order to export correctly.

Exporter MEL Command

Optionally, the MEL command `lithtechWorldExport` can be used to export the scene. This has the same options as the UI front-end.

```
lithtechWorldExport [-all]
                    [-asBrushes]
                    [-scale <int value>]
                    [-detailLevel <float value>]
                    [-useTexInfo <base path>]
                    filename
```

[Top](#)

Shortcut Text	Internet Address
	http://www.touchdownentertainment.com/jupiter.htm
About Tools and Plugins	file:///C:/Users/khara/AppData/Local/Temp/TAbout.htm
Model and Animation Exporters	file:///C:/Users/khara/AppData/Local/Temp/Exportr.htm
Vertex-Animated Models	file:///C:/Users/khara/AppData/Local/Temp/VertAni.htm
Content Considerations	file:///C:/Users/khara/AppData/Local/Temp/ContCons.htm
Quick Start: Viewing your Model in Jupiter	file:///C:/Users/khara/AppData/Local/Temp/ModView.htm
Types of Geometry	file:///C:/Users/khara/AppData/Local/Temp/Geomty.htm
LithTech Resource Types	file:///C:/Users/khara/AppData/Local/Temp/RescType.htm
Tutorial: Adding an Attachment to your Model	file:///C:/Users/khara/AppData/Local/Tutorials/TAddAtch.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\mTools.htm
Primary Editing Tools	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Primary
Additional Editing Tools	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Additional
Plugins	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Plugins
Top	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#top
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\TAbout.htm
Install the Model Exporter	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StepOne
Export your model	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StepTwo
Convert the .LTA File to an .LTB File	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StepThree
Copying Your Resource Files to the Project Folder	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StepFour
Convert your textures in DEdit	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StepFive
Place the Model in the World Using DEdit	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StepSix
Process and run the world through DEdit	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StepSeven
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\ModView.htm
3D Studio Max	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#3DStudioMax
Maya	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Maya
Exporting Models from Maya	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ExporterDialog
Maya Exporter MEL Commands	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ExporterMELCommand
3D Studio MAX Brush Exporter	file:///C:/Users/khara/AppData/Local/MaxExp/MaxExp.htm

Shortcut Text	Internet Address
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\Exportr.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\VertAni.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\ContCons.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\RescType.htm
Models	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Models
WorldModels	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#WorldModels
World Geometry	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#WorldGeometry
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ToolOvr\Geomty.htm
About Dedit	file:///C:/Users/khara/AppData/Local/Temp/AbotDEdt.htm
Basic DEdit Terminology	file:///C:/Users/khara/AppData/Local/Temp/Terms.htm
Working With Brushes	file:///C:/Users/khara/AppData/Local/Temp/WorkWith/WBrush/mBrushz.htm
Working With Geometry	file:///C:/Users/khara/AppData/Local/Temp/WorkWith/WGeom/mGeom.htm
Working With Objects	file:///C:/Users/khara/AppData/Local/Temp/WorkWith/WObj/mObj.htm
Working With Textures	file:///C:/Users/khara/AppData/Local/Temp/WorkWith/WTextrz/mTextrz.htm
Using the Camera	file:///C:/Users/khara/AppData/Local/Temp/Camera.htm
Adding Simulation Objects	file:///C:/Users/khara/AppData/Local/Temp/SimObj.htm
Creating World Models	file:///C:/Users/khara/AppData/Local/Temp/WorldMod.htm
Lighting the Map	file:///C:/Users/khara/AppData/Local/Temp/Lighting.htm
Organizing World Components	file:///C:/Users/khara/AppData/Local/Temp/NodeView.htm
Optimizing Your Level	file:///C:/Users/khara/AppData/Local/Temp/OptLvl.htm
Tutorial: Building Your First World	file:///C:/Users/khara/AppData/Local/Temp/TBldWrld.htm
Tutorial: Using Brushes	file:///C:/Users/khara/AppData/Local/Temp/Brushes.htm
Tutorial: Adding Sky to Your Level	file:///C:/Users/khara/AppData/Local/Temp/TAddSky.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\mDEdit.htm
UI Interface Components	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UIIntefaceComponents
Understanding the Toolbars	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UnderstandingtheToolbars
Using the Viewports	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingtheViewports
Moving the Camera	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MovingtheCamera
Send feedback regarding this	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\AbotDEdt.htm

Shortcut Text	Internet Address
page.	
Editing Shortcut Keys	file:///C:/Users/khara/AppData/Local/Temp/EditKeys.htm
DEdit Shortcut Keys Long List	file:///C:/Users/khara/AppData/Local/DEditKeys.pdf
DEdit Shortcut Keys Short List	file:///C:/Users/khara/AppData/Local/DEditKeysNoDescT.pdf
Brush Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#BrushKeys
Create Box	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreateBox
Create Cylinder	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreateCylinder
Create Dome	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreateDome
Create Pyramid	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreatePyramid
Create Sphere	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreateSphere
Draw Poly	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DrawPoly
Draw Poly.Insert Edge	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DrawPolyInsertEdge
Draw Poly.Undo	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DrawPolyUndo
Extrude Poly	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ExtrudePoly
Extrude Poly.Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ExtrudePolyBrush
Extrude Poly.Snap	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ExtrudePolySnap
Move Immediate Vert	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MoveImmediateVert
Nudge Down	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NudgeDown
Nudge Left	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NudgeLeft
Nudge Right	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NudgeRight
Nudge Up	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NudgeUp
Select Brush Color	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SelectBrushColor
Triangulate Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TriangulateBrushes
Camera, Navigation, and View Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CameraKeys
Lock Focus	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#LockFocus
Maximize Active View	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MaximizeActiveView
Maximize View 1	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MaximizeView1
Maximize View 2	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MaximizeView2
Maximize View 3	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MaximizeView3
Maximize View 4	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MaximizeView4
Nav Move	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavMove
Nav Move Fast	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavMoveFast
Nav Move Perp	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavMovePerp
Nav Orbit	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavOrbit
Nav Orbit Move	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavOrbitMov
Nav Rotate	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavRotate
Nav Rotate Forward	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavRotateBackward
Nav Rotate Backward	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavRotateForward
Navigator Next	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavigatorNext
Navigator Previous	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavigatorPrevious

Shortcut Text	Internet Address
Reset Viewports	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ResetViewports
Zoom.Fast	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ZoomFast
Editing Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#EditingKeys
Brush Size	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#BrushSize
Carve Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CarveBrush
Copy Selection	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CopySelection
Delete	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Delete
Delete Edges	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DeleteEdges
Delete Tagged Polies	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DeleteTaggedPolies
Edit Curve	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#EditCurve
Far Distance	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#FarDistance
Flip Normal	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#FlipNormal
Freeze Selection	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#FreezeSelection
Group Selection	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#GroupSelection
Join Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#JoinBrushes
Move Node Handle	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MoveNodeHandle
Move Node Perp	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MoveNodePerp
Move Node Snap	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MoveNodeSnap
Move Poly Vert	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MovePolyVert
Move Selected Vert	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MoveSelectedVert
Node Move.Clone	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NodeMoveClone
Paste in Place	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#PasteInPlace
Paste to Marker	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#PasteToMarker
Redo	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Redo
Remove Edges	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#RemoveEdges
Rotate Node	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#RotateNode
Rotate Node Step	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#RotateNodeStep
Rotate Selection Left	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#RotateSelectionLeft
Rotate Selection Right	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#RotateSelectionRight
Scale Immediate Vert	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ScaleImmediateVert
Scale Poly	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ScalePoly
Scale Poly Divide	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ScalePolyDivide
Scale Poly Extrude	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ScalePolyExtrude
Split Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SplitBrush
Split Edge	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SplitEdge
Undo	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Undo
Unjoin Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UnjoinBrushes
Grid Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#GridKeys
Expand Grid Spacing	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ExpandGridSpacing
Grid Size	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#GridSize
Grid To Poly	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#GridToPoly
Set Grid Dir Forward	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SetGridDirForward
Set Grid Dir Right	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SetGridDirRight
Set Grid Dir Up	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SetGridDirUp
Shrink Grid Spacing	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ShrinkGridSpacing

Shortcut Text	Internet Address
Hide/Show Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#HideShow
Hide Inverse	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#HideInverse
Hide Selected	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#HideSelected
Toggle Class Icons	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ToggleClassIcons
Unhide Inverse	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UnhideInverse
Unhide Selected	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UnhideSelected
Marker Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MarkerKeys
Center Marker	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CenterMarker
Move Marker	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MoveMarker
Mode Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ModeKeys
Brush Edit Mode	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#BrushEditMode
Geometry Edit Mode	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#GeometryEditMode
Object Edit Mode	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ObjectEditMode
Object Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ObjectKeys
Add Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AddObject
Object Size	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ObjectSize
Selection Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SelectionKeys
Advanced Select	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AdvancedSelect
Select Depth	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SelectDepth
Select None	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SelectNone
Tag All	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TagAll
Tag Poly Verts	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TagPolyVerts
Tag.Add	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TagAdd
Tag.Invert	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TagInvert
Texture Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureKeys
Apply Texture Click	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ApplyTextureClick
Apply Texture To Sel	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ApplyTextureToSel
Fit Texture To Poly	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#FitTextureToPoly
Init Texture Space	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InitTextureSpace
Map Texture Coords	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MapTextureCoords
Remove Textures	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#RemoveTextures
Texture Move	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureMove
Texture Move.Rotate	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureMoveRotate
Texture Move.Scale	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureMoveScale
Texture Move.Step	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureMoveStep
Splitting Brushes	file:///C:/Users/khara/AppData/Local/Temp/WorkWith/WBrush/mBrushz.htm#SplittingaBrush
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit/Keys.htm
DEdit Shortcut Keys	file:///C:/Users/khara/AppData/Local/Temp/Keys.htm
Navigating the Current Key Configuration Dialog	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NavigatingtheCurrentKeyConfigurationDialog
Changing or Adding a Shortcut Key	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ChangingorAddingaShortcutKey

Shortcut Text	Internet Address
Clearing and Reseting Shortcut Keys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ResetingandRemovingShortcutKeys
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit/EditKeys.htm
Appendix G: Glossary	file:///C:/Users/khara/AppData/Local/Appendix/Append-G/Glossary.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit/Terms.htm
Working With Properties	file:///C:/Users/khara/AppData/Local/WPropts/mPropts.htm
Working With Textures	file:///C:/Users/khara/AppData/Local/WTextrz/mTextrz.htm
About Grid Spacing	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AboutGridSpacing
About Brush Editing	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AboutBrushEditing
About Brush Properties	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AboutBrushProperties
Drawing a Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DrawingaBrush
Inserting a Brush Primitive	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaPrimitiveBrush
Sizing a Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SizingaBrush
Moving a Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MovingaBrush
Carving a Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CarvingaBrush
Splitting a Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SplittingaBrush
Rotating a Brush	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#RotatingaBrush
Grouping Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#GroupingBrushes
Setting Normal Visibility	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SettingNormalVisibility
Working With Geometry	file:///C:/Users/khara/AppData/Local/WGeom/mGeom.htm
Setting Normal Visibility Direction	file:///C:/Users/khara/AppData/Local/WGeom/mGeom.htm#SettingNormalVisibilityDirection
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit/WorkWith\WBrush\mBrushz.htm
About Grid Spacing	file:///C:/Users/khara/AppData/Local/WBrush/mBrushz.htm#AboutGridSpacing
Working With Brushes	file:///C:/Users/khara/AppData/Local/WBrush/mBrushz.htm
About Geometry Editing	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AboutGeometryMode
Drawing a Polygon	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DrawingaPolygon
Moving Vertices	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MovingVertices
Deleting Faces	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#DeletingFaces
Setting Normal Visibility Direction	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SettingNormalVisibilityDirection
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit/WorkWith\WGeom\mGeom.htm
General Steps to KeyFraming	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#GeneralStepstoKeyFraming
KeyFraming a WorldModel	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#KeyFramingaWorldModel
Modifying a Key	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ModifyingaKeyPath

Shortcut Text	Internet Address
Path	
Orienting a WorldModel	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#OrientingaWorldModel
Using KeyFramer Commands	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingKeyFramerCommands
Troubleshooting KeyFrame Animations	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TroubleshootingKeyFrames
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\WorkWith\WKeyFrm\mKeyFrm.htm
About the Nodes Tab	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AbouttheNodesTab
Adding a New Container	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AddingaNewContainer
Moving Selections to a Container	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MovingSelectionstoContainer
Hiding and Unhiding Selections	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#HiddingandUnhidingSelections
Freezing and Unfreezing Selections	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#FreezingandUnfreezingSelections
Naming and Renaming Objects	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NamingandRenamingObjects
Naming Groups of Objects	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NamesforGroupsofObjects
Using Containers to Organize Your Level	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingContainerstoOrganizeaLevel
Adding Simulation Objects	file:///C:/Users/khara/AppData/Local/Temp/SimObj.htm#BindingSimulationObjectstoBrushes
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\WorkWith\WNodes\WNodes.htm
About Jupiter Objects	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AboutJupiterObjects
Using Object Commands	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingObjectMessages
Inserting an Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertinganObject
Inserting a Light Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaLightObject
Inserting a LightGroup Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaLightGroupObject
Inserting a Decal Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaDecalObject
Inserting a DirLight Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaDirLightObject
Inserting a Polygrid Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaPolygrid
Inserting a GameStartPoint Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaGameStartPoint
Inserting a VolumeBrush Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaVolumeBrush
Inserting a Spawner Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaSpawnerObject

Shortcut Text	Internet Address
Inserting a SpecialFX Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaSpecialFX
Inserting a SoundFX Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaSoundFXObject
Inserting a WorldProperties Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaWorldPropertiesObject
Inserting a PlayerTrigger Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaPlayerTriggerObject
Inserting a ScaleSprite Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaScaleSpriteObject
Inserting a TeleportPoint Object	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#InsertingaTeleportPointObject
Programming Guide	file:///C:/Users/khara/ProgGuid/mPG.htm
Working With Sprites	file:///C:/Users/khara/AppData/Local/WSprites/mSprites.htm
Using FxEd	file:///C:/Users/khara/FXEd/mFX.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\WorkWith\WObj\mObj.htm
Using Commands With Prefabs	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingCommandsWithPrefabs
Creating a Prefab	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreatingaPrefab
Modifying a Prefab	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ModifyingaPrefab
Using NOLF 2 Prefabs	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingNOLF2Prefabs
Using the Sample Lighting Prefabs	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingtheSampleLightPrefab
Inserting a LightGroup Object	file:///C:/LithtechWork/Jupiter/JupiterToolsDWSite/Dedit/WorkWith/WObj/mObj.htm#InsertingaLightGroupObject
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\WorkWith\WPrfabz\mPrfabz.htm
Inserting a ScaleSprite Object	file:///C:/Users/khara/AppData/Local/WObj/mObj.htm#InsertingaScaleSpriteObject
Working With Objects	file:///C:/Users/khara/AppData/Local/WObj/mObj.htm
Creating a Sprite	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreatingaSprite
Applying a Sprite	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ApplyingaSprite
Editing a Sprite	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#EditingaSprite
Importing a Texture	file:///C:/Users/khara/AppData/Local/WTextrz/mTextrz.htm#ImportingaTexture
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\WorkWith\WSprites\mSprites.htm
Creating a Texture	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreatingaTexture
Importing a Texture	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ImportingaTexture
Applying Textures	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ApplyingTextures
Adjusting Textures	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AdjustingTextures
Using an Alpha	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsinganAlphaMask

Shortcut Text	Internet Address
Mask	
Changing or Adding a Shortcut Key	file:///C:/Users/khara/AppData/EditKeys.htm#ChangingorAddingaShortcutKey
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\WorkWith\WTextrz\mTextrz.htm
Aligning Textures	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AligningTextures
Texture Alignment Hotkeys	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureAlignmentHotkeys
Assigning Properties to Textures	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AssigningPropertiestoTextures
Assigning Attributes to Textures	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AttachingAttributestoTextures
Working With Attributes	file:///C:/Users/khara/AppData/Local/ProgGuid/Prog/Attrib/Attribs.htm
Programming Guide	file:///C:/Users/khara/AppData/Local/ProgGuid/mPG.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\TextAln.htm
Using the Viewports	file:///C:/Users/khara/AppData/Local/Temp/TBldWrld.htm#UsingtheViewports
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\Camera.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\SimObj.htm
Volume Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#VolumeBrushes
Creation	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Creation
Damage	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Damage
Attachments	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Attachments
States	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#States
Messages	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Messages
Options	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Options
Commands	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Commands
Sounds	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Sounds
Animated Lightmaps	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AnimatedLightmaps
Appendix E: WorldModel Reference	file:///C:/Users/khara/AppData/Local/Appendix/Apnd-E/WorldMod.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\WorldMod.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\Lighting.htm
Polygon Counts	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#PolygonCounts
Measuring Polygon Counts	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MeasuringPolgyonCounts
Lightmapping	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Lightmapping
Complex Geometry	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ComplexGeometry
Large Levels	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#LargeLevels
Efficient Visibility	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#EfficientVisibility

Shortcut Text	Internet Address
Blocking Visibility	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#BlockingVisibility
Visibility Control Strategy	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#VisibilityControlStrategy
Wireframe	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Wireframe
Overdraw	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Overdraw
Texture Concerns	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureConcerns
Simulation Object Concerns	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SimulationObjectConcerns
Optimization Warning Signs	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#OptimizationWarningSigns
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\OptLvl.htm
Creating a New World	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#create
Making a Simple Room	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MakeRoom
Adding Objects	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AddingObjects
Processing the World	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ProcessingtheWorld
Testing the World	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TestingTheWorld
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\TBldWrld.htm
Starting the Tutorial	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#StartingtheTutorial
Adjusting the Grid	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AdjustingtheGrid
Creating the Floor	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreatingtheFloor
Making Complex Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#MakingComplexBrushes
Setting Markers	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SettingMarkers
Selecting Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SelectingBrushes
Manipulating Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ManipulatingBrushes
Using Brush Power Tools	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UsingBrushPowerTools
Understanding Invalid Brush Types	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#UnderstandingInvalidBrushTypes
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\Brushes.htm
Adding SkyPortals	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AddSomeSkyPortals
Adding SkyPortal Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AddingSkyPortalbrushes
Adding Clouds	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#_Toc534778186
Moving Clouds	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#_Toc534778187
Index Ordering Multiple Sky Objects	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#_Toc534778189
Adding Fog	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#_Toc534778190
wasabisoft.com	http://www.wasabisoft.com/
Send feedback regarding this	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Dedit\TAddSky.htm

Shortcut Text	Internet Address
page.	
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Level_Processing.htm
Understanding ModelEdit	file:///C:/Users/khara/AppData/Local/Temp/ModEdUI.htm
Controlling Level of Detail	file:///C:/Users/khara/AppData/Local/Temp/LOD.htm
Adding Animations	file:///C:/Users/khara/AppData/Local/Temp/Animat.htm
Editing Animations	file:///C:/Users/khara/AppData/Local/Temp/EditAnim.htm
Animation Blending and Weighting	file:///C:/Users/khara/AppData/Local/Temp/AniBlend.htm
Adding Sockets	file:///C:/Users/khara/AppData/Local/Temp/Socket.htm
Jupiter Model Issues	file:///C:/Users/khara/AppData/Local/Temp/Modlisuz.htm
Creating a Character	file:///C:/Users/khara/AppData/Local/Temp/CharCrea.htm
Setting up a Player View Weapon	file:///C:/Users/khara/AppData/Local/Temp/PlayView.htm
ModelEdit UI Reference	file:///C:/Users/khara/AppData/Local/Temp/UIRef.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\mModEd.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\ModEdUI.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\LOD.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\Animat.htm
Send feedback regarding this page.	EditAnim.htm">mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit>EditAnim.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\AniBlend.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\Socket.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\Modlisuz.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\CharCrea.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\PlayView.htm
File Menu	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#FileMenu
Model Menu	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ModelMenu
Animation Menu	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AnimationMenu
Child Model Menu	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ChildModelMenu
Sockets Menu	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#SocketsMenu
Piece Menu	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#PieceMenu
Options Menu	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#OptionsMenu

Shortcut Text	Internet Address
Model View Window	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ModelViewMenu
The Animation Bar	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TheAnimationBar
Animation Edit	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AnimationEdit
Transform Edit	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TransformEdit
Nodes List	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#NodesList
Animations List	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#AnimationsList
Command String	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CommandString
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ModlEdit\UIRef.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: FXed_Reference.htm
Installation	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Intall
Texture Setup	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#TextureSetup
Creating Level Geometry	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreateLevelGeometry
Applying Textures to Brushes	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ApplyTexturesToBrushes
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: MaxExp\MaxExp.htm
Installation	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#Installation
Importer Options	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ImporterOptions
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: MaxLTA\MaxLTA.htm
Creating Geometry	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#CreateGeometry
Applying Textures to the Geometry	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ApplyTextures
Export Options	file:///C:/Users/khara/AppData/Local/Temp/~hhF93A.htm#ExportingOptions
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: MayaExp\MayaExp.htm